Maven

Table des matières

I - Maven (présentation, positionnement).....4
1. Présentation de Maven.....4
2. Principales fonctionnalités de maven.....4
3. Intérêts de Maven.....5
4. Evolutions, versions.....6
5. gradle = concurrent sérieux de maven.....7
6. Fonctionnement de maven.....7

II - Maven (installation, utilisation).....10
1. Variables d'environnement à bien régler.....10
2. Mise en oeuvre de Maven.....11

III - Maven "Goals".....13
Buts (goals) et phases.....13
2. Quelques options importantes.....17

IV - Fichiers "P.O.M." (structures , ...).....18
1. POM (Project Object Model).....18
2. Structure & syntaxes (pom.xml).....22
3. Configuration multi-modules (avec sous projet(s)).....26
4. Héritage entre projets "maven" (<parent>).....29

5. Archetypes....32
6. Utilisation d'un (nouvel) archetype maven....32
7. Création d'un nouvel archetype maven....33

V - Test unitaire via maven....34
1. Tests unitaires avec maven....34

VI - Liaison avec eclipse (m2e)....38
1. Lien entre maven et eclipse (m2e)....38
2. Lien entre maven et IntelliJ....44
3. Lien entre maven de visual studio code....45

VII - Configurations (référentiels , profils, ...)....46
1. Mise en place d'un référentiel "Maven"....46
2. Repository Manager (Nexus ou ...)....51
3. Profils "maven"....56
4. Filtrage des ressources....60
5. Ajout (et éventuel filtrage) de ressources "web" externes....61

VIII - Génération de documentation (mvn)....63
1. Génération et publication d'une documentation....63

IX - Rapports (et javadoc) avec maven....65
1. Javadoc (via maven)....65
2. Rapports avec maven....65

X - Plugins pour maven (exécution, prog.)....66
1. Plugins pour maven....66
2. Programmation d'un plugin pour maven....69

XI - GConf. maven (lien avec SVN, ...)....73
1. Plugins "scm" et "release" de maven....73

XII - Maven et intégration continue....77
1. Intégration continue....77
2. Maven et l'intégration continue....78

XIII - Annexe – variables "maven" et versions....80

1. Variables et versions (maven)....80
2. Bonnes pratiques....82

XIV - Annexe – Plugin "cargo"....83
1. Déploiement Jee avec CARGO (via maven)....83

XV - Annexe – "B.O.M."....86
1. Gestion avancée des dépendances (BOM)....86

XVI - Annexe – Hudson - Jenkins....90
1. Premiers pas avec Hudson/Jenkins....90
2. Architecture de Jenkins (variantes)....94
3. Préparation machine hôte pour jenkins avec docker....98
4. Jenkins avec docker en mode "docker in docker"....100
5. "credentials jenkins" pour "push d'image docker"....112
6. Notifications des résultats (rss , email , …)....113
7. Différents types de "builds" (avec Jenklins)....118

XVII - Annexe – Bibliographie, Liens WEB + TP....121
1. Bibliographie et liens vers sites "internet"....121
2. TP....121

I - Maven (présentation, positionnement)

1. Présentation de Maven

Apache Maven est un outil logiciel open source pour la gestion et l'automatisation de production des projets logiciels Java/JEE .

Maven utilise un paradigme connu sous le nom de ***Project Object Model*** **(POM)** afin de décrire un projet logiciel, ses dépendances avec des modules externes et l'ordre à suivre pour sa production. Il est livré avec un grand nombre de tâches pré-définies, comme la compilation de code Java .

Chaque projet ou sous-projet est configuré par un fichier ***`pom.xml`*** à la racine du projet qui contient les informations nécessaires à Maven pour traiter le projet (nom du projet, numéro de version, dépendances vers d'autres projets, bibliothèques nécessaires à la compilation, noms des contributeurs ...).

Maven impose une arborescence et un nommage des fichiers du projet selon le concept de ***Convention plutôt que configuration***. Ces conventions permettent de réduire la configuration des projets, tant qu'un projet suit les conventions. Si un projet a besoin de s'écarter de la convention, le développeur le précise dans la configuration du projet.

Voici une liste non-exhaustive des répertoires d'un projet Maven :

- `/src` : les sources du projet
- `/src/main` : code source et fichiers source principaux
- **`/src/main/java`** : code source
- **`/src/main/resources`** : fichiers de ressources (images, fichiers de configurations pour Spring , hibernate , log4j, ...)
- `/src/test` : fichiers de test
- `/src/test/java` : code source de test
- `/src/test/resources` : fichiers de ressources de test
- `/src/site` : informations sur le projet et/ou les rapports générés (checkstyle,javadoc)
- **`/src/webapp`** : webapp du projet
- `/target` : fichiers résultat, les binaires (du code et des tests), les packages générés et les résultats des tests

La gestion des dépendances au sein de Maven est simplifiée par les notions d'héritage et de **transitivité**. La déclaration de ces dépendances est alors limitée.

Les **buts** (***goals*** en anglais) principaux du cycle de vie d'un projet Maven sont:

- `compile` (compiler ".java" --> ".class")
- **`test`** (lancer tous les tests unitaires (JUnit) de la branche src/test/java)
- **`package`** (construire le ".jar" ou le ".war")
- `install` (installer la chose construite sur le référentiel local)
- **`deploy`** (déployer vers un référentiel distant ou sur un serveur)

NB: lancer "package" déclenche *compile* puis *test* puis *package* .
+

- clean (supprimer les ".jar" et .... générés)
- site (générer la documentation)

2. Principales fonctionnalités de maven

Maven (principales fonctionnalités)

Centré sur la notion de projet (api java, module applicatif)
--> toutes les caractéristiques d'un projet sont déclarées dans un fichier "***pom.xml***" (**P**roject **O**bject **M**odel).

Configuration "déclarative" (basée sur des conventions) plutôt qu'explicite. Pas de commandes et chemins précis à renseigner (contrairement à ANT).

Gestion distribuée (sur le web) des ***dépendances*** inter-projets.
---> identification et ***téléchargement automatique des ".jar"*** nécessaires selon les API déclarées en dépendances .

Gère toutes les phases (compile/build , tests unitaires , packaging , stockage dans le référentiel de .jar , lien avec GIT, ...) .

3. Intérêts de Maven

Avec la très ancienne technologie ***ANT*** , il fallait tout **expliciter** dans les scripts (répertoires , listes des ".jar" du classpath , ...). A l'inverse , la technologie ***Maven*** automatise presque tout de manière **implicite** (en se basant sur des conventions de structuration des projets) .

Dans de nombreux projet Java/JEE on a besoin de faire collaborer/cohabiter tout un tas de technologies "open source" qui:

- proviennent de différents éditeurs (jakarta Apache , Jboss Group , SUN , Spring , ...)
- utilisent en interne différents modules d'intérêt général (commons-logging , ...)
- évoluent régulièrement (nouvelles versions pas toujours compatibles avec tout le reste)

La technologie Maven permet de gérer presque automatiquement tous ces problèmes de dépendances car elle s'appuie en interne sur une sorte de référentiel de produits/technologies qui est régulièrement actualisé sur le site web de référence de maven. [*Remarque*: Après une installation de maven sur un poste de développement, un accès internet est ainsi indispensable pour ré-actualiser certains modules]

Il est possible de configurer un nouveau référentiel maven au sein d'une entreprise pour ne plus dépendre des référentiels externes. Ce référentiel se créer comme une copie partielle du référentiel de référence (avec des actualisations régulières conseillées).

NB: il existe maintenant une extension pour ANT appelée "IVY" qui gère également les dépendances de librairies (".jar") , la syntaxe des fichiers de configuration d'IVY est cependant légèrement différente de celle de "maven". Maven est plus orienté "projet" et va plus loin.

4. Evolutions, versions

Versions de Maven

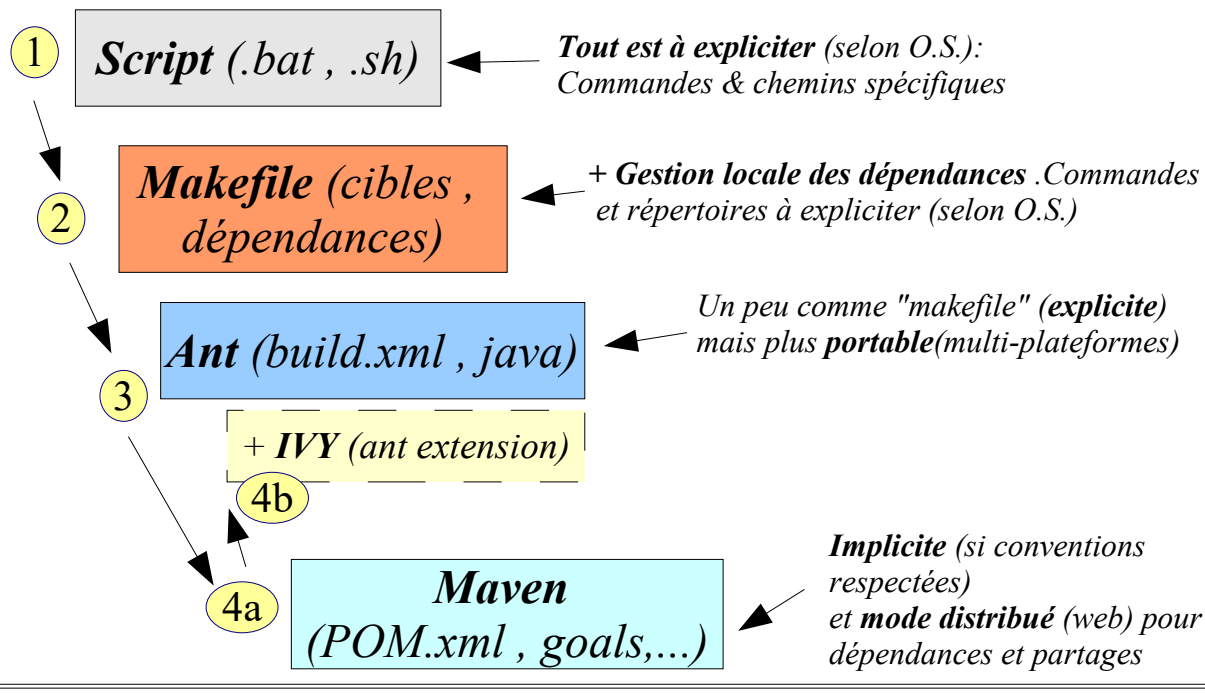
Maven 1 / 2003-2004 (très ancienne version, aujourd'hui obsolète)

Maven 2 / 2005-2009 (beaucoup d'améliorations , changements en profondeur)
→ version mature (avec plein de plugins disponibles)

Maven 3 / 2010-2026 (dans la continuité de la V2)
→ même config que la v2 (syntaxe inchangée , référentiels compatibles)
→ meilleurs performances et quelques améliorations.
→ version 3.10 (encore maintenue) : java 8 minimum

Maven 4 / 2027 ? (release candidate 4 en septembre 2026)
→ meilleurs performances
→ java 17 minimum

évolutions (du script à maven)



5. gradle = concurrent sérieux de maven

Né en 2008 , le produit open source "**gradle**" reprend beaucoup de principes de fonctionnement de maven.

Principales caractéristiques de "gradle" :
- **syntaxe spécifique** (**DSL** (Domain specific Language basé) sur "groovy" et "kotlin" **très compacte** *contrairement à la syntaxe très verbeuse (mais normalisée/standardisée) de maven* .
- cycle de vie plus libre
- **bonnes performances (compilations incrémentales, cache de build , exécutions en // , …)**
- **compatible avec les référentiels de maven ou de ivy**

Exemple de dépendance maven :

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
   <version>4.1.2</version>
   <scope>compile</scope>
</dependency>
```

Dépendance équivalente gradle :

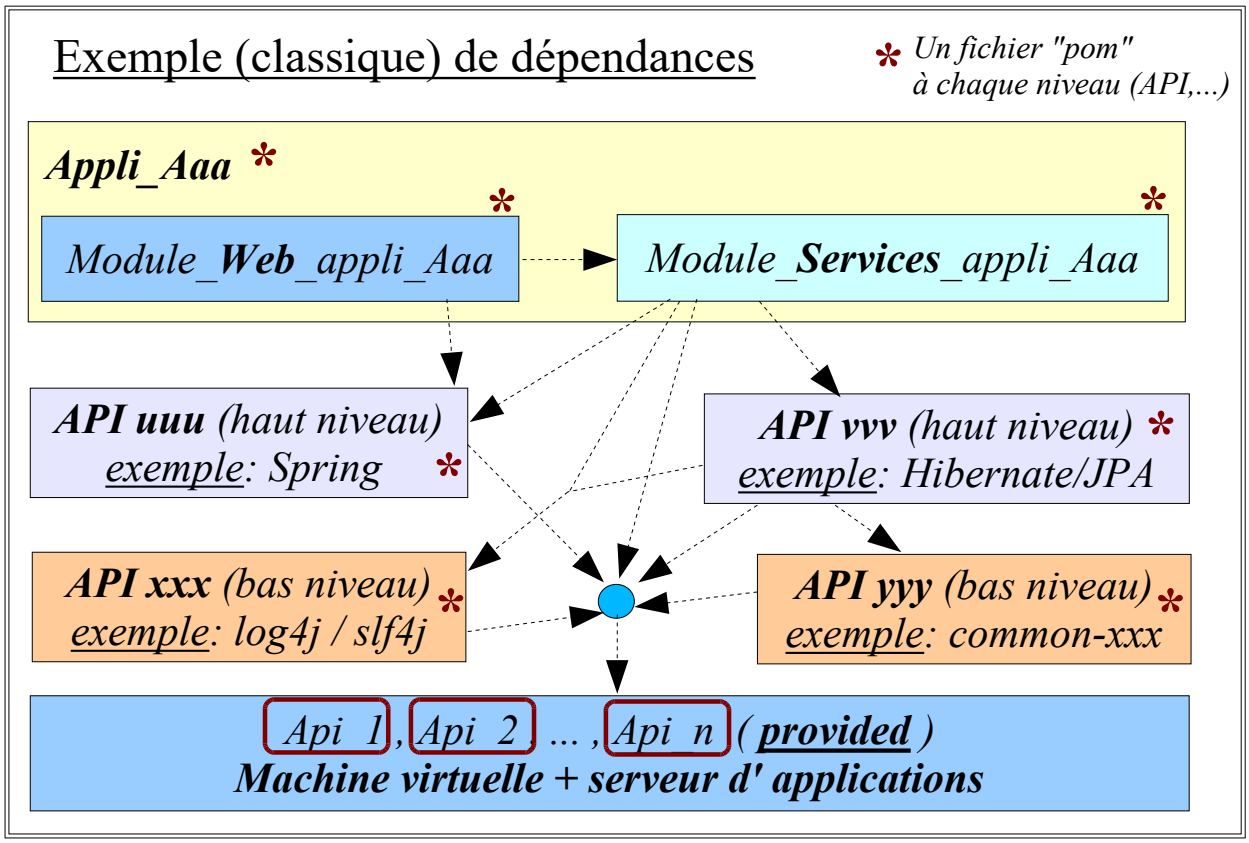
```
// Source: https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-web
implementation("org.springframework.boot:spring-boot-starter-web:4.1.2")
```

Maven est principalement utilisé au niveau du développement d'application android (android-studio) et convient très bien aux frameworks et aux IDEs permettant de gérer du code source en java ou dans des dérivées de java (Kotlin, Scala , Groovy) : SpringBoot , IntelliJ, …

Pour le développement d'applications java traditionnelles (.jar , .war) (backend Jarkarta-ee ou springBoot , ….) la technologie maven reste pour l'instant prédominante et la version 4 sur le point d'être stabilisée apportera un petit gain en performance .

Ceci dit , "maven" et "gradle" sont deux concurrents très sérieux (avec heureusement un passage assez simple de l'un à l'autre en cas de besoin) .

6. Fonctionnement de maven

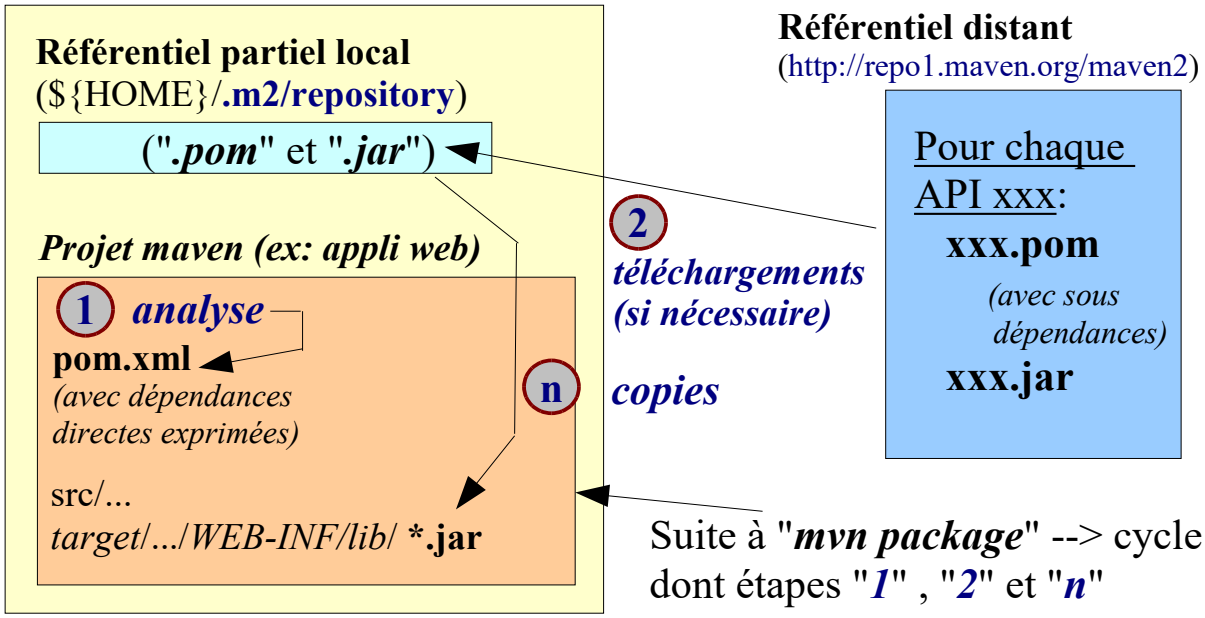


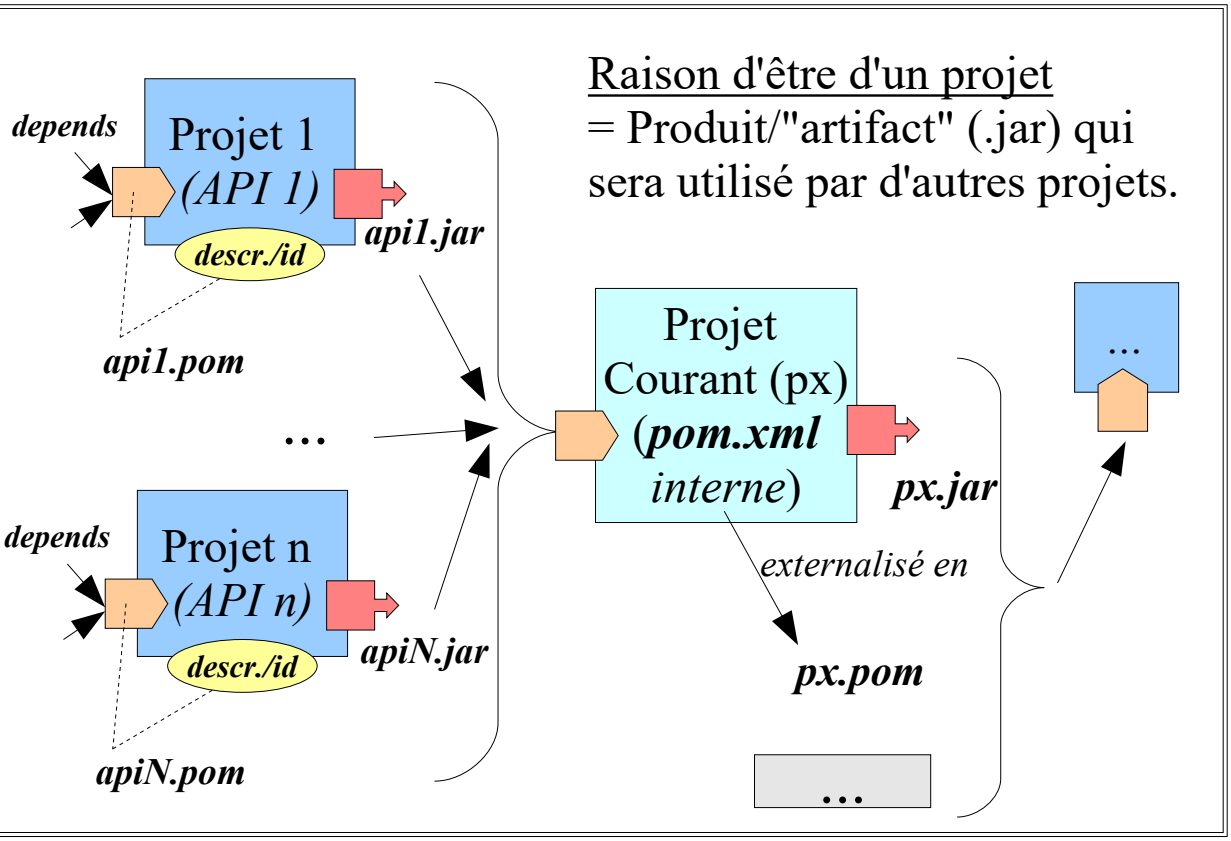
- Une application java/jee est généralement constituée de sous modules (partie principale , sous projets , librairies réutilisables) et toutes ces parties seront packagées comme des ".jar" (ou dérivés) .
- Chaque partie de l'application s'appuie généralement sur un ou plusieurs frameworks de haut niveau (ex : Spring, JPA/Hibernate, …) .
- La plupart des frameworks de haut niveaux dépendent eux mêmes de librairies de bas niveau (ex : slf4j/log4j , …)

⇒ tout ceci forme un graphe de dépendances où chaque ".jar" de haut niveau est le produit ("artifact") d'un projet maintenu par tel ou tel entité/entreprise ("groupId") dans une certaine "version" à partir d'une liste transitive de dépendances (".jar" de plus bas niveau eux mêmes basés sur d'autres projets).

Téléchargement automatique des librairies nécessaires

(avec prise en compte des dépendances indirectes)

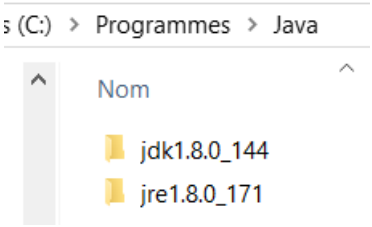




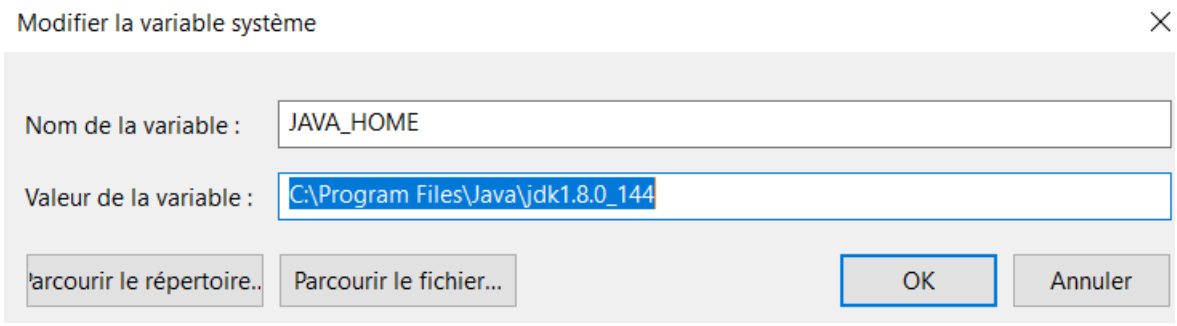
II - Maven (installation, utilisation)

1. Variables d'environnement à bien régler

Maven est une technologie qui s'appuie en interne sur java pour fonctionner. il est donc important de vérifier le bon paramétrage des variables d'environnement **PATH** et **JAVA_HOME** .



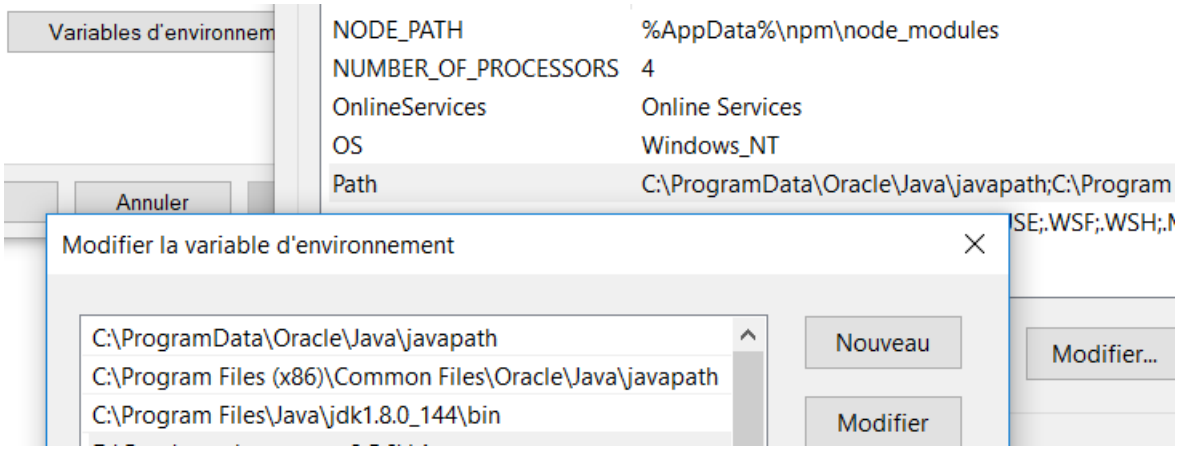
Exemple : **JAVA_HOME**=***C:\Program Files\Java\jdk1.8.0_144***
PATH=.... ;*C:\Program Files\Java\jdk1.8.0_144**bin*** ;....



Sur un ordinateur récent , il est conseillé d'installer une version stable (LTS) de java (**17 ou 21 ou 25 ou ...**)

D'autre part , une fois que maven.3.10....zip aura été recopié et "dezippé/décompressé" sur le poste du développeur, il faudra que le sous répertoire **bin** de ***apache-maven-3.10...*** soit **présent** dans le **PATH** pour que la commande "**mvn**" soit trouvée dans le cadre d'un déclenchement en ligne de commande (depuis une fenêtre "CMD" par exemple sous windows") .

et donc **PATH**=.... ;*C:\Program Files\Java\jdk1.8.0_144**bin*** ;***D:\Prog\apache-maven-3.10.0\bin***;... par exemple .



NB : Selon la version de windows (ex : 10 ou autre) , le paramétrage des variables d'environnement

pourra être effectué depuis une partie du panneau de configuration (que l'on peut souvent rechercher via "**env**") .

2. Mise en oeuvre de Maven

Bien que l'on puisse indirectement utiliser maven via un IDE tel que eclipse ou intelliJ , il est **fortement conseillé d'utiliser directement maven en ligne de commande** pour bien appréhender et comprendre cette technologie.

Un téléchargement de "**apache-maven-3.10.0-bin.zip**" depuis le site officiel https://maven.apache.org/download.cgi suivi d'une extraction du zip et d'une configuration des variables d'environnement (PATH avec …./maven/bin) suffit à bien installer maven sous windows.

Au sein d'un environ linux (ex : debian), l'installation de maven peut généralement s'effectuer via :

```
sudo apt update
sudo apt install maven
```

Tester l'installation de maven

mvn --version
→
Apache Maven 3.10.0 *(c43a36b8d67be7e0805a411bc0898af1a51f5472)*
Maven home: C:\Prog\apache-maven-3.10.0
Java version: 25.0.2*, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-25.0.2*
Default locale: fr_FR, platform encoding: UTF-8, time zone: Europe/Paris
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

Pour faire ses premiers pas avec maven, on peut créer rapidement une nouvelle petite application java en s'appuyant sur un modèle prédéfini d'application élémentaire (***maven-archetype-quickstart /*** "hello-world") .

Ligne de commande pour créer un nouveau projet "maven"

```
mvn archetype:generate  -DarchetypeGroupId=org.apache.maven.archetypes
-DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.5
-DgroupId=com.mycompany.app   -DartifactId=my-app
```

Arborescence de répertoires et fichiers créée :

```
my-app
|-- pom.xml
`-- src
    |-- main
    |   `-- java
    |       `-- com
    |           `-- mycompany
    |               `-- app
    |                   `-- App.java  ( "Hello world !" )
    `-- test
        `-- java
            `-- com
                `-- mycompany
                    `-- app
                        `-- AppTest.java (ok / true)
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany.app</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0-SNAPSHOT</version>
  <!-- <packaging>jar</packaging> -->

  <name>my-app</name>
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
  </properties>

 <dependencyManagement>
    <dependencies>
       <dependency>
          <groupId>org.junit</groupId>
          <artifactId>junit-bom</artifactId>
          <version>5.11.0</version>
          <type>pom</type>
          <scope>import</scope>
    </dependency>
   </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
...
</project>
```

Ligne de commande pour construire le projet:

```
mvn package
```

--> résultat dans "target" (.class et .jar) .

```
java -cp target/my-app-1.0-SNAPSHOT.jar com.mycompany.app.App
```

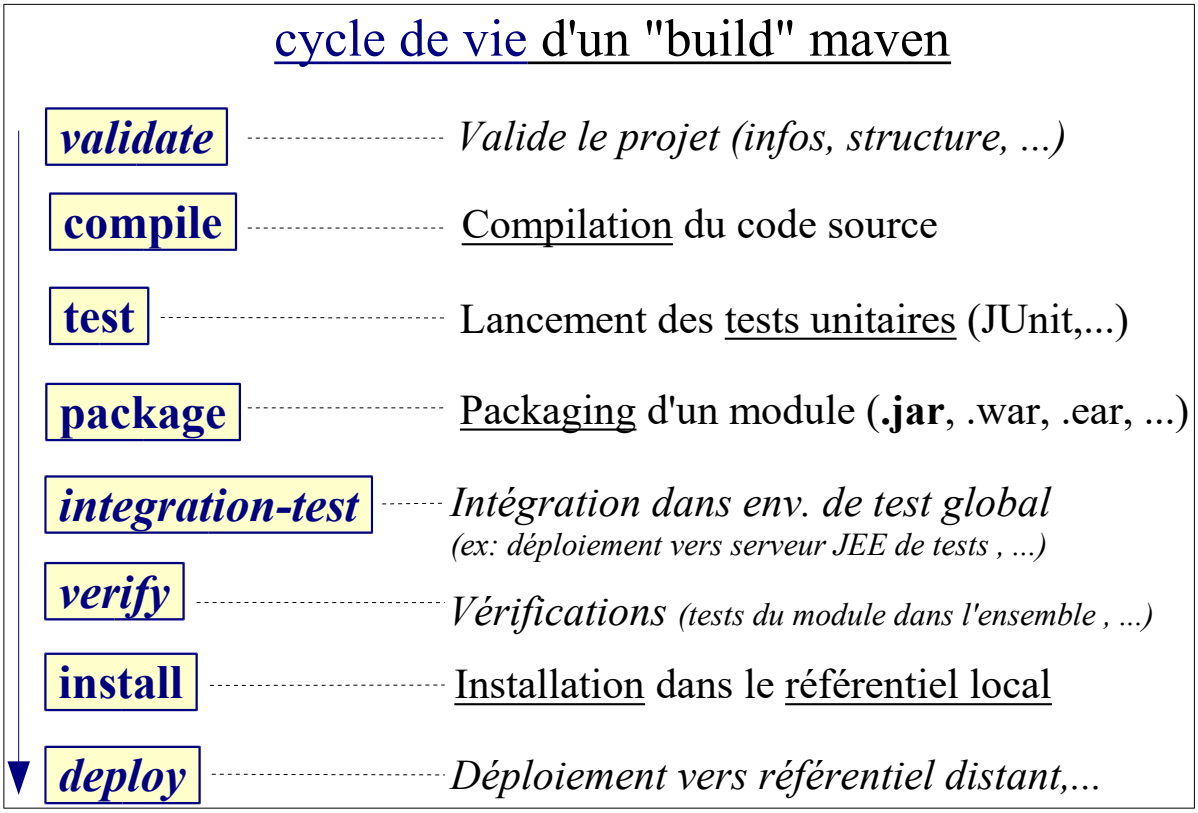
Hello World !

III - Maven "Goals"

Buts (goals) et phases

1.1. "buts/goals" liés au cycle de construction d'un projet maven

Principaux buts (goals):



Phases du cycle de construction

Lorsque l'on déclenche une *ligne de commande* "**mvn <goal>**" (ex: **mvn install**) ,
on ***demande explicitement à atteindre un but*** .

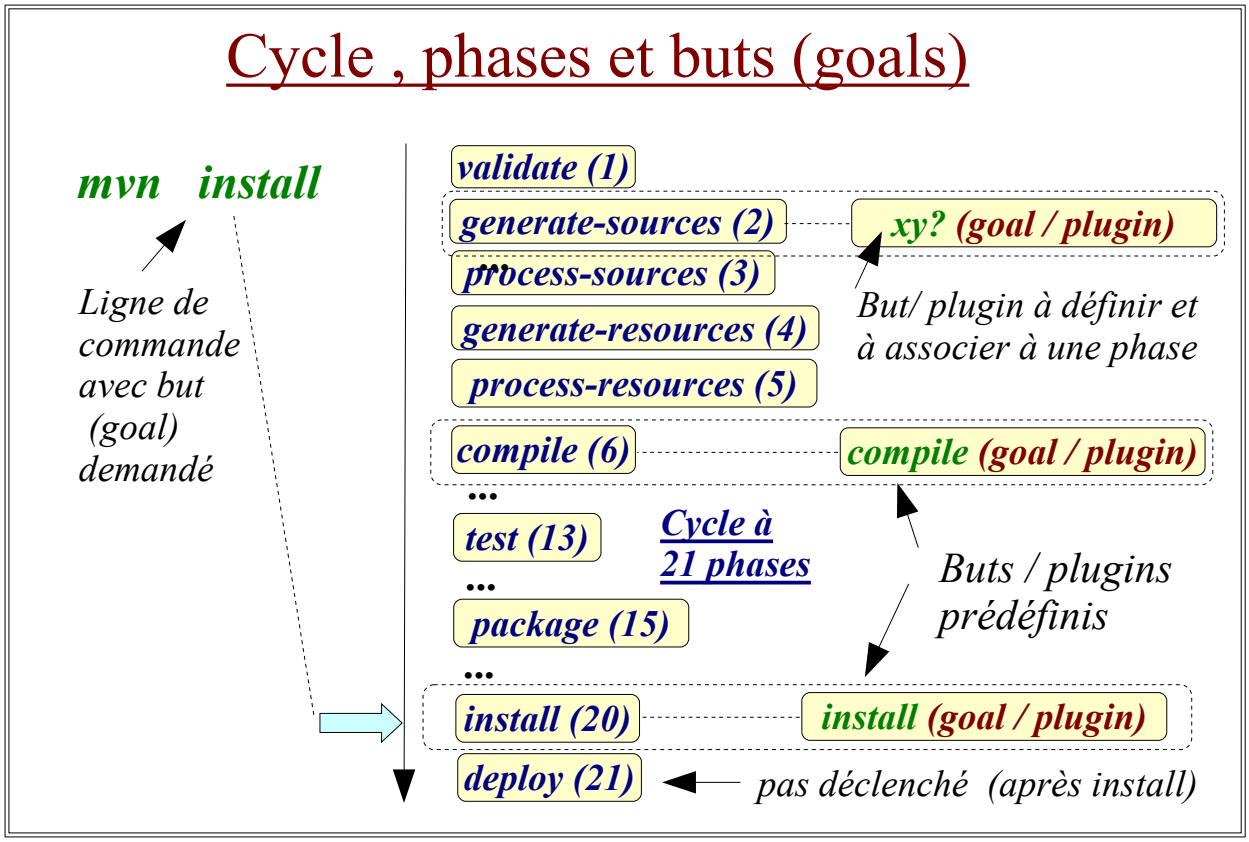
Pour atteindre ce but , maven va déclencher un processus de construction qui comporte au maximum 21 phases (dans la version actuelle).

Ces phases ont des "ordres" et "noms" bien déterminés (ex: validate(1) , generate-sources (2) , ...).

Selon le paramétrage du projet (pom.xml) , les phases de 1 à n-1 seront (ou pas) associées à des plugins (actions / buts préalables) à déclencher.

La demande d'atteinte du but associé à la phase n , déclenche dans l'ordre l'exécution de tous les plugins associés aux phases 1 , ..., n-1 puis n .

lien entre phases et buts (goals):



Phases du cycle de construction par défaut:
Plugins (et goals) prédéfinis et activés lors du cycle par défaut (pour packaging "jar").

Nom de la phase	*plugin:goal*
process-resources	resources:resources
compile	compiler:compile
process-test-resources	resources:testResources
test-compile	compiler:testCompile
test	surefire:test
package	jar:jar (*)
install	install:install
deploy	deploy:deploy

(*) ou war:war , ejb:ejb3 , ear:ear ... selon autre type de packaging du projet .

NB:
- Chaque but (goal) est codé dans un plugin maven (packagé comme un ".jar") .
- Un plugin maven peut contenir plusieurs buts (goals)

[ex: deploy:deploy , deploy:deploy-file , deploy:....]

Liste des 20 à 23 (*) phases du principal cycle de construction "maven" (par défaut):

(*) selon version de maven

validate	Valide si le projet est correcte et que toutes les informations nécessaires sont disponibles.
initialize	initialise la construction (set properties, create directories).
generate-sources	Génération éventuelle de code source à inclure dans la future compilation (ex: xdoclet , apt , ...)
process-sources	Traite le code source code, (ex: filtrage: remplacement de variables par des valeurs selon paramétrage de pom.xml).
generate-resources	génération de ressources (fichiers de configuration ou de données) pour future inclusion dans un package.
process-resources	Copie et traitement des ressources au sein du répertoire destination , prêt pour le packaging.
compile	compile le code source du projet.
process-classes	Traite après coup les fichiers créés par la compilation,par exemple pour enrichir le "bytecode" des classes Java.
generate-test-sources	éventuelle génération de code source spécifique au test .
process-test-sources	Traitement du code source de test, (ex: filtrage).
generate-test-resources	Éventuelle création de ressources pour les tests.
process-test-resources	copie et traite les ressources dans le répertoire de destination pour les tests.
test-compile	compile le code source des tests et place le résultat dans "target/test/...."
process-test-classes	Traite éventuellement après coup les fichiers créés par la compilation des tests,par exemple pour enrichir le "bytecode" des classes Java.
test	Lancement des tests unitaires (via Junit ou autre). Ces tests ne nécessite pas un packaging ni un déploiement du code des tests.
prepare-package	éventuelle préparation du packaging (ajustement de ... , ...)
package	Packaging du code compilé et des ressources (.jar , .war , ...).

pre-integration-test	éventuelle préparation des tests d'intégration (ex: préparer l'environnement d'exécution).
integration-test	Traite et déploie si nécessaire le package dans un environnement d'exécution (ex: serveur JEE , ...) au sein duquel les tests d'intégration peuvent s'exécuter.
post-integration-test	Éventuels post-traitements pour les test d'intégration (ex: "clean" , ...)
verify	Lance d'éventuelles vérifications du package pour vérifier sont intégrité et sa qualité.
install	Installe le package dans le référentiel local pour qu'il puisse être réutilisé depuis d'autre projets maven (du même poste de développement).
deploy	Copie en plus le package créé dans un référentiel partagé de l'entreprise (ex: référentiel géré par archiva).

Les 4 phases du cycle générant la documentation (Site Lifecycle):

pre-site	préparation
site	génération de la documentation du projet (site)
post-site	Finalisation et éventuelle préparation au déploiement
site-deploy	déploiement de " site documentation " vers le serveur web spécifié

À déclencher via ***mvn site*** ou ***mvn site-deploy*** .

1.2. Autres Buts (goals) fondamentaux (clean , ...)

mvn clean ..... pour supprimer ce qui a été (anciennement) généré dans "target" .

Cycle spécifique au "clean":

Clean Lifecycle

pre-clean	Pre....
clean	Supprime tous les fichiers de target (générés via anciens builds)
post-clean	Post....

Autres buts:
selon plugins (et documentation associée)

2. Quelques options importantes

L'option **-U** (alias ***--update-snapshots***) de maven signifie "***force update of snapshots***"
et demande à télécharger une éventuelle "nouvelle version plus récente" d'une dépendance du projet dont la version se termine par "-SNAPSHOT" .

NB : Une version ordinaire (ex : 1.3 ou 1.3-RELEASE) qui ne se termine pas par -SNAPSHOT est considérée par maven comme "définitive" et n'est pas "re-téléchargée" pour rien .

Un artifact (ex : .jar) dont la version se termine par "0.0.1-SNAPSHOT" peut être re-généré (ré-écrasé) en cours de développement par une version améliorée (code différent) ayant pourtant le même numéro de version " 0.0.1-SNAPSHOT" et l'option -U peut alors être utile .

L'option **-C** (ou bien ***--strict-checksums***) permet de demander une vérification des cohérences de "checksums" (entre .jar et .sha1 par exemple) .

Sans cela simple warning (passant souvent inaperçu) , grâce à cela , échec du build .

Dans certains cas (heureusement assez rares) , le réseau informatique est lent (ou un peu défectueux)
et les téléchargements des librairies peuvent être interrompus avant leurs fins normales : il manque alors quelques octets dans un ".jar" mal téléchargé d'une partie du référentiel local (dans un des répertoires de ${HOME}/.m2/repository/ ...).

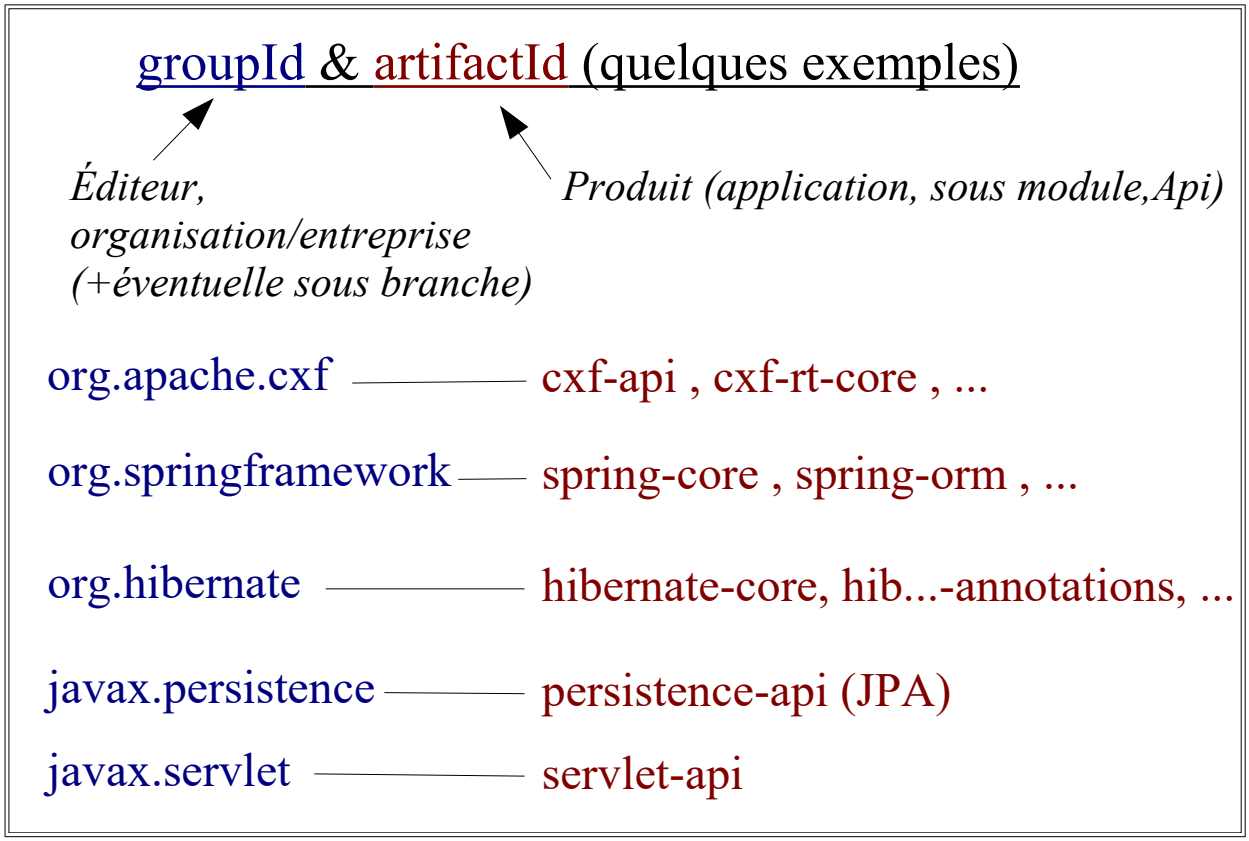
La technologie maven croit alors que le ".jar" est correct (alors qu'il ne l'est pas) . comme le ".jar" est tout de même présent , un re-téléchargement n'est pas retenté en mode "release" dans que le répertoire contenant le ".jar" défectueux n'est pas supprimé .

La commande **mvn dependency:purge-local-repository** (à lancer comme d'habitude depuis le répertoire du projet courant) est prévue pour supprimer et re-télécharger (dans le référentiel local ${HOME}/.m2/repository/) les dépendances (directes et indirectes) du projet courant paramétré par pom.xml .

IV - Fichiers "P.O.M." (structures , ...)

1. POM (Project Object Model)

1.1. GroupId & artifactId



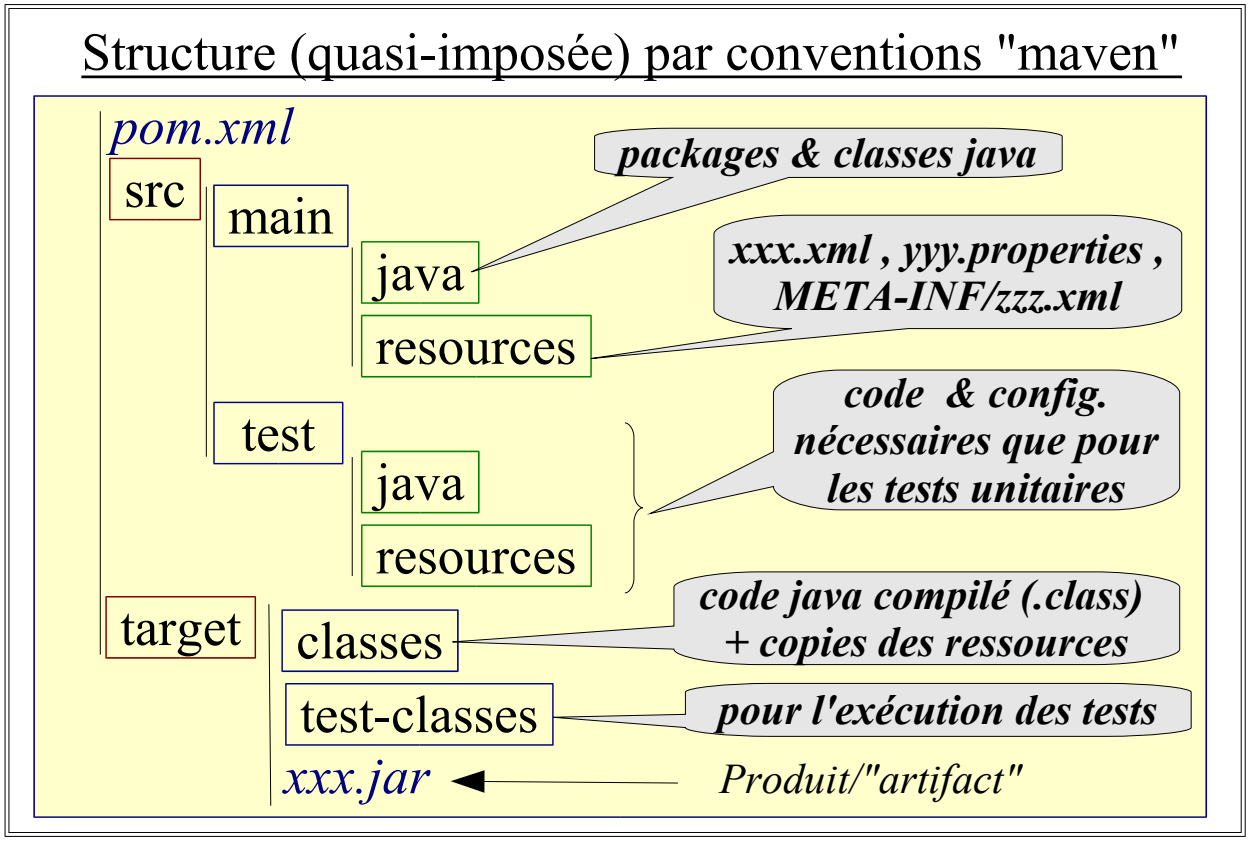
NB (changements historiques) :

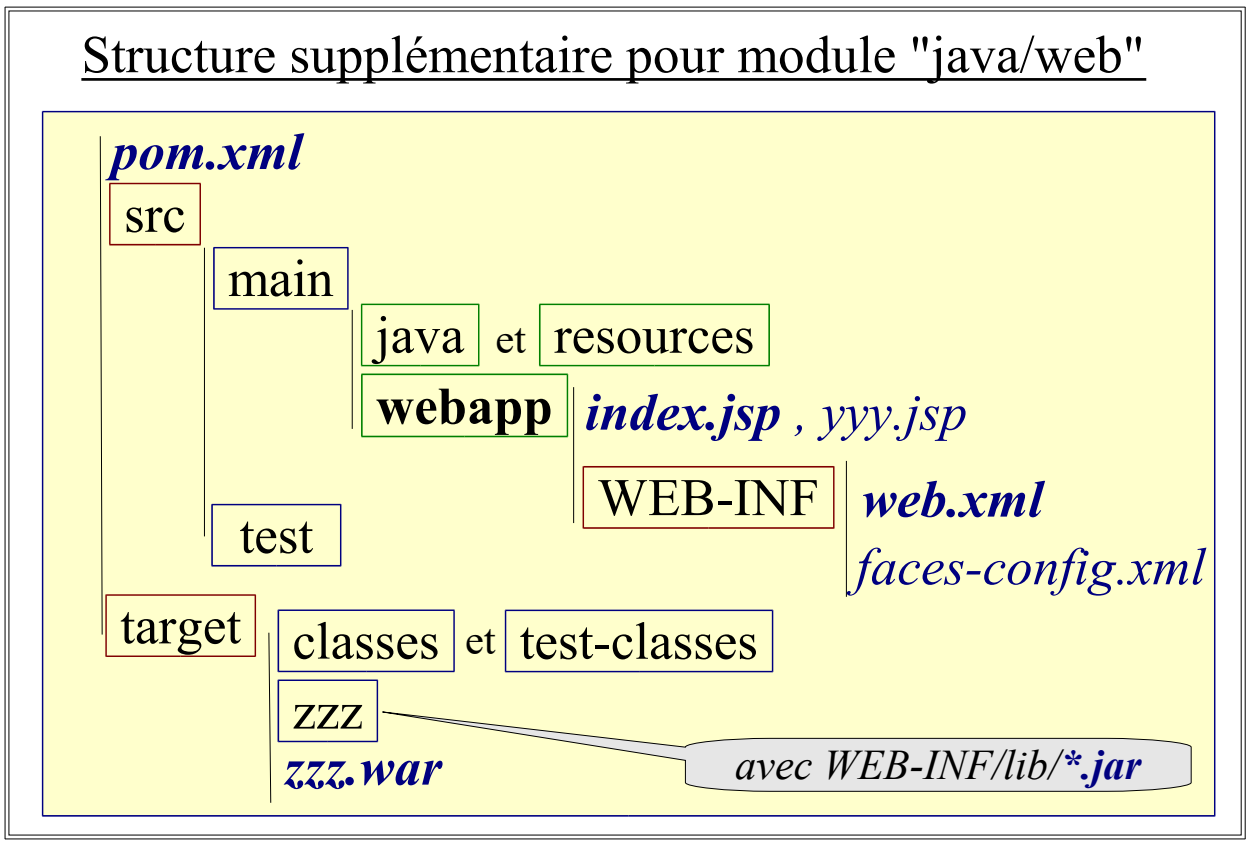
En 2009 , l'entreprise SUN (qui a créé le langage java) a été rachetée par l'entreprise Oracle.

Entre 2018 et 2020 , l'entreprise Oracle a donné la partie JEE (pas rentable) à la communauté open source "jakarta"
Beaucoup de technologies java ont changé de "groupId" a cette époque :

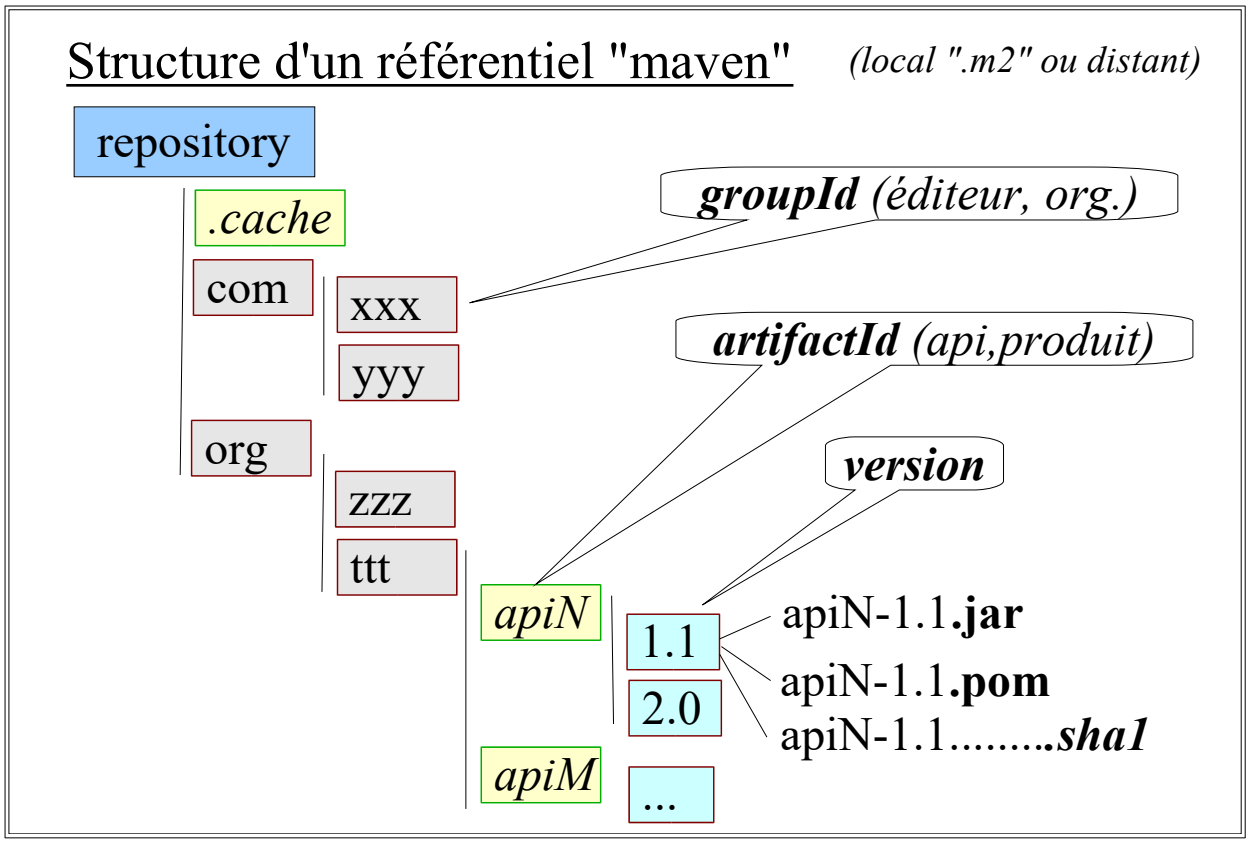
javax.servlet → **jakarta.servlet**
javax.persistence → **jakarta.persistence**
...

1.2. Arborescences conventionnelles





NB : dans le cas particulier d'une application "springBoot" autonome , pas de partie src/main/webapp mais un sous répertoire "static" en dessous de src/main/resources car packaging souvent "jar" plutôt que "war" .



NB : le fichier apiN-...**pom** est une copie du fichier pom.xml qui a servi à générer le .jar
En analysant ce fichier , la technologie maven est capable de récupérer les dépendances indirectes/transitives .

En plus de cela , la technologie maven gére automatiquement des fichiers de type ***maven-metadata.xml*** ou ***maven-metadata-local.xml*** de manière à mémoriser les dernières versions existantes (ex : timestamp de la dernière version "SNAPSHOT") .

1.3. portées ("scope") des dépendances:

Principaux types de dépendances "maven" (*scope*)

. **compile** *(par défaut)*
--> ***nécessaire pour l'exécution et la compilation*** *(dépendance directe puis transitive)* [diffusé dans tous les "classpath"].

. **runtime**
--> ***nécessaire à l'exécution*** *(dépendance indirecte* ***transitive****)*

.**provided**
--> ***nécessaire à la compilation*** mais ***fourni par l'environnement d'exécution (JVM + Serveur JEE)****[diffusé uniquement dans les "classpath" de compilation et de test, dépendance non transitive]*

. **test**
--> uniquement nécessaire pour les ***tests*** *(ex: spring-test.jar , junit4.jar)*

Diffusé dans quel(s) "classpath" ?

Type de dépendances	compilation	Tests unitaires	exécution	Transitivité (dans futur projet utilisateur / propagation)
compile (C)	x	x	x	C(C) -->C*(*)*, P(C) -->P T(C) -->T, R(C) -->R
provided (P)	x	x	x *(provided)*	--> pas propagé , à ré-expliciter si besoin
test (T)	x	x		--> pas propagé
runtime (R)		x	x	R(R) --> R, C(R)-->R T(R)-->T, P(R)-->P

(*) bizarrement quelquefois "compile" plutôt que "runtime" dans le cas où l'on souhaite ultérieurement étendre une classe par héritage.

2. Structure & syntaxes (pom.xml)

Fichier "POM" (éléments essentiels)

servlet-api-2.3.pom (exemple)

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>javax.servlet</groupId>
  <artifactId>servlet-api</artifactId>
  <version>2.3</version>
</project>
```

Version du modèle interne de maven

biblio-web-....pom

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                         http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>  <parent> ... </parent>
  <groupId>tp</groupId>  <artifactId>biblio-web</artifactId>
  <packaging>war</packaging>  <version>0.0.1-SNAPSHOT</version>
  <name>biblio-web JEE5 Webapp</name>
  .... <dependencies> ... </dependencies>  <build>....</build>
</project>
```

Fichier "POM" (déclaration des dépendances / partie 1)

```
<project ...>
  <modelVersion>4.0.0</modelVersion>  <parent> ... </parent>
  <groupId>tp</groupId>  <artifactId>biblio-web</artifactId>
  <packaging>war</packaging>  <version>0.0.1-SNAPSHOT</version>
  <name>biblio-web JEE5 Webapp</name>....
<dependencies>
        <dependency>
                <groupId>jakarta.servlet</groupId>
                <artifactId>jakarta.servlet-api</artifactId>
                <version>6.1.0</version>
                 <scope>provided</scope>
      </dependency>
            <dependency>
                <groupId>org.slf4j</groupId>
                <artifactId>slf4j-api</artifactId>
                <version>2.0.20</version>
                <scope>compile</scope>
      </dependency> ...
</dependencies>
 <build>....</build>
</project>
```

Fichier "POM" (déclaration des dépendances / partie 2)

```
...
<dependency>
        <groupId>org.hibernate</groupId>  <artifactId>hibernate-core</artifactId>
        <version>3.5.1-Final</version> <scope>compile</scope>
        <exclusions>
                <exclusion>
                        <groupId>javax.transaction</groupId>
                        <artifactId>jta</artifactId>
                </exclusion>
                <exclusion>
                        <groupId>asm</groupId>  <artifactId>asm</artifactId>
                </exclusion>
        </exclusions>
</dependency>
<dependency>
        <groupId>javax.transaction</groupId>
        <artifactId>jta</artifactId>  <version>1.1</version>
</dependency>
...
```

exclusion(s) explicite(s) de dépendance(s) indirecte(s) transitive(s)

Contrôle direct de la version souhaitée pour éviter des conflits ou des doublons

NB: au sein de certains exemples précédents, les technologies correspondent à d'anciennes versions. Sur des projets récents, on peut utiliser des versions plus récentes de hibernate. Néanmoins, les structures/syntaxes maven de pom.xml restent stables et pertinentes .

Mise au point des dépendances (partie 3)

- La ***mise au point des dépendances*** peut éventuellement être délicate en fonction des différents points suivants:
  - * potentiel ***doublon*** (2 versions différentes d'une même librairie) à partir de plusieurs dépendances transitives indirectes.
  - * potentiel ***conflit*** de librairie à l'exécution (incompatibilité entre une librairie "A" en version "runtime" et une librairie complémentaire "B" en version "provided" imposée par le serveur JEE)
  - * autres mauvaises surprises de "murphy" .
- Eléments de solutions:
  - * ***étudier finement les compatibilités/incompatibilités*** et re-paramétrer les "***version***" et "***exclusion***"
  - * (tester , ré-essayer , re-tester) de façon itérative

Fichier "POM" (partie "build")

```
<project ...>
 <modelVersion>4.0.0</modelVersion> <parent> ... </parent>
 <groupId>tp</groupId> <artifactId>biblio-web</artifactId>
 <packaging>war</packaging> <version>0.0.1-SNAPSHOT</version>
 <name>biblio-web Webapp</name>....
<dependencies>
        <dependency>...</dependency> ...
</dependencies>
 <build>
   <plugins>    <plugin>
         <groupId>org.apache.maven.plugins</groupId>
         <artifactId>maven-compiler-plugin</artifactId>
         <version>3.16.0</version>
            <configuration>
                 <release>17</release>
            </configuration>
      </plugin>...   </plugins>
  <finalName>biblio-web</finalName>
</build>
</project>
```

NB : <finalName>….</finalName> pour choisir nom du .jar à générer (${project.artifactId}).

Fichier "POM" (parties "repositories" et "properties")

```
<project ...>
 <modelVersion>4.0.0</modelVersion> <parent> ... </parent>
 <groupId>tp</groupId> <artifactId>biblio-web</artifactId>
 <packaging>war</packaging> <version>0.0.1-SNAPSHOT</version>
 <name>biblio-web JEE5 Webapp</name>....
<repositories> <!-- en plus de  http://repo1.maven.org/maven2 -->
        <repository> <!-- specific repository needed for richfaces -->
           <id>jboss.org</id>
           <url>http://repository.jboss.org/maven2/</url>
         </repository>    ...
</repositories>
<properties>
   <org.springframework.version>3.0.5.RELEASE</org.springframework.version>
   <org.apache.myfaces.version>2.0.3</org.apache.myfaces.version>
</properties>
<dependencies> <dependency>...
    <version>${org.springframework.version}</version>
</dependency> ...</dependencies>  <build>....</build>
</project>
```

Propriétés classiques :

```
<properties>
 <failOnMissingWebXml>false</failOnMissingWebXml>
                <java.version>17</java.version>
                <!-- <maven.compiler.release>17</maven.compiler.release> -->
                <spring.version>5.3.21</spring.version>
                <junit.jupiter.version>5.8.1</junit.jupiter.version>
                <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
                <!-- windows/preferences/general/workspace / UTF8 avec eclipse coherent -->
</properties>
```

et ….

```
<dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>${junit.jupiter.version}</version>  <scope>test</scope>
</dependency>
```

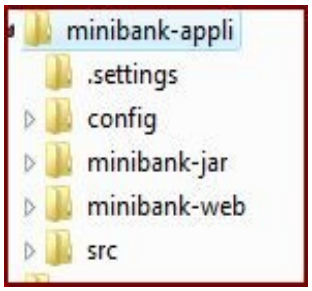
et ….

```
                        <plugin>
                                <groupId>org.apache.maven.plugins</groupId>
                                <artifactId>maven-compiler-plugin</artifactId>
                                <version>3.12.1</version>
                                <configuration>
                                        <source>${java.version}</source>
                                        <target>${java.version}</target>
                                        <release>${java.version}</release>
                                </configuration>
                        </plugin>
```

NB :
La propriété **maven.compiler.release** suffit pour choisir la version de java qui sera utilisée pour le build .
<configuration><release>…</release>… de *maven-compiler-plugin* est une configuration équivalent plus développée qui a pour intérêt d'être facilement combinable avec d'autres paramétrages complémentaires .

3. Configuration multi-modules (avec sous projet(s))

Mode "multi-projets" [parent/enfants , ear(war,jar)]



pom.xml (parent)

```
<project ....>...
 <artifactId>minibank-appli</artifactId>
 <packaging>pom</packaging>
 <modules>
    <module>minibank-jar</module>
    <module>minibank-web</module>
 </modules>
</project>
```

pom.xml (mod. web)

```
<project ....>...
 <parent> ...
  <artifactId>minibank-appli</artifactId>
 </parent>  <groupId>tp</groupId>
 <artifactId>minibank-web</artifactId>
 <packaging>war</packaging>
 <dependencies>    <dependency>
        <groupId>tp</groupId> ...
        <artifactId>minibank-jar</artifactId>
        <scope>runtime ou compile</scope>
   </dependency> ...<dependencies>...
</project>
```

pom.xml (sous module de services)

```
<project ....>...
 <parent>
  <artifactId>minibank-appli</artifactId>
  <groupId>tp</groupId> ...
 </parent>
 <groupId>tp</groupId>
 <artifactId>minibank-jar</artifactId>
</project>
```

NB (changements à prévoir dans toute nouvelle version 4 de maven) :

Un fichier pom.xml de la version 4 de maven sur le point de sortir a une entête de ce type :

```
<project
   xmlns="http://maven.apache.org/POM/4.1.0"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.1.0   http://maven.apache.org/xsd/maven-4.1.0.xsd">
 <modelVersion>4.1.0</modelVersion> ….
```

et
pour éviter toute confusion entre "module java>=9" et "module maven" , les nouvelles balises suivantes

```
<subprojects>
   <subproject>project-a</subproject>
   <subproject>project-b</subproject>
</subprojects>
```

sont prévues en remplacement très progressif de *<modules><module>...</module>...*

Mode "multi-modules" (suite)

Arborescence globale conseillée:

my-global-app
- pom.xml
- minibank-jar (module de services)
  - ...
  - pom.xml
- mywebapp
  - pom.xml
  - ...

Un module retrouve son projet parent dans le répertoire parent « .. »
tandis qu'un projet retrouve son éventuel projet parent dans un référentiel maven (local ou distant).

Principal intérêt du mode multi-module :
* ne pas devoir construire de multiples projets séparés un par un (dans l'ordre attendu en fonction des dépendances)
* simplement lancer la construction du projet principal pour que tous les sous-modules soient automatiquement (re-)construits dans le bon ordre (selon inter-dépendances)

pom.xml (de niveau projet global)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.mycompany.app</groupId>
  <artifactId>my-global-app</artifactId>
  <packaging>pom</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>....</name>
  <modules>
    <module>services</module>  <!-- ou <module>../services</module>  -->
    <module>mywebapp</module>  <!-- ou <module>../mywebapp</module>  -->
  </modules>
</project>
```

....

pom.xml (de niveau sous projet / module)

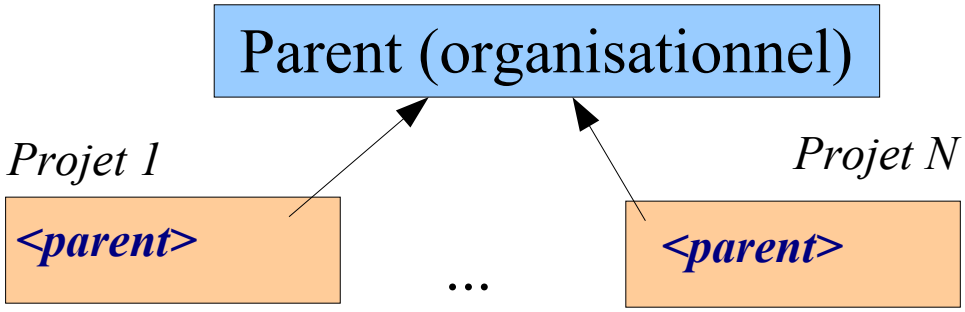
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.mycompany.app</groupId>
  <artifactId>mywebapp</artifactId> <!-- même nom que sous module courant -->
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging> <!-- ou jar -->
  ...
  <parent>
    <groupId>com.mycompany.app</groupId>
    <artifactId>my-global-app</artifactId>
  </parent>
  ...
  <dependencies>
    <!-- ici le module de présentation (ihm web) utilise le module frère "services"
         et la dépendance sera alors interprétée comme une dépendance directe (source)-->
    <dependency>
      <groupId>${pom.groupId}</groupId>
      <artifactId>services</artifactId>
      <version>1.0-SNAPSHOT</version>
    </dependency>
    ...
  </dependencies>

</project>
```

4. Héritage entre projets "maven" (<parent>)

Héritage "organisationnel" au niveau de "maven"

Dans l'absence d'une organisation multi-modules, la balise **<parent>** d'un fichier "**pom.xml**" permet de définir un lien d'héritage entre le projet courant et le projet parent :
Une certaine partie de la configuration du projet parent est ainsi héritée (sans devoir être répétée).
--> intérêt du projet parent: **factoriser** une **configuration commune** entre différents projets "fils".

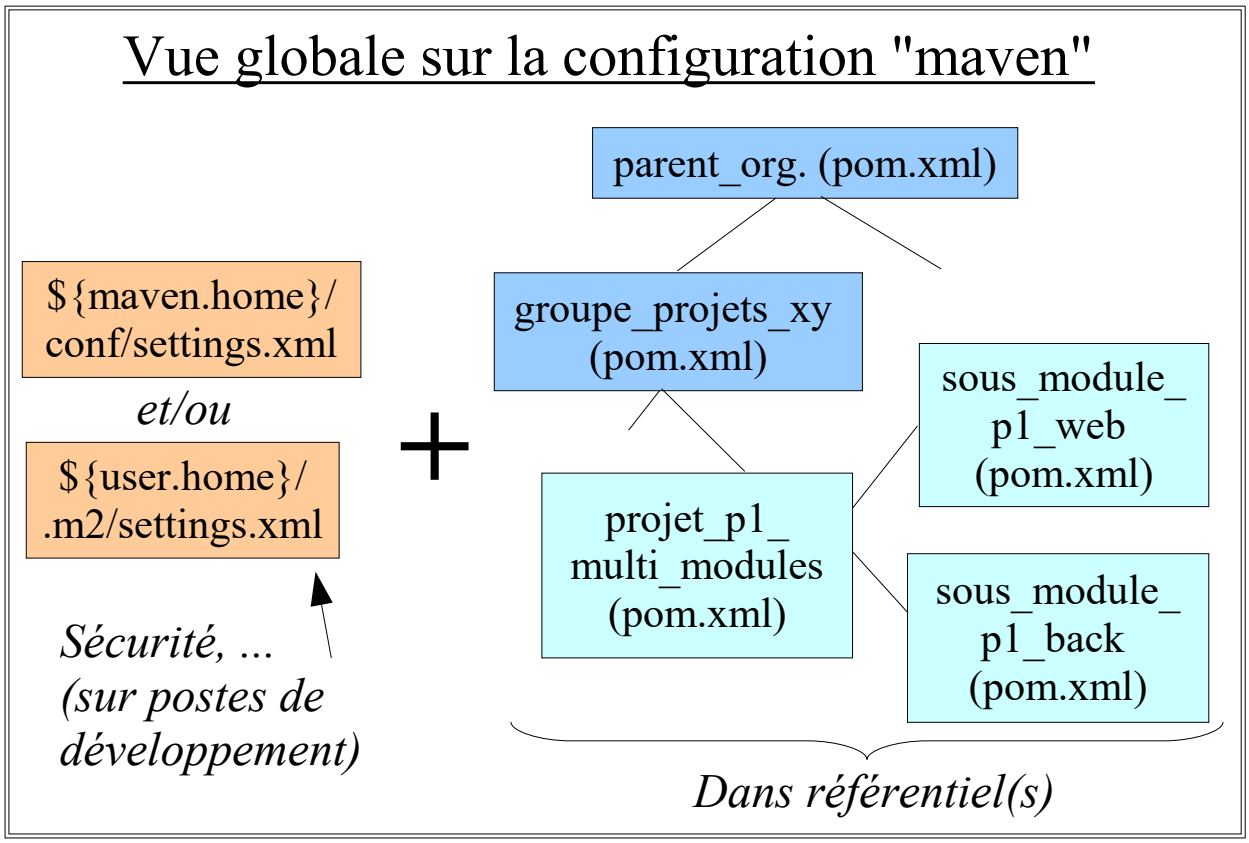


Eléments couramment hérités (pom.xml)

Un projet parent "organisationnel" *(sans obligation d'être placé dans un répertoire parent)* sert généralement à factoriser les points suivants:

- *des informations générales sur l'entreprise* (organisation, e-mails).
- *liste (avec URL) des référentiels "maven" et "svn/git" de l'entreprise et/ou des référentiels externes* (**<repositories>** , **<distributionManagement>** , ...) .
- *section <**dependencyManagement**> servant à préciser des versions et des exclusions au sein des futures dépendances qui seront exprimées au cas par cas dans les projets fils.*
- ....

Conseil: plusieurs niveaux de parents (parent intermédiaire).

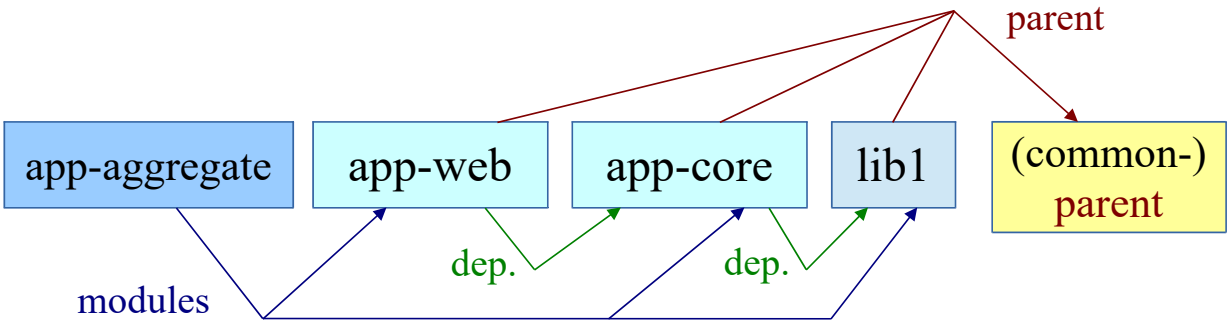


Notion de projet "aggregate" (depuis maven 3)

Un projet "**aggregate**" de type "**pom**" ***sert simplement à tout reconstruire d'un coup*** via "**mvn install**" .

Les **mécanismes "reactor" de maven** vont **tenir compte des inter-dépendances entre les projets référencés comme des modules** et **tout construire DANS LE BON ORDRE**.

Cependant, contrairement à un projet multi-modules classique **les projets référencés ne sont pas des sous-répertoires** **mais des répertoires de même niveau** et **ces différents projets ont un autre parent que le projet courant "app-aggregate"**



app-aggregate/pom.xml

```
<project ...>... <packaging>pom</packaging>
 <artifactId>app-aggregate</artifactId> <version>0.0.1-SNAPSHOT</version>
 <modules>
      <module>../common-parent</module>
       <module>../lib1</module>
       <module>../app-core</module>
       <module>../app-web</module>
 </modules> </project>
```

app-core/pom.xml

```
… <parent>...
<artifactId>common-parent</artifactId>
<version>0.0.2-SNAPSHOT</version>
<relativePath>../common-parent
</relativePath>
</parent>
<groupId>tp</groupId>
<version>0.0.1-SNAPSHOT</version>
<artifactId>app-core</artifactId>

 ... <dependency>
      <groupId>tp</groupId>
      <artifactId>lib1</artifactId>
      <version>0.0.1-SNAPSHOT</version>
  </dependency> ...
```

app-web/pom.xml

```
<parent> ...
 <artifactId>common-parent</artifactId>
 <version>0.0.2-SNAPSHOT</version>
<relativePath>../common-parent
</relativePath>
 </parent>
 <groupId>tp</groupId>
 <version>0.0.1-SNAPSHOT</version>
 <artifactId>app-web</artifactId>
 <packaging>war</packaging>
...    <dependency>
        <groupId>tp</groupId>
        <artifactId>app-core</artifactId>
        <version>0.0.1-SNAPSHOT</version>
    </dependency> ...
```

5. Archetypes

Archetype "maven" *(prédéfini ou à définir)*

- Un "***archetype***" est un ***modèle (template) de configuration de projet*** qui est accessible depuis un référentiel "maven" et qui permet de rapidement définir un nouveau fichier "pom.xml" sans devoir tout préciser en partant de "zéro".
  --> ce "*point de départ*" est évidemment à personnaliser au cas par cas selon les spécificités de chaque projet.

- Quelques exemples ("archetypes" / "projet type"):
  * "j2ee-web"
  * "jee5-..."
  * "java5"
  * "spring3+jpa2+jsf2+cxf_pour_tc6" *à mettre au point*
  * ...

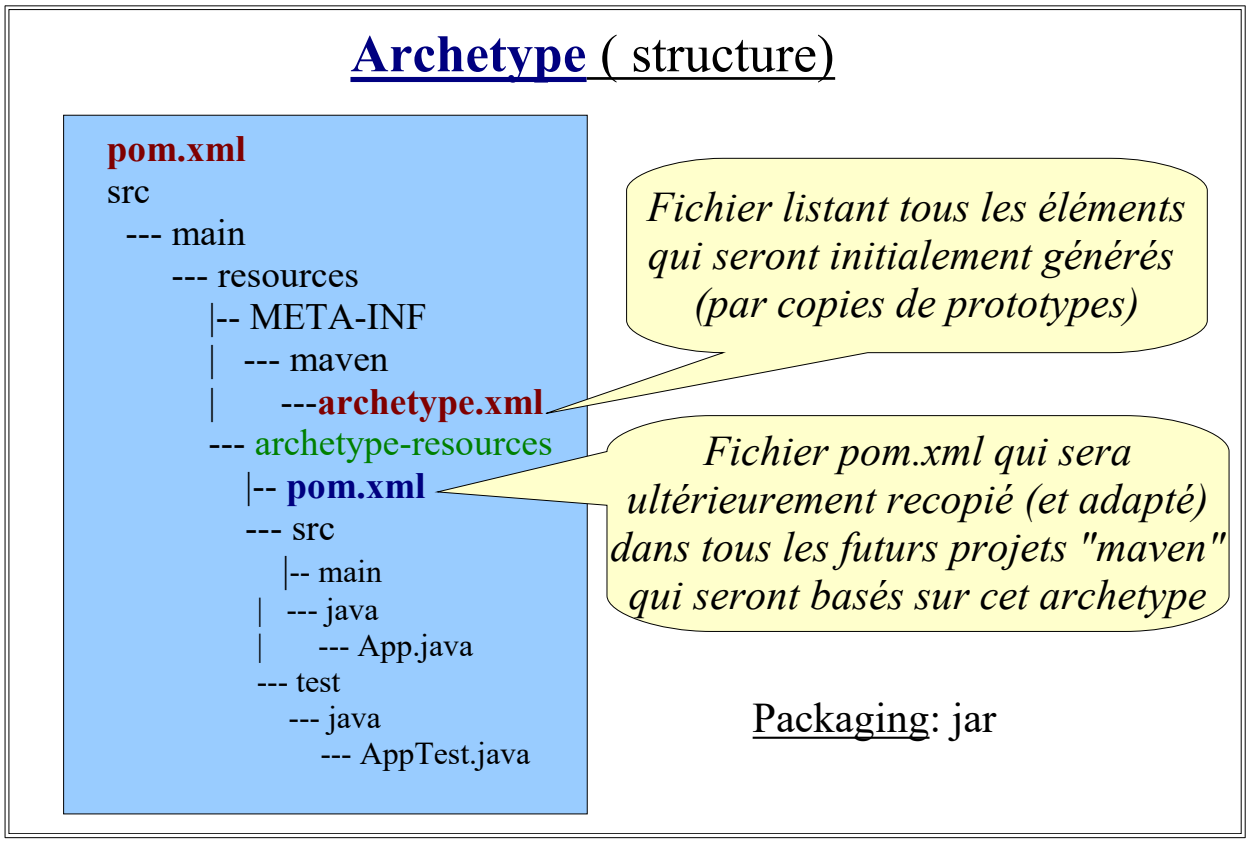
6. Utilisation d'un (nouvel) archetype maven

```
mvn archetype:generate
      -DarchetypeGroupId=com.mycompany.app1 \
      -DarchetypeArtifactId=my-java-archetype1 \
      -DarchetypeVersion=1.0-SNAPSHOT \
      -DgroupId=com.mycompany.app.via.archetype1 \
      -DartifactId=my-java-via-archetype1 \
```

NB: version attendue="RELEASE" si archetypeVersion n'est pas précisé .

+ suite habituelle (édition de code , mvn package , ...) .

7. Création d'un nouvel archetype maven



7.1. Construction d'un archetype maven depuis une application exemple/modèle .

Pour ***construire un nouvel archetype maven*** , le mode opératoire conseillé est le suivant :

1) Développer (et tester) une application exemple/modèle "*appli-exemple*" très simple (*de type point de départ avec structure conseillée et un petit exemple*) .
   Bien veiller à ce que les débuts des noms de packages coïncident avec le groupId .

2) Effectuer un "***mvn clean***" pour supprimer les parties "target" et si le développement se fait sous eclipse alors générer une copie du projet en dehors d'eclipse et en supprimant tous les fichiers "eclipse" inutiles à maven (ex : .eclipse , .project , .settings , …)

3) lancer la commande "**mvn archetype:create-from-project**" (depuis un projet mono module ou bien depuis le projet principal parent des autres modules) .
   ==> ceci permet de construire tout les fichiers sources d'un nouvel archetype.
   Résultat *(pom.xml + src)* dans **target/generated-sources/archetype** .

   4) recopier le code généré dans un projet "*appli-exemple*-**archetype**" puis lancer la commande
      "mvn ***install***" ou "mvn ***deploy***" pour que l'archetype soit enregistré et utilisable .

V - Test unitaire via maven

1. Tests unitaires avec maven

1.1. Rappels sur la structure d'un projet (partie "test")

src/test/java

Test Junit 5:

```
package package com.mycompany.app1;
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
/**
 * Unit test for simple Calculateur. (JUnit 5 with annotations)
 */
public class CalculateurTest
{
    private Calculateur c;

   /* @BeforeAll : pour initialiser une seule fois des choses statiques
      @BeforeEach ou bien "default constructor": (re)déclenchement avant chaque test
      pour initialiser des choses "non static"  [ une instance de la classe par @Test !!! ] */
    @BeforeEach
    public void mySetUp(){
      c = new Calculateur();
    }

    @Test
    public void myTestAdd(){
       Assertions.assertEquals( c.add(5,6) , 11 , 0.000001 );
    }

    @Test
    public void myTestMult(){
       Assertions.assertEquals( c.mult(5,6) , 30 , 0.000001 );
    }
}
```

1.2. Lancement des tests unitaires via maven

mvn test **-> lance tous tests dont les noms de classes commencent ou se terminent par "Test"**

mvn test -Dtest=XxxTest -> lance que le test "XxxTest"

mvn test -Dtest=*xxTest -> lance tous les tests finissants par "xxTest"

NB: par défaut , les tests unitaires sont systématiquement déclenchés lors d'un build ordinaire.
L'option "**-D*skipTests=true***" de mvn permet d'annuler/sauter l'exécution des tests.

1.3. Suite de tests JUnit5

⇒ pratique pour variantes en intégration continue (build rapide de jour , build long et sophistiqué de nuit)

Au sein d'un projet récent (prise en charge au sein d'un IDE récent , ex : eclipse >=2023-03) , une suite de tests moderne junit5 sera codée de cette manière :

```
...
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectClasses({
        TestCalculsSimples.class,
        TestEmptySerie.class,
        TestSerie.class
})
public class CalculsSuite {
}
```

→ **mvn test -Dtest=*CalculsSuite***

```
@Suite
@SelectPackages({"tp.calculs", "tp.converter" })
public class AllTestsSuite {
}
```

1.4. Suite de Tests avec Tags

@Tag("nomDeTag") de JUnit5 peut être placé sur une classe ou bien une méthode de test

Ces noms de tags (préalablement appelés catégories en Junit4) permettront d'effectuer ultérieurement des filtrages sur les tests à lancer ou pas .

Il est éventuellement possible de placer plusieurs tags complémentaires au dessus d'un même test

(ex : @Tag("**prod**") @Tag("***dev***")) .

Exemple :

```
...
import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.Test;

public class TestWithCategoryTag {
        private static Logger logger = LoggerFactory.getLogger(TestWithCategoryTag.class);

        @Test
        @Tag("SlowTests")
        public void testA() {
```

```
        logger.trace("slow testA");
    }

    @Test
    @Tag("FastTests")
    public void testB() {
        logger.trace("fast testB");
    }

    @Test
    @Tag("FastTests")   @Tag("ImportantTests")
    public void testC() {
        logger.trace("important fast testC");
    }

    @Test
    @Tag("SlowTests")   @Tag("SecondaryTests")
    public void testD() {
        logger.trace("secondary slow testD");
    }
}
```

```
...
import org.junit.platform.suite.api.ExcludeTags;
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
//@IncludeTags({"FastTests"})
@ExcludeTags({"SlowTests", "SecondaryTests"})
@SelectClasses({
    TestWithCategoryTag.class ,
    TestWithAssumptions.class
})
public class FastTestsSuite {
}
```

NB :

- par défaut , @SelectPackages(...) sélectionne tous les sous-packages également .
- On peut éventuellement ajouter @IncludePackages(...) ou bien @ExcludePackages(...) pour filtrer les sous-packages à sélectionner.
- On peut également ajouter @IncludeClassNamePatterns({ "^.*Simple$" }) pour filtrer les noms des classes de Tests à lancer .
- Possibilité de filter les category/tags au sein de la config maven:

```
<plugin>
    <artifactId>maven-surefire-plugin</artifactId>
    <configuration>
      <properties>
        <includeTags>FastTests</includeTags>
      </properties>
    </configuration>
</plugin>
```

VI - Liaison avec eclipse (m2e)

1. Lien entre maven et eclipse (m2e)

Depuis très longtemps (années >=2010) l'IDE eclipse comporte un plugin "**m2e**" (maven 2 eclipse) permettant de piloter des projets au format maven dans l'IDE eclipse.

Ce plugin fondamental est quasiment toujours déjà installé .

> Plugin eclipse "m2e" pour maven (partie 1)
>
> Depuis eclipse , on peut installer le plugin "m2e" (maven to eclipse) via l'update site suivant: http://m2eclipse.sonatype.org/sites/m2e .
>
> On peut ensuite créer de nouveaux projets eclipse de type "maven" via le menu habituel. Le choix d'un archetype peut être effectué dès la création du projet (mais n'est pas obligatoire).
>
> Lorsque l'on restructure en profondeur le fichier pom.xml , il est conseillé de déclencher le menu contextuel "***Maven/Update Project Configuration***" d'eclipse pour que la configuration du projet eclipse s'adapte à la configuration "maven".
>
> Le menu contextuel "***Run as ...***" d'éclipse permet de déclencher les "build" ordinaires de maven (***clean*** , ***test***, ***package*** , ***install***, ...).

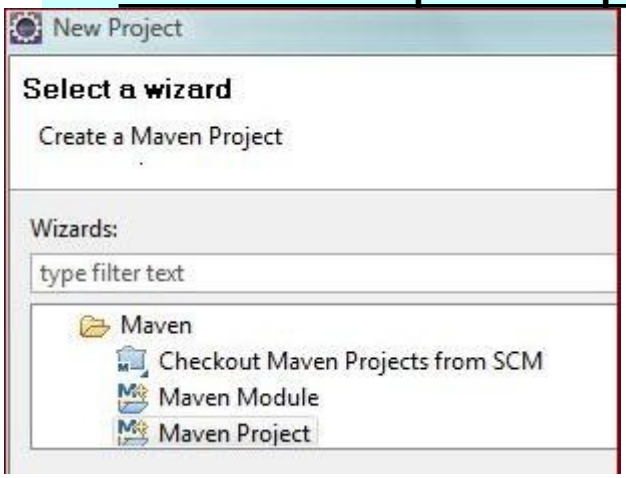
NB : URL exacte à ajuster selon version d'eclipse

1.1. Utilisation du plugin eclipse "m2e"

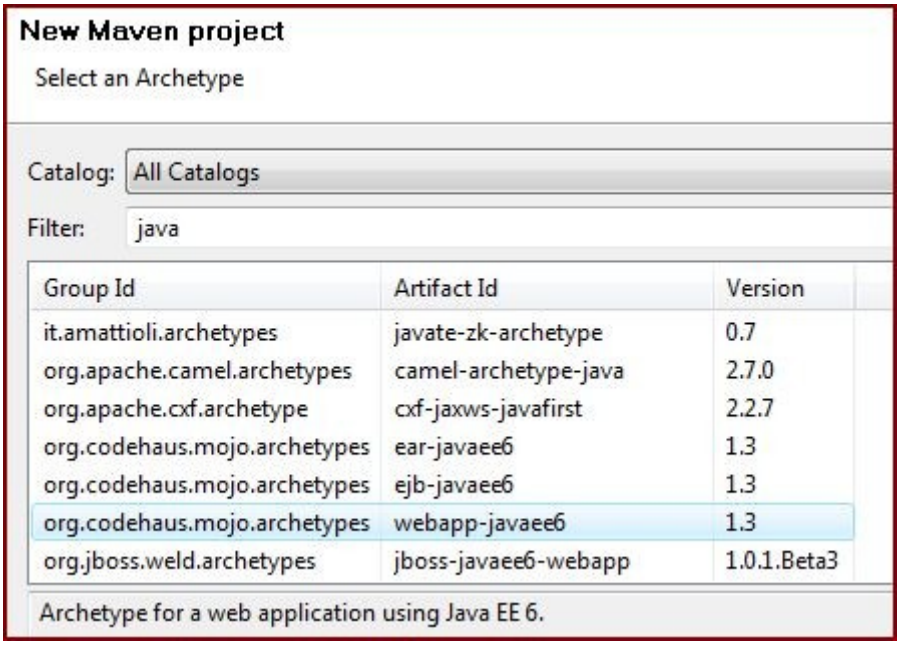
+ nouveau projet "maven"
+ éditer manuellement le fichier "pom.xml"
+ éditer le code au bon endroit (dans src/main/java, ...)
  - **run as / maven .... sur le projet** (ou run as / ... sur des sous parties (test junit))

Attention : pour ouvrir un projet maven existant sous eclipse
il faut activer le menu **"import / maven / existing maven project"**
et pas "~~import / General / existing project into workspace"~~

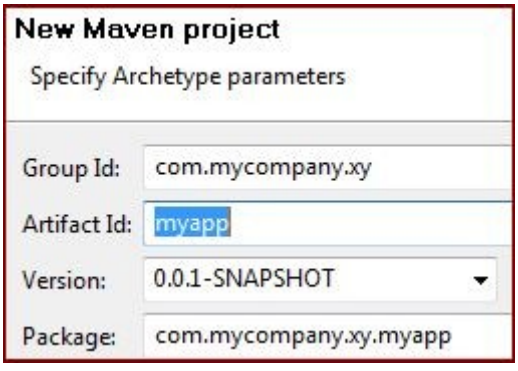
1.2. Assistants "eclipse/m2e" pour créer un nouveau projet maven



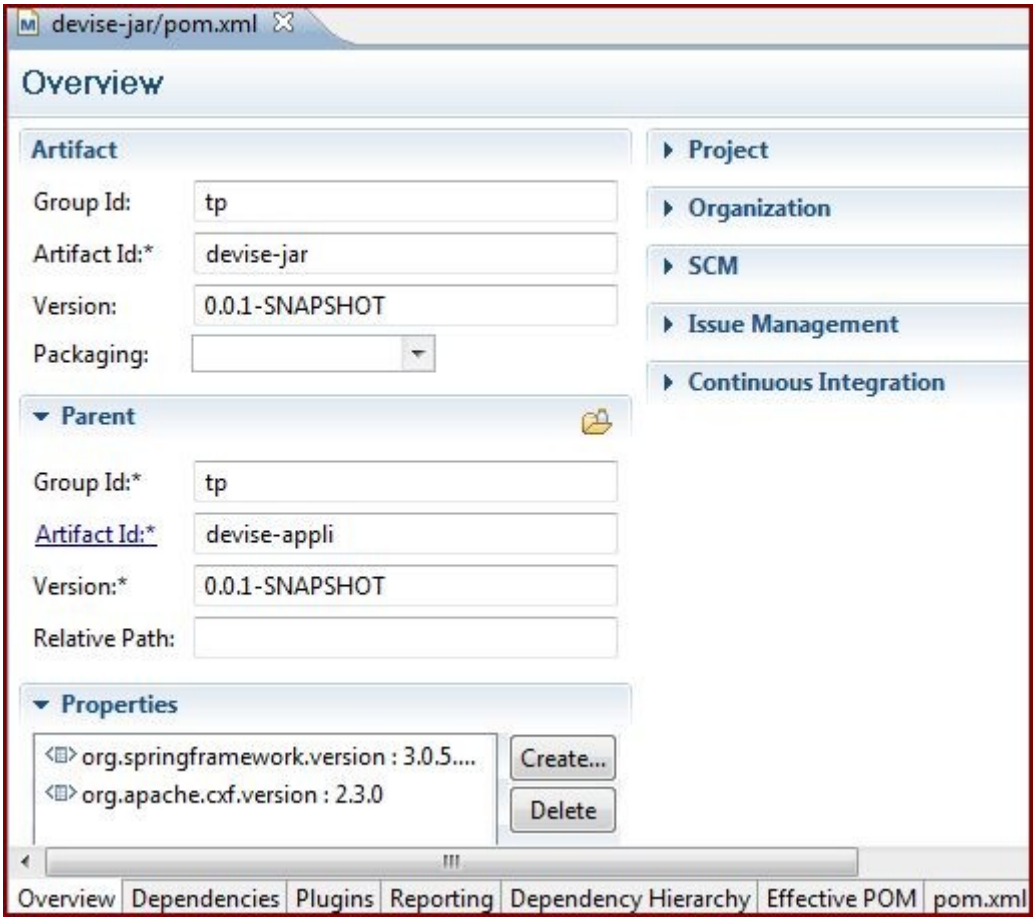


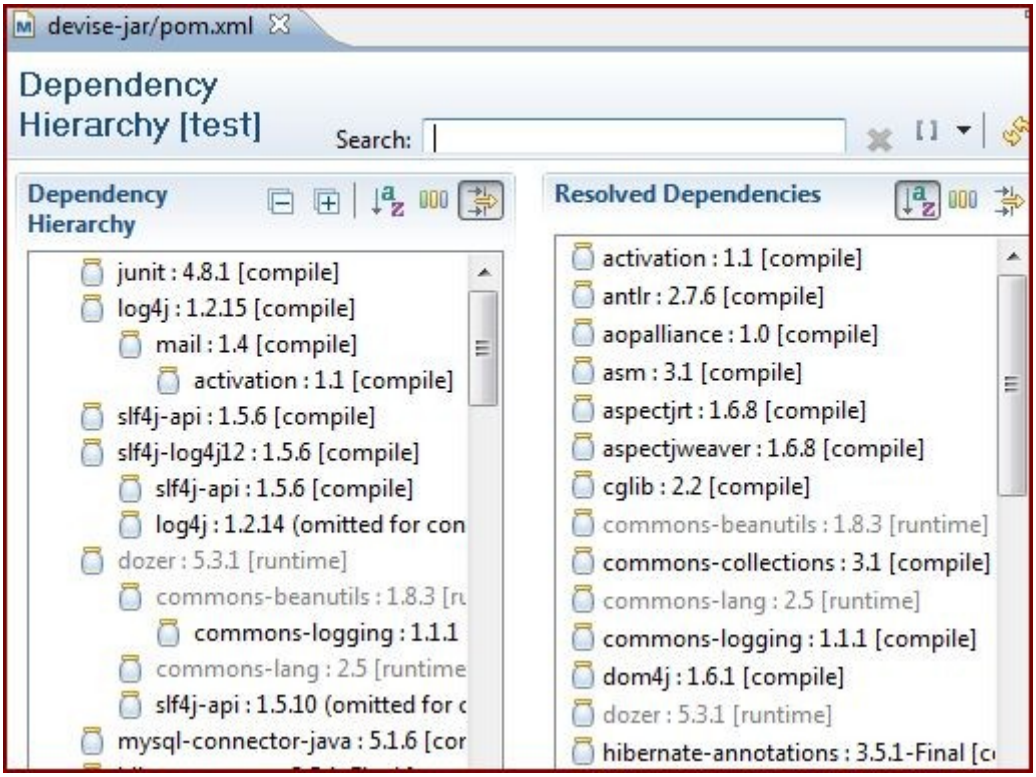


NB : Un archetype coorespond à un modèle de nouveau projet. Cela doit se préparer. **Tant qu'aucun archetype existant ne soit suffisamment intéressant, il est préférable de sauter la sélection d'un archetype en cochant la case "create simple project (skip archetype selection)"**.



1.3. Assistants "eclipse/m2e" pour paramétrer/visualiser pom.xml





NB :

- l'onglet "**pom.xml**" correspond au contenu exact du fichier "pom.xml" du projet courant.
- l'onglet "**Effective POM**" correspond à **la somme du contenu de pom.xml et de toute la configuration héritée** (projet parent + config par défaut) .
- l'onglet "**Dependency Hierarchy**" permet de bien visualiser les dépendances indirectes ce qui permet quelquefois de choisir la meilleur version en cas de conflit de versions et/ou d'alléger la configuration en n'explicitant que les dépendances essentielles .

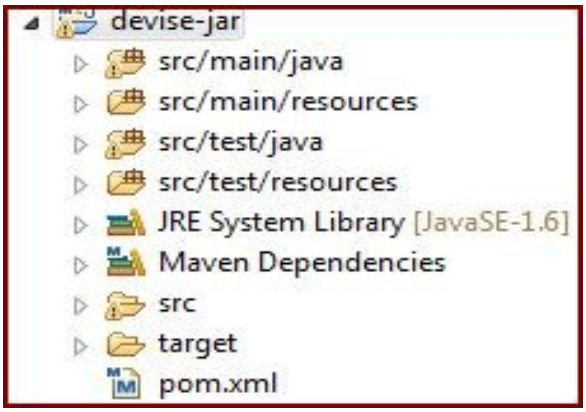
1.4. Structure des projets "eclipse_maven"

Plugin eclipse "m2e" pour maven (partie 2)

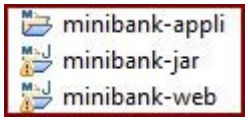
Via le plugin "m2e" , eclipse peut intégrer "maven" à travers des **projets "maven_dans_eclipse"** qui :

* ont une structure "maven" classique (***src/main/java, ...***)

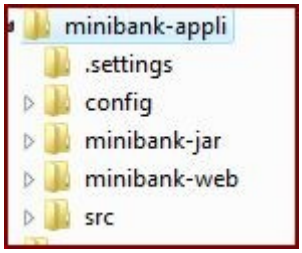
* n'ont pas la structure classique d'un projet java (~~src,bin~~)
ni la structure d'un "dynamic web projet" (~~webContent~~,...)



Cas d'une structure multi-modules:



(structure vue à plat dans eclipse)



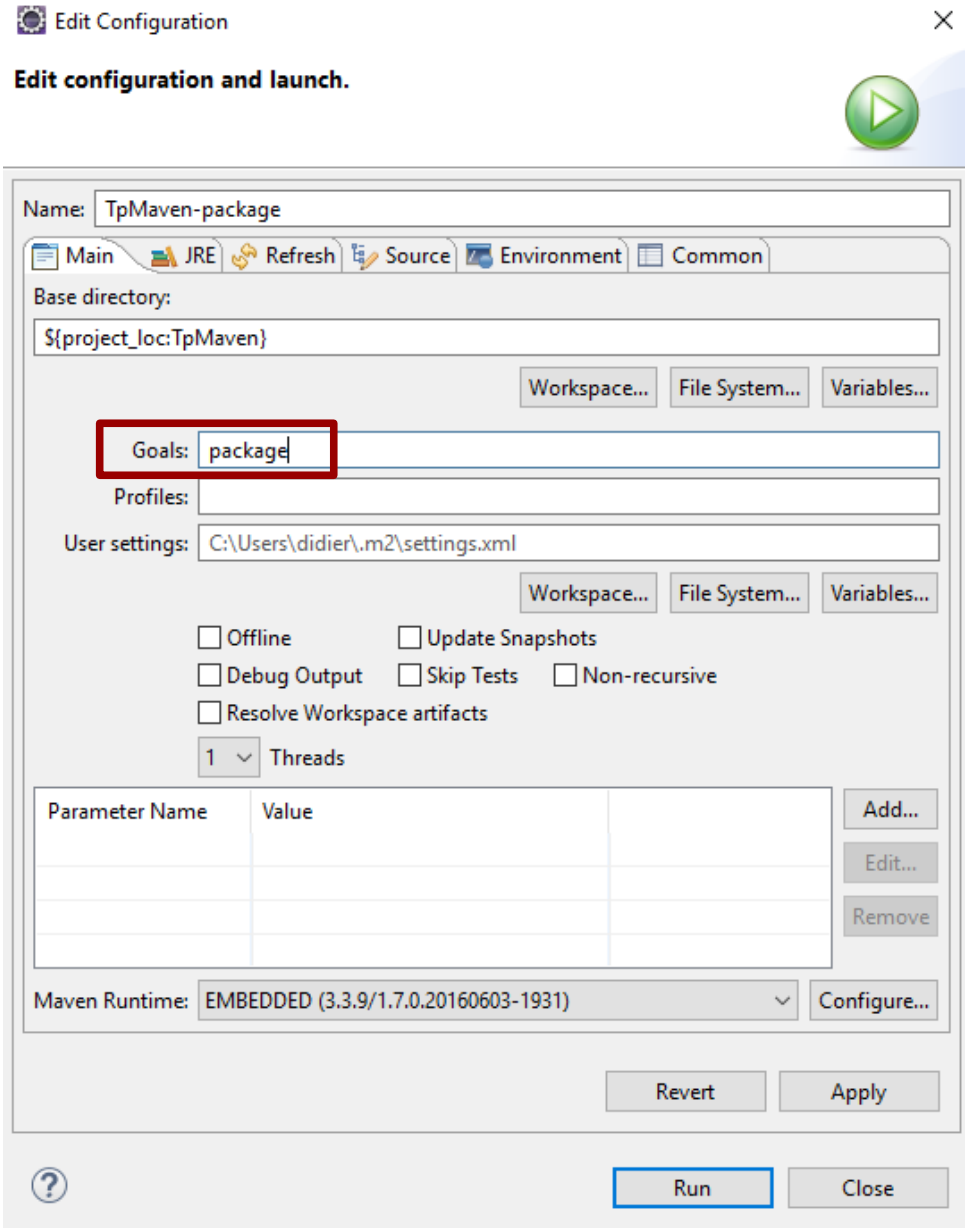
(structure arborescente sur le file system hôte).

1.5. Lancement d'un build maven depuis eclipse

Au sein des menus **"Run as / maven ...."** certains buts (classiques) sont directement accessibles :

- run as / **maven clean**
- run as / **maven compile**
- run as / **maven test**
- run as / **maven install** (package + copy artifact in .m2/repository)

Le menu "run as / **maven build ...**" permet de **faire apparaître une boite de dialogue où l'on peut saisir plein de détails dont le nom du but/goal à atteindre (ex : *package*)**



Il est souvent intéressant de donner un nom logique parlant à cette configuration (exemple : "*maven-package*") de façon à pouvoir relancer rapidement celle ci via le menu "run as / maven build" et une sélection de configuration antérieure .
NB : il est possible de retoucher certains paramétrages via le menu "project / run configurations ..."

1.6. Diverses astuces "eclipse/maven" :

- Un "Maven 3" est intégré au plugin m2e pour eclipse et une installation préalable de maven n'est pas indispensable (bien que possible et paramétrable dans le workspace eclipse).

- **Le plugin eclipse "m2e " nécessite absolument le jdk complet (plutôt que le JRE) pour fonctionner**. Il faut donc pointer vers le répertoire "**jdk** …" plutôt que "jre …" au niveau du menu "**Windows préférences / Java / Installed JREs …**"

- En cas de désynchronisation "maven-eclipse" , on peut tenter la séquence suivante :
  run as / maven clean
  run as / maven install
  project / clean (eclipse)
  server / tomcat / clean (si projet web)
  run as / run on server (si projet web)
  refresh navigateur (si projet web)

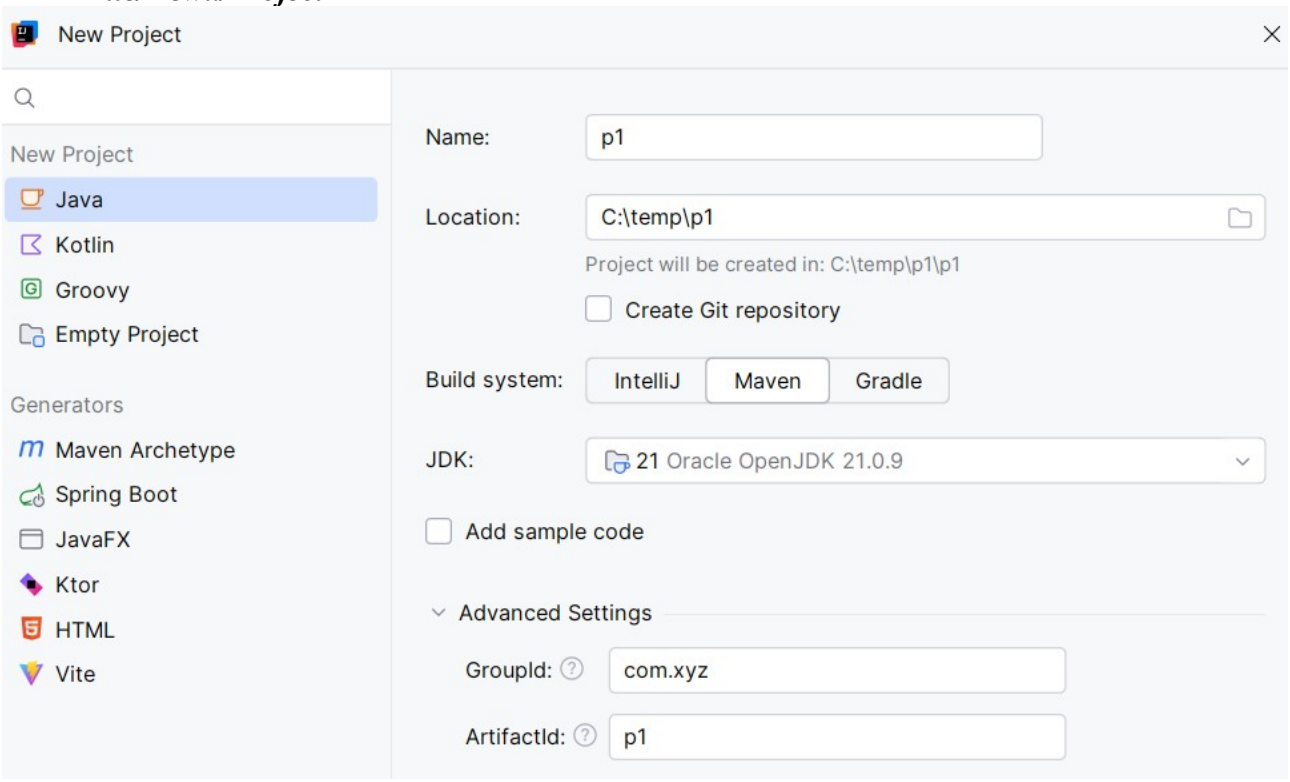
- En cas de bug inexpliqué et non identifié , il peut quelquefois être utile de redémarrer entièrement eclipse (comportant quelquefois quelques bugs ou bien rendu instable suite à des manipulations erronées) .

- Quelques fois , certains ".jar" sont mal téléchargés (mal recopiés , fichiers corrompus) et il faut manuellement **supprimer certains sous répertoires de $HOME/.m2/repository** en fonction des groupId/artifactId du message d'erreur maven .

2. Lien entre maven et IntelliJ

2.1. Création d'un nouveau projet IntelliJ/maven

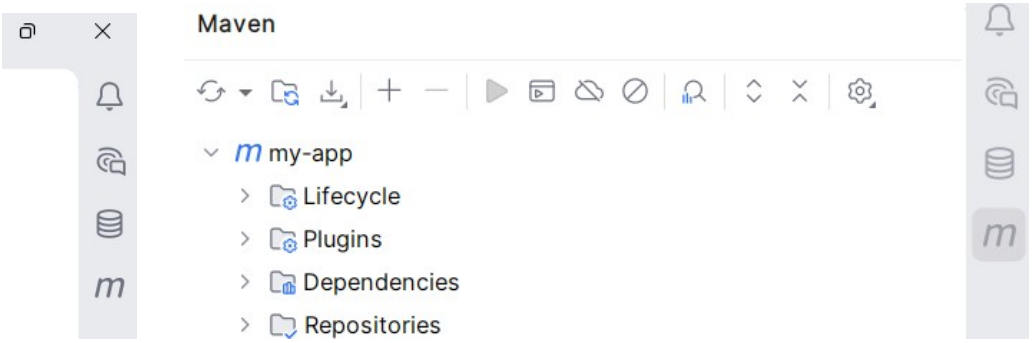
Menu ***File/New../Project***



Il est important de développer la partie "*advanced Settings*" pour choisir le *groupId* et *l'artifactId* du nouveau projet .

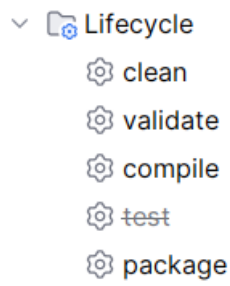
2.2. Gestion d'un projet maven existant avec IntelliJ

Lorsque l'IDE **IntelliJ** charge (via ***File/ Open…***) un projet structuré avec maven , on a tout intérêt à vérifier les paramétrages via le menu "***File/project structure ..."*** et on dispose (du coté droit) d'un panneau "**m**" dédié à maven .



Après une modification/amélioration manuelle du fichier pom.xml il est généralement nécessaire de cliquer sur l'icône de manière à activer une resynchronisation du projet (maven et intelliJ).
Sans cela , certaines dépendances sont en rouge (car par encore téléchargées).

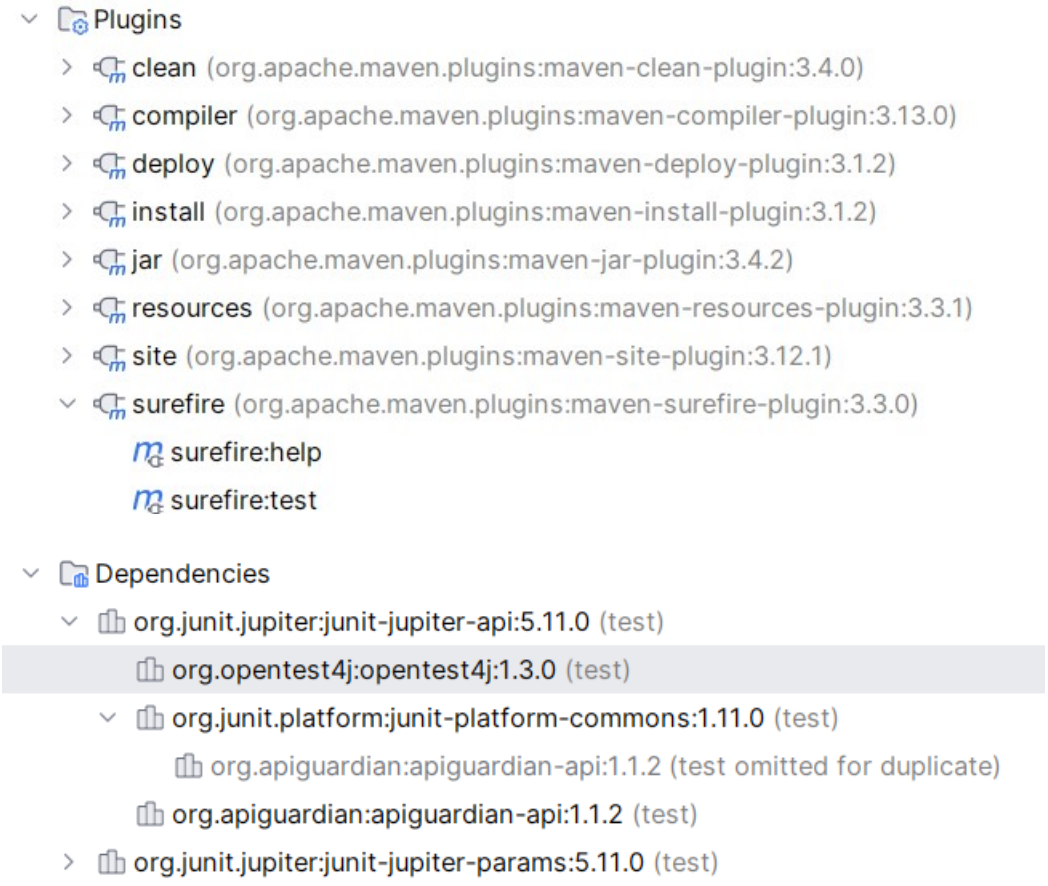
L'icône sens interdit permet d'activer si besoin l'option **`-DskipTests=true`**



Depuis l'arborescence "**Lifecycle**" , on peut intuitivement sélectionner un goal/but (ex : package) et déclencher "**run Maven Build**" .

Si nécessaire un ***click droit / Modify Run Configuration…*** permet d'effectuer certains paramétrages fins .

Autres vues fournies par intelliJ sur la configuration maven :



3. Lien entre maven de visual studio code

L'IDE "Visual Studio Code" est avant tout prévu pour le front (langage "javascript" et "typescript"). Via l'ajout d'un plugin "java" , Visual-studio-code est capable de gérer des projets structurés avec maven .

VII - Configurations (référentiels , profils, ...)

1. Mise en place d'un référentiel "Maven"

Configuration des référentiels "maven" (partie 1)

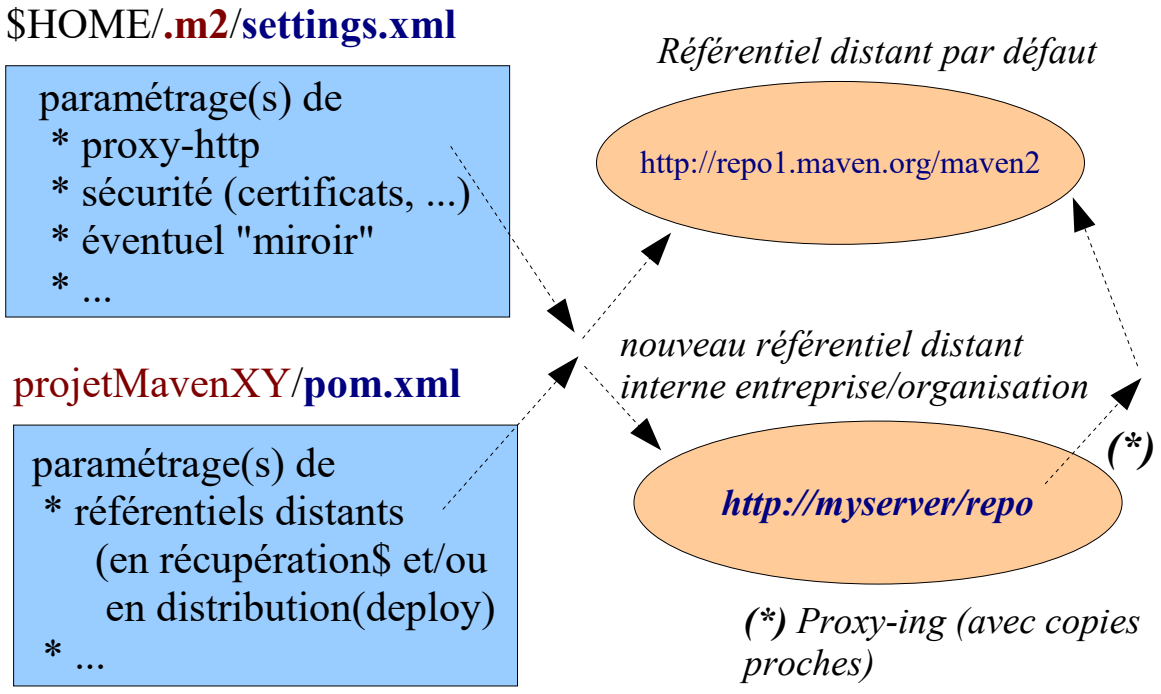
Un nouveau référentiel maven (interne à une entreprise/organisation) est simplement structuré comme le référentiel local (.m2/repository).
--> même contenu (à recopier ou alimenter)
même structure arborescente (selon groupId , artifactId et version)

Simples différences:

* Accès distant (en lecture) via **http** (ou ***https***)
* Accès distant en distribution (déploiement) de nouveaux "artifacts" via "***scp***" , "***ftp***" , "***http***" , "***webdav***" ou autre .

Les accès sécurisés (scp , https, ...) nécessitent une configuration au sein du fichier $HOME/.m2/***settings.xml***

Configuration des référentiels "maven" (partie 2)



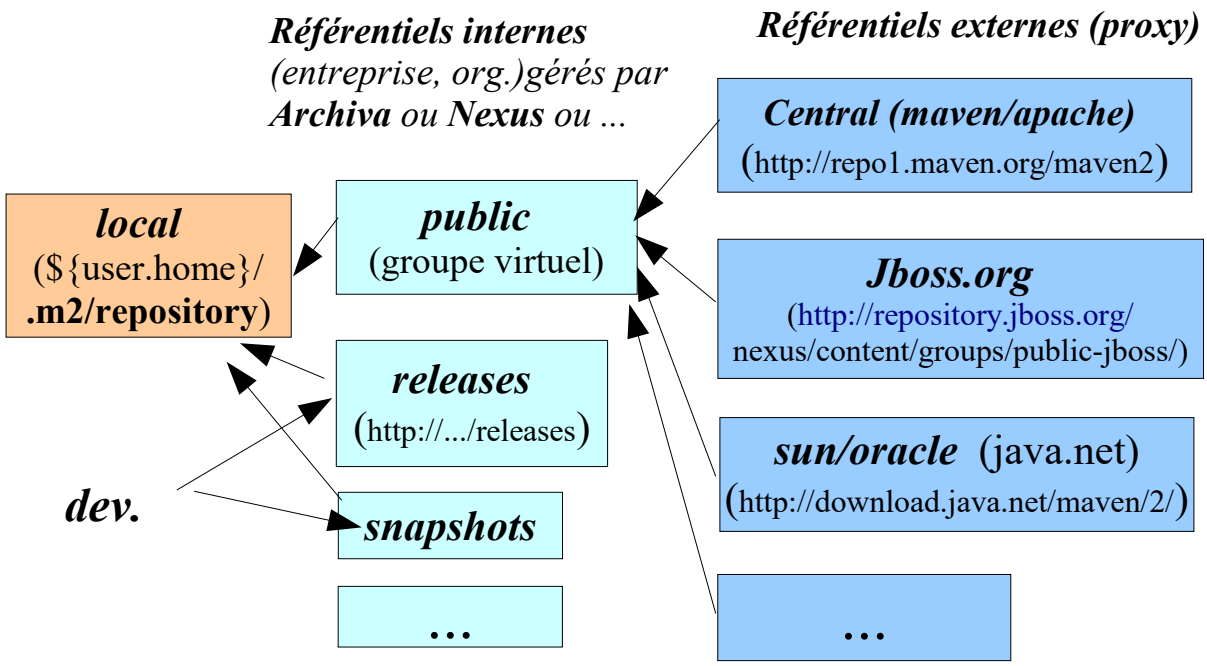
Configuration des référentiels "maven" (partie 3)

Un référentiel maven est très souvent géré par un logiciel spécialisé de type "***repository manager***".
Les produits les plus connus sont *Proximity* , *Nexus* et *Archiva* .

Archiva gère par exemple les fonctionnalités suivantes:

* ***proxy-ing vers d'autres référentiels externes*** *(avec constitution de copies des packages).*
* ***indexation*** des éléments du référentiel pour navigations et recherches rapides
* ***accès sécurisé*** vers le référentiel en écriture (via http ou autre , ...) - *comptes utilisateurs configurables*.

Vue globale sur les référentiels



1.1. Configuration du référentiel local (.m2/settings.xml)

Par défaut, lorsque rien n'est configuré , maven utilise le référentiel distant "http://repo1.maven.org/maven2" et le référentiel local $HOME/**.m2/repository** où "**.m2**" est un répertoire caché placé dans le répertoire de l'utilisateur .
...
Le fichier "$HOME/**.m2/settings.xml** peut éventuellement être modifié pour configurer le référentiel local :

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- empty maven settings.xml to copy in C:\Users\UserName\.m2\
    if C:\Users\UserName\.m2\settings.xml does not exist -->

<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0  http://maven.apache.org/xsd/settings-1.0.0.xsd">

   <!-- <localRepository>/path/to/local/repo/</localRepository>  -->


</settings>
```

1.2. Eventuelle configuration d'un proxy http (pour maven)

$HOME/.m2/settings.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<settings>
  <proxies>
    <proxy>
      <active>true</active>
      <protocol>http</protocol>
     <!-- <username>username</username>
          <password>pwd</password> -->
      <port>8080</port>
      <host>my.proxy.url</host>
      <!-- <nonProxyHosts>www.google.com|*.somewhere.com</nonProxyHosts> -->
      <!-- <id>idOfProxy</id> -->
    </proxy>
  </proxies>
</settings>
```

1.3. Référentiel spécifique (interne à l'entreprise)

Il est possible de configurer de nouveaux référentiels au sein d'une organisation (entreprise).
...
Une fois en place , ce nouveau référentiel pourra être référencé/utilisé de la façon suivante:

pom.xml

```
<project>
...
<repositories>
   <repository>
     <id>my-internal-site</id>
     <url>http://myserver/repo</url>
   </repository>
 </repositories>
</project>
```

ou bien être configuré comme un mirroir dans .m2/settings.xml:

```
<settings>
  ...
  <mirrors>
    <mirror>
      <id>internal-repository</id>
      <name>Maven Repository Manager running on repo.mycompany.com</name>
      <url>http://repo.mycompany.com/proxy</url>
      <mirrorOf>*</mirrorOf>
    </mirror>
  </mirrors>
  ...
</settings>
```

1.4. Préciser le référentiel de distribution (produits)

Pour déployer les ".jar" produits vers un référentiel Maven spécifique , on peut indiquer l'url du référentiel destination dans le "pom.xml" :

pom.xml

```
<project>
<distributionManagement>
  <repository>
    <id>mycompany-repository</id>
    <name>MyCompany Repository</name>
    <url>scp://repository.mycompany.com/repository/maven2</url>
    <!-- ou bien <url>file://localhost/z:/internalrepository/maven2</url>
     si un lecteur distant windows ou un lien nsf unix est configuré pour atteindre un répertoire
     partagé distant -->
  </repository>
 </distributionManagement>
</project>
```

Selon le mode de connexion (ici "scp") , des informations d'authentification vis à vis du référentiel sont quelquefois nécessaires et elles doivent être placées dans le fichier "settings.xml":

settings.xml

```
<settings xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                      http://maven.apache.org/xsd/settings-1.0.0.xsd">
  ...
  <servers>
    <server>
      <id>mycompany-repository</id>
      <username>jvanzyl</username>
      <!-- Default value is ~/.ssh/id_dsa -->
      <privateKey>/path/to/identity</privateKey> (default is ~/.ssh/id_dsa)
      <passphrase>my_key_passphrase</passphrase>
    </server>
  </servers>
  ...
</settings>
```

2. Repository Manager (Nexus ou ...)

Pour prendre en charge un référentiel interne (spécifique à une entreprise) , il faut au **minimum** un **serveur Http** (ex: Apache ou Tomcat) .

L' idéal consiste à installer un gestionnaire de référentiel (*Repository Manager*) qui prendra en charge quelques unes des fonctionnalités suivantes:

- **indexation**
- **redirection vers référentiels externes** (avec constitution automatique de copies locales)
- **sécurisation des ajouts au référentiel** (via username/password , ...)
- ....

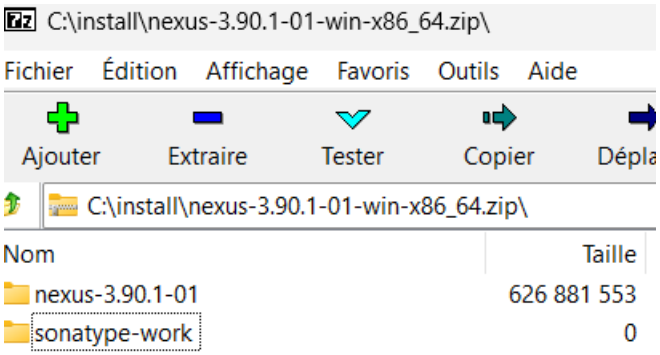
Les "***Repository Manager***" disponibles pour Maven sont (pour les plus connus):

- **Proximity** (assez ancien)
- **Nexus** de "*SonaType*" (avec bon système d'indexation) et simple à configurer/utiliser
- **Archiva** d'*Apache* (simple à configurer/utiliser)

2.1. Installation et configuration de "Sonatype Nexus"

Historiquement , l'ancienne version open source s'appelait "**nexus oss**" .
Les versions récentes de nexus sont maintenant "community edition" et "pro".
L'utilisation de la version "community" comporte quelques restrictions en terme de volume ….

Il suffit de télécharger une archive telle que "**nexus-3.90.1-01-win-x86_64**" et d'extraire son contenu sur une machine adéquate (linux ou windows) pour effectuer l'installation du produit.



Le produit nexus 3 comporte en interne son propre jdk21.

Juste après l'extraction de l'archive , on peut (sur une machine windows) lancer (dans une console d'admin) ***install-nexus-service.bat*** *(click droit / exécuter en tant qu'administrateur).*
Le nom du service (tâche de fond) est "**Sonatype Nexus Repository**" (à démarrer/stopper via "services") .

Sans le mode "démarrage comme un service" , le démarrage et l'arrêt du serveur peut quelquefois s'effectuer via la commande "bin/**nexus start ou stop**"

Le numéro de port peut être facilement changé dans le fichier **sonatype-work\nexus3\etc*nexus.properties*** (ex: ***8081 par défaut***) .

L'URL menant à la console de nexus ressemble à "***http://localhost:8081***"

Par défaut (au moment de l'installation) , le mot de passe temporaire de l'administrateur (user="**admin**") est stocké dans le fichier \sonatype-work\nexus3*admin.password* (ex : bffa319b-424a-4ce8-be53-f4e83a7c24c6) Il doit absolument être modifié lors du premier lancement (ex : "**admin123**") .

Les accès ultérieurs seront effectués en mode anonyme (bridé en exploration/lecture) ou en mode "authentifié" (avec accès plus ou moins complet (selon rôle) pour administrer Nexus) .

2.2. Configuration d'un référentiel sous Nexus

Par défaut (lors de l'installation) , Nexus est configuré avec les référentiels suivants:

- **releases (en V2)** ou **maven-releases (en V3)**
  (pour les releases de notre entreprise/organisation)

- **3rd party** (en V2) et à créer soit même en V3
  (pour des ".jar" qui ne sont pas d'origine maven)

- **public** (en V2) ou **maven-public (en V3)** (groupe virtuel)
  vers "**central**" + "**...**"

Ces référentiels peuvent être reconfigurés et on peut également créer d'autres référentiels .

L'administration (configuration) d'un référentiel peut se faire via la console web

---

NB : au sein de l'ancienne version 2 de Nexus,
L'url en lecture/download etait par défaut ***http://localhost:8080/nexus/content/groups/public/***
Les urls en envois (snapshots et releases) étaient par défaut les suivantes :

```
http://localhost:8080/nexus/content/repositories/snapshots
http://localhost:8080/nexus/content/repositories/releases
```

NB : La nouvelle **version 3** de Nexus utilise par défaut le port **8081** .
L'url en lecture/download est par défaut **http://localhost:8081/repository/maven-public/**
Les urls en envois (snapshots et releases) sont par défaut les suivantes :
http://localhost:8081/repository/maven-snapshots
http://localhost:8081/repository/maven-releases

2.3. Utilisation de Nexus en "lecture/téléchargement"

Si l'on souhaite (comme souvent) , toujours passer indirectement par le référentiel "***public***" de nexus pour accéder aux référentiels externes (central maven , jboss , ...) , il faut commencer par éditer le fichier **settings.xml** de la façon suivante:

```
<settings>
  ...
  <mirrors>
    <mirror>
      <id>nexus</id>
```

```
    <!--   <url>http://localhost:8484/nexus/content/groups/public/</url>  -->
       <url>http://localhost:8081/repository/maven-public/</url>
 <!-- <mirrorOf>central,jboss.org,maven2-repository.dev.java.net</mirrorOf> -->
      <mirrorOf>*</mirrorOf>
     </mirror>
    </mirrors>
   ...
</settings>
```

Ceci permet de changer le comportement par défaut (qui habituellement consistait à interroger d'office en premier le référentiel "central" externe).

Il faut également quelquefois ajouter la configuration d'un profil particulier "**nexus**" et activé d'office dans le fichier ***settings.xml*** :

```
<profiles>
  <profile>
   <id>nexus</id>
   <!--Enable snapshots for the built in central repo to direct -->
   <!--all requests to nexus via the mirror -->
   <repositories>
    <repository>
     <id>central</id>
     <url>http://repo1.maven.org/maven2</url>
     <releases><enabled>true</enabled></releases>
     <snapshots><enabled>true</enabled></snapshots>
    </repository>
   </repositories>
  <pluginRepositories>
    <pluginRepository>
     <id>central</id>
     <url>http://repo1.maven.org/maven2</url>
     <releases><enabled>true</enabled></releases>
     <snapshots><enabled>true</enabled></snapshots>
    </pluginRepository>
   </pluginRepositories>
  </profile>
 </profiles>
 <activeProfiles>
  <!--make the profile active all the time -->
  <activeProfile>nexus</activeProfile>
 </activeProfiles>
```

il faut également ajouter dans ***settings.xml*** les éléments de sécurité nécessaires:

```
<settings>  ...
  <servers>
     <server>
      <id>public</id>
      <username>admin</username>
      <password>admin123</password>
     </server>
```

```
  ...
  </servers>  ...
</settings>
```

2.4. Utilisation de Nexus en "écriture/alimentation"

Pour que "**mvn deploy**" puisse déployer l'artifact construit vers le référentiel géré par nexus il faut au minimum :

- Créer un (ou plusieurs) nouveau(x) **compte d'utilisateurs** au sein de nexus (avec [username, password] pour le déploiement. *(NB : en tp/formation on peut éventuellement utiliser le compte "admin/admin123")*
- Associer/affecter le rôle '***Nexus Deployment Role***' pour chaque user / repository vers lesquels on souhaite effectuer des déploiements.
- **Ajuster la sécurité dans 'settings.xml'**:

```
<settings> ...
 <servers>
  <server>
   <id>snapshots</id>  <username>admin</username> <password>admin123</password>
  </server>
  <server>
   <id>releases</id>
   <username>admin</username>     <!-- or specific deployment user -->
   <password>admin123</password> <!-- or specific deployment user password-->
  </server>
  ...
 </servers>  ...
</settings>
```

il faut ensuite paramétrer (dans un ***pom.xml***) les **URLs pour le déploiement vers nexus** :

```
<project>
  ...
  <distributionManagement>
  <repository>
    <id>releases</id>
   <!-- <url>http://localhost:8080/nexus/content/repositories/releases</url> -->
    <url>http://localhost:8081/repository/maven-releases</url>
  </repository>
  <snapshotRepository>
    <id>snapshots</id>
  <!-- <url>http://localhost:8080/nexus/content/repositories/snapshots</url> -->
    <url>http://localhost:8081/repository/maven-snapshots</url>
  </snapshotRepository>
  </distributionManagement>
  ...
</project>
```

L'URL peut être formulée avec plusieurs protocoles supportés par nexus (selon contexte et O.S.) :

HTTP	**http://localhost:8484/nexus/content/repositories/releases en V2** **ou http://localhost:8081/repository/maven-releases en V3**
WebDAV	dav:http://....

Il est éventuellement possible d'alimenter un référentiel à partir d'un artifact tierce-partie (".jar" issu

d'un projet non maven):

```
mvn deploy:deploy-file -Dfile=filename.jar -DpomFile=filename.pom
    -DrepositoryId=thirdparty
    -Durl=http://localhost:8484/nexus/content/repositories/thirdparty
```

2.5. Autres fonctionnalités de nexus

- Suppression d'un artifact maven via le menu contextuel "**delete**" de la console web
- Rendre caduque la valeur en cache/proxy pour forcer un nouveau futur téléchargement via le menu contextuel "**Expire Cache**" *[NB : ceci est très pratique et utile dans le cas où une première tentative de téléchargement a échoué suite à un problème de communication réseau / xxx.pom présent mais xxx.jar absent]*
- Navigation & recherche dans un référentiel
- Paramétrer un proxy http à usage interne (*Administration/server/default http proxy*)

3. Profils "maven"

Profils "maven" (utilité , configuration)

Pour **personnaliser une configuration d'un projet** ***selon certaines spécificités d'un environnement d'exécution*** *(ex: selon version jdk , selon o.s. , selon variable d'environnement, ...)* tout en gardant une bonne ***portabilité*** au niveau du fichier ***pom.xml*** , on peut paramétrer ***différents profils en parallèle*** (avec des ***variantes*** dans la ***configuration***).

Lors d'une invocation de maven , le **profil** adéquat sera automatiquement **activé** (en fonction du contexte ou d'un certain paramétrage).

```
<profiles> <profile>
  <activation>   ....  </activation>
  ...
</profile></profiles>
```

Types de configuration possible des profiles "maven":

Portée	*Localisation de la configuration*
projet	**pom.xml** du projet
utilisateur	**%USER_HOME%/.m2/settings.xml**
global	**%M2_HOME%/conf/settings.xml**
Selon conf "profiles maven 3"	....

3.1. Activation selon la version de java

```
<profiles>
  <profile>
    <activation>
      <jdk>1.4</jdk>
    </activation>
    ...
  </profile>
</profiles>
```

Et depuis la version 2.1 , possibilité d'exprimer des plages :

```
<profiles>
  <profile>
    <activation>
      <jdk>[1.3,1.6)</jdk>  <!-- du 1.3 au 1.5 (1.6 exclus) -->
    </activation>
    ...
  </profile>
</profiles>
```

3.2. Activation selon le système d'exploitation (OS):

```
<profiles>
  <profile>
    <activation>
      <os>
        <name>Windows XP</name>
        <family>Windows</family>
        <arch>x86</arch>
        <version>5.1.2600</version>
      </os>
    </activation>
    ...
  </profile>
</profiles>
```

3.3. Activation selon une propriété système java

(paramètre -Dxx.yy d'une ligne de commande mvn groupId:artifactId:goal)

```
<profiles>
  <profile>
    <activation>
      <property>
        <name>debug</name>
        <!-- isDefined , any value -->
      </property>
    </activation>
    ...
  </profile>
</profiles>

<profiles>
  <profile>
    <activation>
      <property>
        <name>environment</name>
        <value>test</value>      <!-- if -Denvironment=test -->
      </property>
    </activation>
    ...
  </profile>
</profiles
```

Pour activer selon variable d'environnement:
-Dxxx.yyy=%VAR_XX_YY% dans .bat
ou -Dxxx.yyy=${VAR_XX_YY} dans .sh

3.4. Activation selon un fichier manquant ou existant

```
<profiles>
  <profile>
    <activation>
      <file>
        <missing>target/generated-sources/xxx/yyy</missing>
        <!-- ou bien  <exists>...</exists> -->
      </file>
    </activation>
    ...
  </profile>
</profiles>
```

3.5. Profils nommés à activer explicitement

```
<profiles>
  <profile>
    <id>profile-1</id>

    ...
  </profile>
  <profile>
    <id>profile-2</id>

    ...
  </profile>
</profiles>
```

et

mvn groupId:artifactId:goal -P profile-n

Eventuel profil nommé et activé "en dur":

```
<settings>
  ...
  <activeProfiles>
    <activeProfile>profile-1</activeProfile>
  </activeProfiles>
  ...
</settings>
```

ou encore

```
<profiles>
  <profile>
    <id>profile-1</id>
    <activation>
      <activeByDefault>true</activeByDefault>
    </activation>
    ...
  </profile>
</profiles>
```

3.6. Exemple concret (switch de configuration)

```
<project>
  ...
  <build>
    <plugins>
      <plugin>
        <groupId>....</groupId>
        <artifactId>....n</artifactId>
        <version>1.0</version>
        <configuration>
          <appserverHome>${appserver.home}</appserverHome>
        </configuration>
      </plugin>
      ...
    </plugins>
  </build>
  ...
</project>
```

avec ***${appserver.home}*** ***défini alternativement en "/path/to/dev/appserver" ou "/path/to/dev/appserver2" selon -Denv=dev ou bien -Denv=dev2 .***

```
<project>
  ...
  <profiles>
    <profile>
      <id>appserverConfig-dev</id>
      <activation>
        <property>
          <name>env</name>
          <value>dev</value>
        </property>
      </activation>
      <properties>
        <appserver.home>/path/to/dev/appserver</appserver.home>
      </properties>
    </profile>

    <profile>
      <id>appserverConfig-dev-2</id>
      <activation>
        <property>
          <name>env</name>
          <value>dev-2</value>
        </property>
      </activation>
      <properties>
        <appserver.home>/path/to/another/dev/appserver2</appserver.home>
      </properties>
    </profile>
  </profiles>
  ..
</project>
```

4. Filtrage des ressources

Principe:
Certains fichiers de ressources (.properties , .xml) peuvent éventuellement comporter des valeurs basées sur des **variables qui seront renseignées lors de la construction via maven**.
Ceci permet d'obtenir ***plusieurs configurations différentes*** (pour les bases de données par exemple) e***n fonction d'un choix de profile***.

Exemples:

exemple.properties (dans src/main/resources)

```
# les valeurs seront remplacees par celles qui seront choisies
# dans la configuration du profile maven actif
# en mode "filtering resource" (voir resultats dans "target")
xxx.yyy=${xxx.yyy}
xxx.zzz=${xxx.zzz}
```

exemple.xml (dans src/main/resources)

```
<exemple>
  <xxx_yyy>${xxx.yyy}</xxx_yyy>
  <xxx_zzz>${xxx.zzz}</xxx_zzz>
</exemple>
```

Configuration maven:

```
<project ...>. ...
  <profiles>
  <profile> <id>p1</id>
          <properties>
                  <xxx.yyy>jetty</xxx.yyy>    <!-- valeur de remplacement pour ${xxx.yyy} -->
                   <xxx.zzz>jetty2</xxx.zzz>
          </properties>
    </profile>
    <profile> <id>p2</id>
          <properties>
                  <xxx.yyy>tomcat</xxx.yyy>
                   <xxx.zzz>tomcat6</xxx.zzz>
          </properties>
    </profile>
  </profiles>  ...
  <build><plugins> ... </plugins>
          <resources>
            <resource>
               <directory>src/main/resources</directory>
               <filtering>true</filtering>
            </resource>
          </resources>
  </build>
</project>
```

NB: Pour activer le **filtrage de ressources** sur la partie "***tests unitaires***" , il faut ajouter (dans ***pom.xml***) la configuration suivante :

```
<testResources>
    <testResource>
        <directory>src/test/resources</directory>
        <filtering>true</filtering>
    </testResource>
</testResources>
```

en plus de <resources><resource> … pour la partie src/main/resources

5. Ajout (et éventuel filtrage) de ressources "web" externes

```
<project>  ...
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>2.1.1</version>
        <configuration>
          <webResources>
            <resource>
              <!-- this is relative to the pom.xml directory -->
              <directory>resource2</directory>
            </resource>
          </webResources>
        </configuration>
      </plugin>
    </plugins>
  </build>
  ...
</project>
```

---> **effet/résultats (dans target)** : tout le contenu du répertoire "resource2" est ajouté au contenu de src/main/webapp .

Possibilités de <includes> , <excludes> :

```
...
        <configuration>
          <webResources>
            <resource>
              <directory>resource2/</directory>
              <!-- the list has a default value of ** -->
              <includes>
                <include>image2/*.jpg</include>
              <includes>
              <!-- there's no default value for this -->
              <excludes>
                <exclude>**/*.jpg</exclude>
              </excludes>
            </resource>
          </webResources>
        </configuration>
        ...
```

Via **<targetPath>...</targetPath>** , un contenu externe peut être ajouté à un endroit précis (ex:

WEB_INF) .

D'autre part , certaines variables de certains fichiers de configurations peuvent être filtrées (c'est à dire remplacées par des valeurs paramétrables).

Exemple:

via le filtre

configurations/properties/**config.prop**

```
interpolated_property=some_config_value
```

la valeur de la variable ${interpolated_property} du fichier de configuration suivant sera automatiquement remplacée:

configurations/**config.cfg**

```
<another_ioc_container>
  <configuration>${interpolated_property}</configuration>
</another_ioc_container>
```

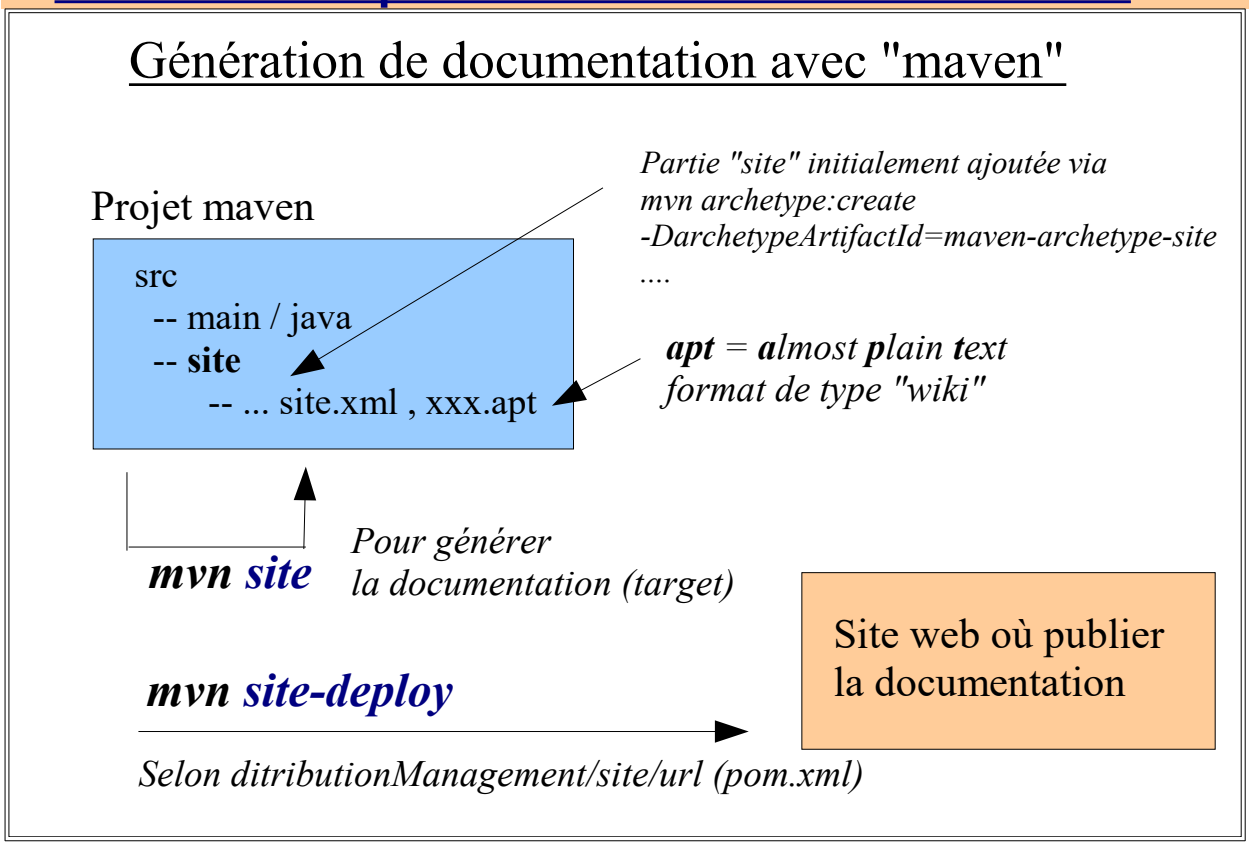
pom.xml

```
...
      <configuration>
        <filters>
         <filter>properties/config.prop</filter>
        </filters>
        <nonFilteredFileExtensions>
          <!-- default value contains jpg,jpeg,gif,bmp,png -->
          <nonFilteredFileExtension>pdf</nonFilteredFileExtension>
        </nonFilteredFileExtensions>
        <webResources>
          <resource>
            <directory>resource2</directory>
            <!-- it's not a good idea to filter binary files -->
            <filtering>false</filtering>
          </resource>
          <resource>
            <directory>configurations</directory>
             <!-- override the destination directory for this resource →
            <targetPath>WEB-INF</targetPath>
             <filtering>true</filtering>

            <excludes>
              <exclude>**/properties</exclude>
            </excludes>
          </resource>
      ...
```

VIII - Génération de documentation (mvn)

1. Génération et publication d'une documentation



Commande pour ajouter la partie "site" sur un projet maven existant:

```
mvn archetype:create \
  -DarchetypeGroupId=org.apache.maven.archetypes \
  -DarchetypeArtifactId=maven-archetype-site \
  -DgroupId=com.mycompany.app \
  -DartifactId=my-app-site
```

Arborescence générée:

```
my-app-site
|-- pom.xml
`-- src
    `-- site
        |-- apt
        |   |-- format.apt
        |   `-- index.apt
        |-- fml
        |   `-- faq.fml
        |-- fr
        |   |-- apt
        |   |   |-- format.apt
        |   |   `-- index.apt
        |   |-- fml
        |   |   `-- faq.fml
        |   `-- xdoc
        |       `-- xdoc.xml
        |-- xdoc
        |   `-- xdoc.xml
        |-- site.xml
```

```
        `-- site_fr.xml
```

NB: apt = "almost plain text" = format de type "wiki"
+ *voir la documentation de référence pour approfondir le sujet .*

Ajout de liens hypertextes dans **site.xml**

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<project name="Maven" xmlns="http://maven.apache.org/DECORATION/1.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/DECORATION/1.0.0
http://maven.apache.org/xsd/decoration-1.0.0.xsd">

  <body>
    <links>
      <item name="Apache" href="http://www.apache.org/" />      ...
    </links>

    <menu name="Maven 2.0">
       <item name="FAQ" href="faq.html"/>
      <item name="javadoc (api)" href="apidocs/index.html"/>
      <item name="checkstyle-report" href="checkstyle.html"/>
      <item name="jdepend-report" href="jdepend-report.html"/>
    </menu>
  </body>
</project>
```

pom.xml

```
<project>
  ....
  <distributionManagement>
    <site>
      <id>website</id>
      <url>scp://webhost.company.com/www/website</url>
      <!-- ou en file: si répertoire distant partagé via nfs ou autre -->
    </site>
  </distributionManagement>
  <build>
    <plugins>
      <plugin>
        <artifactId>maven-site-plugin</artifactId>
        <configuration>
          <locales>en,fr</locales>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

mvn site *(pour générer la doc html)*

mvn site-deploy *(pour déployer la doc générée)*

IX - Rapports (et javadoc) avec maven

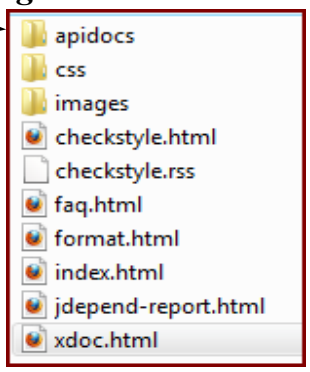
1. Javadoc (via maven)

Générer **javadoc** avec "maven"

```
... (dans pom.xml)
<reporting>
 <plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-javadoc-plugin</artifactId>
    <version>2.7</version>
    <!-- <configuration>
      <minmemory>128m</minmemory>
      <maxmemory>512m</maxmemory>
    </configuration> -->
  </plugin>
   ...
</reporting>
```

mvn javadoc:javadoc

target/site/



2. Rapports avec maven

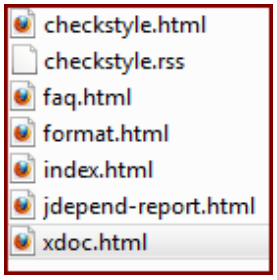
rapports avec "maven"

```
... (dans pom.xml)
<reporting>
 <plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-checkstyle-plugin</artifactId>
    <version>2.6</version>
  </plugin>
  <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>jdepend-maven-plugin</artifactId>
      <version>2.0-beta-2</version>
  </plugin>
  <!-- maven-pmd-plugin not found -->
 </plugins>
</reporting>
```

mvn checkstyle:checkstyle

mvn jdepend:generate

target/site/



X - Plugins pour maven (exécution, prog.)

1. Plugins pour maven

Plugins "maven" (utilité , structure)

• Les mécanismes internes de maven **délèguent** toutes les ***sous tâches spécialisées*** à des "***plugins***" (ex: un plugin pour "***clean***" , un autre plugin pour packager le code sous forme de fichier zip [".war", ".jar" , ".ear" , ...]).
• Il est possible de **programmer** de nouveaux ***plugins personnalisés*** et de les configurer pour qu'ils soient activés.

• Le ***développement d'un plugin*** s'effectue généralement sous la forme d'une **classe java** ("***MOJO***" : *M*aven P*OJO*) comportant une méthode "***execute***()" et codée au sein d'un projet maven de type "plugin" .

• L'***activation*** s'effectue souvent en ***associant un but d'un plugin en tant que tâche supplémentaire à exécuter lors d'une des phases de construction (ex: "compile" , "package" , ...)***

1.1. execution

Configuration d'un plugin "maven" à activer

```
<build>
  <plugins>
    <plugin>
      <groupId>sample.plugin</groupId>
      <artifactId>hello-maven-plugin</artifactId>
      <version>1.0-SNAPSHOT</version>
      <executions>
        <execution>
          <phase>compile</phase>
          <goals>
            <goal>sayhi</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

1.2. Liste des principaux plugins de maven

** (Type): **B**uild or **R**eporting plugin*

Plugin	Type *	Version	Release Date	Description
Core plugins				**Plugins corresponding to default core phases (ie. clean, compile). They may have muliple goals as well.**
clean	B	2.4.1	2010-05-12	Clean up after the build.
compiler	B	2.3.2	2010-09-09	Compiles Java sources.
deploy	B	2.5	2009-12-24	Deploy the built artifact to the remote repository.
failsafe	B	2.8	2011-03-13	Run the Junit integration tests in an isolated classloader.
install	B	2.3.1	2010-05-21	Install the built artifact into the local repository.
resources	B	2.5	2011-02-27	Copy the resources to the output directory for including in the JAR.
site for Maven 2	B	2.2	2010-11-28	Generate a site for the current project.
site for Maven 3	B	3.0-beta-3	2010-10-18	Generate a site for the current project.
surefire	B	2.8	2011-03-13	Run the Junit unit tests in an isolated classloader.
verifier	B	1.0	2010-01-30	Useful for integration tests - verifies the existence of certain conditions.
Packaging types / tools				**These plugins relate to packaging respective artifact types.**
ear	B	2.5	2011-01-29	Generate an EAR from the current project.
ejb	B	2.3	2010-09-18	Build an EJB (and optional client) from the current project.
jar	B	2.3.1	2010-05-21	Build a JAR from the current project.
rar	B	2.2	2007-02-28	Build a RAR from the current project.
war	B	2.1.1	2010-11-03	Build a WAR from the current project.
app-client	B	1.0	2011-03-31	Build a JavaEE application client from the current project.
shade	B	1.4	2010-08-11	Build an Uber-JAR from the current project, including dependencies.

Reporting plugins				**Plugins which generate reports, are configured as reports in the POM and run under the site generation lifecycle.**
changelog	R	2.2	2010-05-28	Generate a list of recent changes from your SCM.
changes	B+R	2.4	2011-01-29	Generate a report from issue tracking or a change document.
checkstyle	B+R	2.6	2010-09-25	Generate a checkstyle report.
doap	B	1.1	2011-01-15	Generate a Description of a Project (DOAP) file from a POM.
docck	B	1.0	2008-11-16	Documentation checker plugin.
javadoc	B+R	2.7	2010-05-04	Generate Javadoc for the project.
jxr	R	2.2	2010-06-05	Generate a source cross reference.
linkcheck	R	1.1	2010-11-13	Generate a Linkcheck report of your project's documentation.
pmd	B+R	2.5	2010-05-04	Generate a PMD report.
project-info-reports	R	2.3.1	2010-12-21	Generate standard project reports.
surefire-report	R	2.8	2011-03-13	Generate a report based on the results of unit tests.
Tools				**These are miscellaneous tools available through Maven by default.**
ant	B	2.3	2009-11-11	Generate an Ant build file for the project.
antrun	B	1.6	2010-10-11	Run a set of ant tasks from a phase of the build.
archetype	B	2.0	2010-10-28	Generate a skeleton project structure from an archetype.
assembly	B	2.2.1	2011-02-27	Build an assembly (distribution) of sources and/or binaries.
dependency	B+R	2.2	2011-02-22	Dependency manipulation (copy, unpack) and analysis.
enforcer	B	1.0	2010-11-08	Environmental constraint checking (Maven Version, JDK etc), User Custom Rule Execution.
gpg	B	1.2	2011-03-24	Create signatures for the artifacts and poms.
help	B	2.1.1	2010-03-26	Get information about the working environment for the project.
invoker	B	1.5	2009-	Run a set of Maven projects and verify

			10-26	the output.
jarsigner	B	1.2	2009-09-30	Signs or verifies project artifacts.
one	B	1.2	2007-09-12	A plugin for interacting with legacy Maven 1.x repositories and builds.
patch	B	1.1.1	2010-01-06	Use the gnu patch tool to apply patch files to source code.
pdf	B	1.1	2009-12-13	Generate a PDF version of your project's documentation.
plugin	B+R	2.7	2011-02-09	Create a Maven plugin descriptor for any mojos found in the source tree, to include in the JAR.
release	B	2.1	2010-10-08	Release the current project - updating the POM and tagging in the SCM.
reactor	B	1.0	2008-09-27	Build a subset of interdependent projects in a reactor
remote-resources	B	1.2	2011-02-27	Copy remote resources to the output directory for inclusion in the artifact.
repository	B	2.3.1	2010-07-21	Plugin to help with repository-based tasks.
scm	B	1.4	2010-08-08	Generate a SCM for the current project.
source	B	2.1.2	2010-05-21	Build a JAR of sources for use in IDEs and distribution to the repository.
stage	B	1.0-alpha-2	2009-07-14	Assists with release staging and promotion.
toolchains	B	1.0	2009-11-01	Allows to share configuration across plugins.
IDEs				**Plugins that simplify integration with integrated developer environments.**
eclipse	B	2.8	2010-02-25	Generate an Eclipse project file for the current project.
idea	B	2.2	2008-08-08	Create/update an IDEA workspace for the current project (individual modules are created as IDEA modules)

2. Programmation d'un plugin pour maven

2.1. Plugin maven développé en java ("mojo")

Plugin "maven" basé sur java ("mojo")

```
/**
 * @goal touch
 * @phase process-sources
 */
class XxxMojo extends AbstractMojo {

/**
* @parameter expression="${project.build.directory}"
* @required
*/
private File outputDirectory;

public void execute(){
      ...
      }
}
```

NB: **Un plugin important (packagé comme un ".jar") est souvent constitué de plusieurs classes complémentaires** :

- classe "XxxMojo1" pour le but 1
- classe "XxxMojo2" pour le but 2
- classe "XxxMojoN" pour le but N
- ....
- classes "CommonUtil1" , "CommonUtilN" pour le code commun partagé entre les différents "buts/goals" .

Création d'un projet "plugin":

```
mvn  archetype:generate
        -DarchetypeArtifactId=maven-archetype-mojo
        -DgroupId=com.mycompany.plugin1
        -DartifactId=myplugin1-maven-plugin
```

==> Coder/Paramétrer la classe java
et construire le plugin via "**mvn install**"

Utilisation (invocation) directe:

```
REM  mvn goupIdOfPlugin:articfactIdOfPlugin:goalName
mvn  com.mycompany.plugin1:myplugin1-maven-plugin:touch
```

Utilisation dans la construction d'un autre projet:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany.app1</groupId>
  <artifactId>my-java-app1</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  ...
  <dependencies>...  </dependencies>
   <build>
     <plugins>
       <plugin>
         <groupId>com.mycompany.plugin1</groupId>
         <artifactId>myplugin1-maven-plugin</artifactId>
         <version>1.0-SNAPSHOT</version>
         <executions>
          <execution>
          <!-- <phase>process-sources</phase> already defined with @phase process-sources -->
          <!-- <configuration>
           </configuration> -->
          <goals>
             <goal>touch</goal>
          </goals>
         </execution>
       </executions>
      </plugin>
    </plugins>
  </build>

</project>
```

2.2. Plugin maven basé sur ant

Plugin "maven" basé sur l'intégration d'un script "ant"

Dans ***src/main/scripts***:

xxx.build.xml (le script ant)

xxx.mojos.xml *(le document de liaison "maven goal – ant target")*

```
<pluginMetadata>
 <mojos> <mojo>
   <goal>hello</goal>
   <call>hello</call> <!-- Ant -->
   <description> ...</description>
  </mojo> </mojos>
</pluginMetadata>
```

pom.xml

```
<project>... <packaging>maven-plugin</packaging>
 <dependencies> <dependency>
   <groupId>org.apache.maven</groupId><artifactId>maven-script-ant</artifactId>
   <version>2.0.6</version> </dependency> </dependencies>
  <build><plugins> <plugin>
    <artifactId>maven-plugin-plugin</artifactId> <version>2.5</version>
     <dependencies> <dependency>
      <groupId>org.apache.maven.plugin-tools</groupId>
      <artifactId>maven-plugin-tools-ant</artifactId> <version>2.5</version>
     </dependency></dependencies>
    <configuration> <goalPrefix>hello</goalPrefix> </configuration>
   </plugin> </plugins> </build></project>
```

XI - GConf. maven (lien avec SVN, ...)

1. Plugins "scm" et "release" de maven

1.1. Liaison avec un référentiel de code source SVN

Lien entre "maven" et le référentiel de code source (scm)

```
<project>  ...
  <scm>    < !-- scm = configuration utilisée par les plugins "scm" et "release" -->
    <!-- ro -->
    <connection>scm:svn:http://127.0.0.1/svn/my-project</connection>

    <!-- rw -->
    <developerConnection>scm:svn:https://127.0.0.1/svn/my-project
    </developerConnection>
    <tag>HEAD</tag>
    <url>http://127.0.0.1/websvn/my-project</url>
  </scm>
  … </project>
```

Ou bien url de type :
scm:git:file:///media/sf_ext/tp/local-git-repositories/env-ic-my-java-app1
ou encore **scm:git:http://www.xy.com/my-repo.git**
ou encore **scm:cvs** :... , **scm:hg**:url_repo_mercurial

Le plugin maven "scm" permet de gérer uniformément "csv" , "svn" , "git" ou "mercurial" .

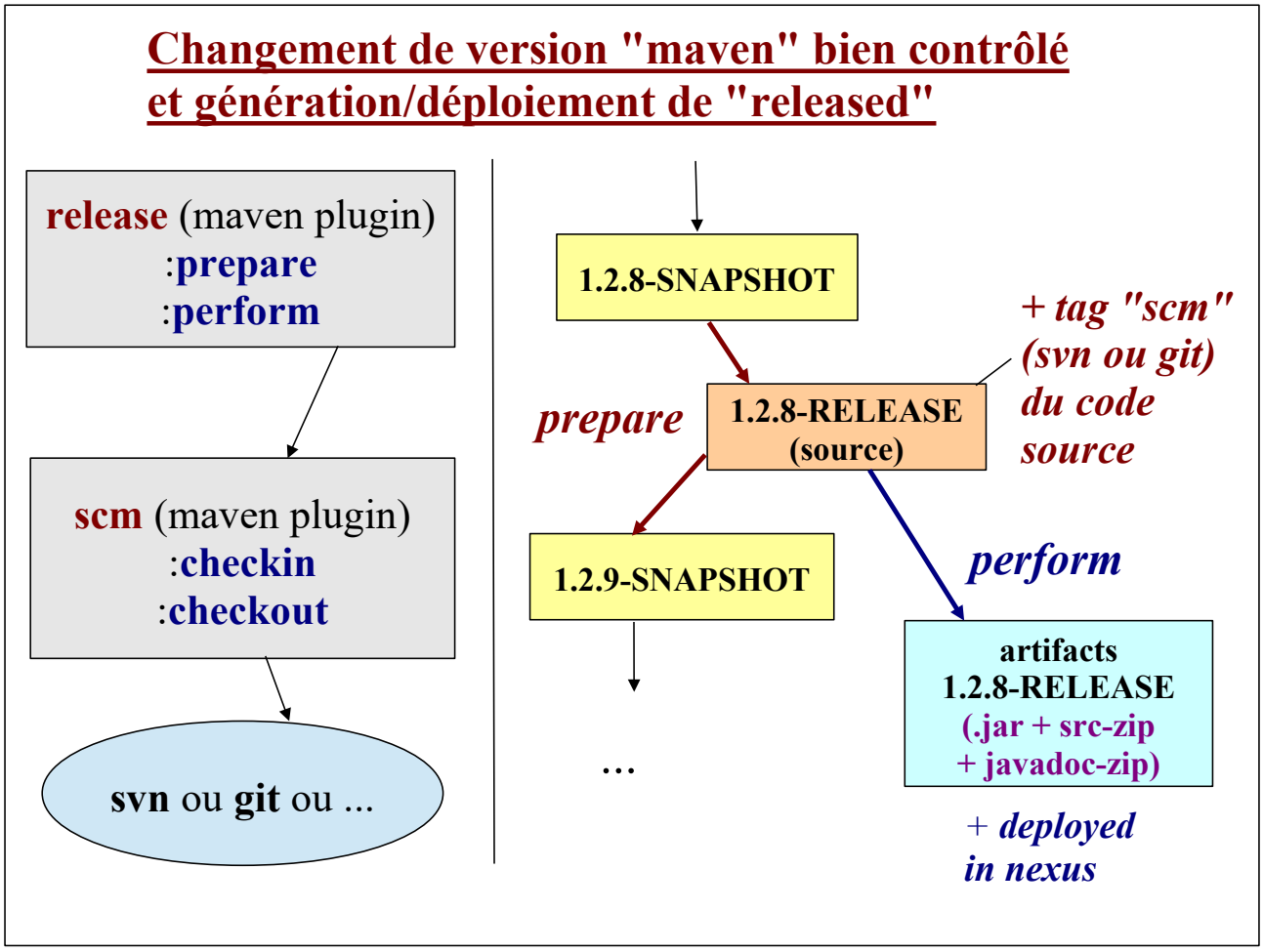
mvn scm:*goal_xy* ...

scm:checkin - command for commiting changes
scm:checkout - command for getting the source code
scm:status - command for showing the scm status of the working copy
scm:update - command for updating the working copy
with the latest changes
scm:tag - command for tagging a certain revision

===> étudier la documentation de référence pour consulter les détails.

NB : Le principal intérêt du plugin "scm" de maven tient dans le fait qu'il est a son tour réutilisé par un plugin de plus haut niveau intitulé "release" .

1.2. Gestion des "releases" pilotées par maven



Le principal intérêt du plugin "scm" de maven tient dans le fait qu'il est a son tour réutilisé par un plugin de plus haut niveau intitulé "release" .
Le plugin "**release**" sert à changer de version "maven" (ex : 1.1-SNAPSHOT vers 1.1-RELEASE) de façon cohérente à tous les niveaux (pom.xml de l'application , des sous modules , dans SCM).

Préparation d'une version "released" via maven

(a) ***mvn_scm_release_prepare_my_java_app1.sh***

```
cd my-java-app1
# vérifications effectuées par release:prepare :
#     * pas de modifications locales (sans commit) : même en TP !!!
#     * pas de dépendances vers des xxx-SNAPSHOT
# informations suggérées / demandées (que l'on peut saisir si pas -B):
#     * new mvn/pom relased version (ex: 1.2.8.-RELEASE)
#     * new SCM(git or svn or ...) released tg version (ex: my-java-app1-v1.2.8)
#     * new mvn/pom developpement version (ex: 1.2.9-SNAPSHOT)
# actions qui seront exécutées (si aucune erreur):
#     * met à jour "version realeased" dans le pom
#       (après sauvegarde dans pom.xml.releaseBackup)
#     * build & test with new version (clean verify)
#     * tag and commit released version in SCM(git or svn or ...)  et génère des lignes
#       dans release.properties (pour futur release:perform ou  release:rollback)
#     * update to new developpement version  (ex: 1.2.9-SNAPSHOT) in the pom
#     * commit in SCM
mvn  -B  release:prepare
# -B or --batch-mode is for non interactive mode (utile pour integration continue)
echo "fin"; read fin
```

Génération/construction d'une version "released" via maven

(b) ***mvn_scm_release_perform_my_java_app1.sh***

```
cd my-java-app1
# a lancer après "mvn release:prepare"
# utilise release.properties ( normalement généré par release:prepare) pour :
#       vérifier que  release:prepare s'est déroulé sans erreur
#       récupération / checkout (from "tag" ) des sources de la version "released"
#       depuis le scm   (svn ou git ou …)  et construction des "artifacts".
#       déploiement des "artifacts" en version"released" vers le référentiel maven
#       (ex: nexus)
#       trois ".jar" sont construits et déployés (artifact.jar , artifact-sources.jar ,
#                                                 artifact-javadoc.jar)
mvn release:perform
echo "fin"; read fin
```

NB: il existe "mvn release:rollback" en cas d'échec
de "mvn release:prepare" .

XII - Maven et intégration continue

1. Intégration continue

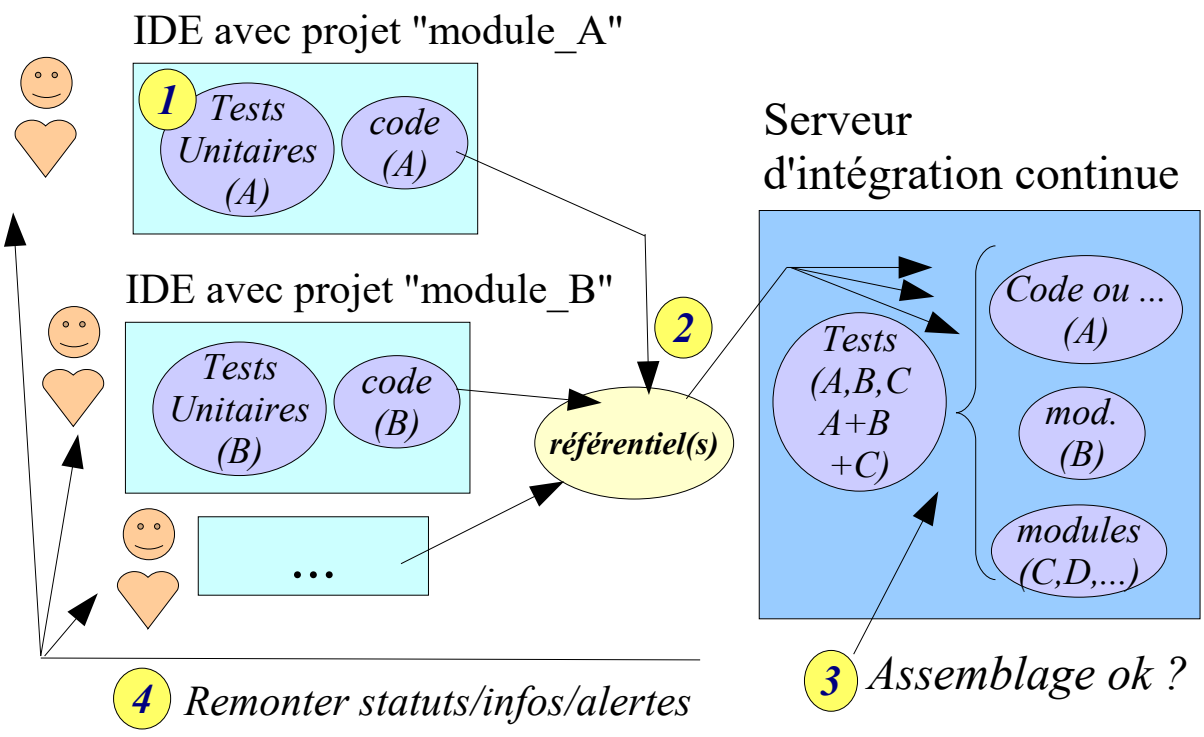
Intégration continue

Principes (intégration continue):
En plus des tests unitaires (à portée limitée à un module) , il est généralement nécessaire de ***régulièrement tester l'ensemble d'une application (tests d'intégration)***.

Il existe pour cela, des produits spécialisés dit "***d'intégration continue***" dont les principales fonctionnalités sont:

* récupérer les différents modules (src ou jar) selon version.
* recompiler certaines parties pour valider les liaisons.
* lancer quelques tests pour valider le bon comportement de l'assemblage (dans le cadre d'une certaine nouvelle version).
* remonter des statuts/alertes aux développeurs/... concernés (ex: succès / échecs des tests , nombre d'erreurs , ...).

Intégration continue



2. Maven et l'intégration continue

Maven et intégration continue

Principaux logiciels d'intégration continue (monde Java):
- **CruiseControl** (le pionnier)
- **Hudson** (évolution récente: **Jenkins**)
- **Continuum** (apache)

Ces différents logiciels sont (entre autres) capables de **s'appuyer sur "maven"** pour automatiser la récupération des nouvelles versions des modules, les compilations et les lancements des tests.

....

--> consulter éventuellement une des annexes pour approfondir la mise en oeuvre d'un serveur d'intégration continue.

NB: le serveur d'intégration "***Hudson/Jenkins***" est aujourd'hui l'un des plus populaire car il très facile à installer/configurer/utiliser .

NB : Le serveur **Jenkins** reste aujourd'hui l'un des plus utilisé dans un contexte interne d'entreprise.

Ceci dit , l'entreprise **GitLab** (concurrente de GitHub) propose des fonctionnalités d'intégration continue d'un bon niveau (semblable au pipeline de jenkins) et gérant bien le contexte du cloud computing.

En gros, avec **Jenkins** on fait un peu ce que l'on veut (via une multitude de plugins) mais il faut configurer son serveur Jenkins.

Avec **GitLab CI** on dispose d'une solution un peu plus clef en main avec avec certaines limitations ou dépendances.

ANNEXES

XIII - Annexe – variables "maven" et versions

1. Variables et versions (maven)

1.1. Variables prédéfinies de maven

NB: Tous les éléments (xml) présents dans le fichier ***pom.xml***, peuvent être référencés via le préfixe "**project.**" (ou l'ancien "pom." maintenant obsolète).
D'autre part, tous les éléments (xml) présents dans le fichier ***settings.xml***, peuvent être référencés via le préfixe "***settings.***"

Depuis Maven 3.0, toutes les propriétés "`pom.*`" sont "deprecated".
Il faut utiliser les propriétés en "`project.*`" à la place .

variables	***Significations (contenus)***
${basedir}	Répertoire contenant le fichier pom.xml
${version} (équivalent à ${project.version})	Version du projet courant
${project.build.directory}	Répertoire "target"
${project.build.outputDirectory}	Répertoire "target/classes"
${project.build.finalName}	Nom du fichier créé (xxx.war , xxx.jar)
${project.xxx.yyy}	Valeur de <xxx><yyy>...</yyy></xxx> dans pom.xml
...	
${settings.localRepository}	Référentiel local de l'utilisateur
${settings.xxx.yyy}	Valeur de <xxx><yyy>...</yyy></xxx> dans settings.xml
...	
${env.XXX}	Valeur de la variable d'environnement XXX
...	
${java.home} , ${java.version} , ...	JRE_HOME , ...
${user.name} , ${user.home} , ...	Toutes les "propriétés systèmes" de java
...	
`${project.parent.xxx}`	Propriétés du projet "parent"
...	

1.2. Versions des artifacts "maven"

Par convention, **une pré-version en cours de développement d'un projet** voit son numéro de version suivi d'un **-SNAPSHOT**.

Dans la gestion des dépendances, Maven va chercher à mettre à jour les versions SNAPSHOT régulièrement pour prendre en compte les derniers développements.

Utiliser une version SNAPSHOT permet de bénéficier des dernières fonctionnalités d'un projet, mais en contre-partie, cette version peut être appelée à être modifiée de façon importante, sans aucun préavis.

Exemples:

0.0.1-SNAPSHOT
1.0-SNAPSHOT
2.0-SNAPSHOT

Structure habituelle d'un numéro de version:

<major>.<mini>[.<micro>][-<qualifier>[-<buildnumber>]]

Incréments
Major : changement majeur
pas de rétro-compatibilité (descendante) garantie
Mini : ajouts fonctionnels
rétro-compatibilité garantie
Micro : maintenance corrective (*bug fix*)

1.3. Création d'une "release" en ligne de commande via le plugin "release"

La page ***guide-releasing.html*** de la documentation de référence "maven" explique comment créer proprement une "release" via un plugin adéquat .

2. Bonnes pratiques

Les noms des packages java d'une application doivent idéalement commencer par la valeur du "groupId" .

Exemple : si groupId = "*tp.myapp.xy*"
alors packages
"*tp.myapp.xy*.itf.domain.dto" , "*tp.myapp.xy*.itf.domain.service" ,
"*tp.myapp.xy*.impl.domain.service" , "*tp.myapp.xy*.impl.persistence.dao" , "
tp.myapp.xy.impl.persistence.entity" , …

Décomposer une application en plein de sous modules et de projet annexes :

Exemple :

app
- app-services-itf *(.jar , interfaces des services et structures dto)*
- app-services-local-impl *(.jar , implémentation locale avec spring/hibernate/jpa)*
- app-services-remote-delegate *(.jar ,business delegate vers web services distants)*
- app-web *(.war , ihm web)*

my-test-framework *(.jar , petit framework de test basé sur Junit et DBUnit)*
my-persistence-framework *(.jar , DAO génériques basés sur Hibernate ou JPA)*
my-xy-framework *(.jar , autre petit framework réutilisable)*

XIV - Annexe – Plugin "cargo"

1. Déploiement Jee avec CARGO (via maven)

Le produit "CARGO" de "CodeHaus" permet d'effectuer des déploiements d'application "Jee" vers différents serveurs/"conteneurs web" : Tomcat, JBoss , WebSphere , WebLogic .

CARGO peut (selon le type de serveur) gérer le conteneur web dans 1,2 ou 3 des trois grands modes suivants:

- "**embbeded**" (en mémoire dans même JVM -- possible avec "Jetty")
- "**installed**" (local)
- "**remote**"

Le déclenchement de "cargo" peut s'effectuer d'une des 3 façons suivantes:

- par code java (via API spécifique)
- via un script "ant
- via maven

La suite de cette annexe présente l'utilisation de "cargo" via maven.

Le déclenchement (depuis maven) du déploiement JEE (et du lancement du serveur) se fait via:

```
mvn  cargo:start
```

C'est à peu près l'équivalent "maven" du "run as / run on server" d'eclipse.

1.1. Configuration pour Tomcat 6 (en mode local/installed)

```
...
<build>
  <plugins>
    <plugin>
      <groupId>org.codehaus.cargo</groupId>
      <artifactId>cargo-maven2-plugin</artifactId>
      <version>1.4.12</version>
       <!-- tomcat6x or tomcat7x (not tomcat6 / tomcat7) -->
       <!-- installed or remote : ok with tomcat , embedded ok only with Jetty  -->
       <!-- configuration "existing" nécessaire (en plus de installed) !!! -->
       <configuration>
         <wait>true</wait>          <!-- waiting for Ctrl-C -->
         <container>
            <containerId>tomcat6x</containerId>
             <type>installed</type>
             <home>C:\Prog\java\ServApp\Tomcat_6.0</home>
         </container>
         <configuration>
               <type>existing</type>
               <home>C:\Prog\java\ServApp\Tomcat_6.0</home>
         </configuration>
      </configuration>
    </plugin>
  </plugins>
</build>
```

1.2. Configuration pour Jetty (en mode "embedded")

```
...
```

```
<dependencies>
  <dependency>
      <groupId>javax.el</groupId>   <artifactId>el-api</artifactId>
      <version>2.2</version>    <scope>provided</scope>
 </dependency>
 <dependency>
      <groupId>org.glassfish.web</groupId>   <artifactId>el-impl</artifactId>
      <version>2.2</version>   <scope>provided</scope>
 </dependency>
     ....
<dependencies>

<build>
 <plugins>
  <plugin>
    <groupId>org.codehaus.cargo</groupId>
    <artifactId>cargo-maven2-plugin</artifactId>
    <version>1.4.12</version>
     <configuration>
       <wait>true</wait>          <!-- waiting for Ctrl-C -->
       <container>
          <containerId>jetty8x</containerId>
          <type>embedded</type>
           <dependencies>
            <dependency>
                  <groupId>javax.el</groupId>
                    <artifactId>el-api</artifactId>          < !--reference sans version -->
              </dependency>
              <dependency>
                    <groupId>org.glassfish.web</groupId>
                      <artifactId>el-impl</artifactId>
             </dependency>
             <dependency>
                    <groupId>javax.servlet.jsp</groupId>
                    <!-- <artifactId>jsp-api</artifactId> ancienne version -->
                    <artifactId>javax.servlet.jsp-api</artifactId>
              </dependency>
               <dependency>
                     <groupId>javax.servlet</groupId>
                     <artifactId>jstl</artifactId>
                 </dependency>
          </dependencies>
       </container>
    </configuration>
  </plugin>
 </plugins>
</build>
```

1.3. Configuration pour Jboss 5.1 (en mode local/installed)

```
<!-- ..... dans le pom.xml du sous projet "....-ear" ..... -->
<build>
   <plugins><plugin>
       <groupId>org.apache.maven.plugins</groupId>
```

```
        <artifactId>maven-ear-plugin</artifactId>
        <version>2.3.1</version>
        <configuration>
                <generateApplicationXml>true</generateApplicationXml>
                <includeJar>false</includeJar>
                <defaultLibBundleDir>lib</defaultLibBundleDir>
                <modules>
                        <webModule>
                                <groupId>com.mycompany.jee5app1</groupId>
                                <artifactId>my-jee5app1-web</artifactId>
                                <contextRoot>my-jee5app1-web</contextRoot>
                        </webModule>
                        <ejbModule>
                                <groupId>com.mycompany.jee5app1</groupId>
                                <artifactId>my-jee5app1-ejb</artifactId>
                        </ejbModule>
                </modules>
        </configuration>
</plugin>
<plugin>
    <groupId>org.codehaus.cargo</groupId>
    <artifactId>cargo-maven2-plugin</artifactId>
    <version>1.4.12</version>
     <configuration>
       <wait>true</wait>          <!-- waiting for Ctrl-C -->
       <container>
          <containerId>jboss51x</containerId>
           <type>installed</type>
           <home>C:\Prog\java\ServApp\jboss-5.1.0.GA</home>
       </container>
       <configuration>
             <type>existing</type>
             <home>C:\Prog\java\ServApp\jboss-5.1.0.GA\server\default</home>
       </configuration>
    </configuration>
  </plugin>
 </plugins>
 <!-- finalName : "my-jee5app1" (.ear) not "my-jee5app1-ear" (.ear) -->
 <finalName>my-jee5app1</finalName>
</build>
```

Autres possibilités/configurations

--> voir le site de référence du produit "cargo"

XV - Annexe – "B.O.M."

1. Gestion avancée des dépendances (BOM)

1.1. DependencyManagement

Une expression simple des dépendances s'effectue via une liste de <dependency> au sein de la partie
<project><dependencies> ... </dependencies></project> .

Il est possible d'exprimer certaines dépendances en deux phases:

- détailler certaines dépendances (à réutiliser) dans la partie <dependencyManagement>
- faire référence (sans détails) à ces dépendances dans la partie project/dependencies .

Détails apparaissant généralement dans la partie "**dependencyManagement**":

- version précise
- portée (scope)
- exclusions
- (et groupId/artifactId) , ...

Elements à préciser au sein d'une référence à une dépendance :

- groupId , artifactId
- ~~version~~
- packaging (si différent de "jar")

NB:

- La partie "**dependencyManagement**" n'est réellement intéressante que si elle peut être partagée (ou factorisée) .
- La réutilisation d'une partie "dependencyManagement" passe par l'une des deux solutions suivantes:
  * héritage de configuration depuis un projet "parent".
  * import de dépendances depuis un projet "BOM"

1.2. import de gestion de dépendances et "BOM"

"**BOM**" signifie "***B**ill **O**f **M**aterials*" (soit à peu près en français "*note de configuration technique*").
Il s'agit d'un projet spécial (ayant le **packaging** fixé à "**pom**" et hébergeant au moins un bloc "**dependencyManagement**" prévu pour être ultérieurement importé .

Exemple1 (concret):

Utilisation de richeFaces4 (de jboss) avec versions paramétrées via un "bom":

```
<repositories>
        <!-- specific repository needed for richfaces  4 (different for 3.3) -->
            <repository>
                    <id>jboss.org</id>
                    <url>http://repository.jboss.org/nexus/content/groups/public-jboss/</url>
            </repository>
</repositories>

    <properties>
        <org.richfaces.bom.version>4.0.0.Final</org.richfaces.bom.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                    <groupId>org.richfaces</groupId>
                    <artifactId>richfaces-bom</artifactId>
                    <version>${org.richfaces.bom.version}</version>
                    <scope>import</scope>
                    <type>pom</type>
            </dependency>
        </dependencies>
    </dependencyManagement>

<dependencies>
        ...
  <dependency>
      <groupId>org.richfaces.ui</groupId>
      <artifactId>richfaces-components-ui</artifactId>
      <!-- pas de version à choisir ici , déjà précisée dans le BOM -->
   </dependency>
   <dependency>
      <groupId>org.richfaces.core</groupId>
      <artifactId>richfaces-core-impl</artifactId>
      <!-- pas de version à choisir ici , déjà précisée dans le BOM -->
   </dependency>
<dependencies>
...
```

Exemple2 ("BOM" maison pour slf4j-log4j):

```
<?xml version="1.0"?>
<project xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd" xmlns="http://maven.apache.org/POM/4.0.0"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
 <modelVersion>4.0.0</modelVersion>
 <artifactId>slf4j-log4j-bom</artifactId>
 <groupId>com.mycompany.bom.util</groupId>  <version>1.0-SNAPSHOT</version>
 <name>slf4j-log4j-bom</name>
 <packaging>pom</packaging>

 <dependencyManagement>
 <dependencies>
        <dependency>
        <groupId>log4j</groupId>
        <artifactId>log4j</artifactId>
        <version>1.2.17</version>
        <scope>runtime</scope>
        </dependency>

        <dependency>
                 <groupId>org.slf4j</groupId>
                <artifactId>slf4j-api</artifactId>
                <version>1.7.7</version>
                <scope>compile</scope>
        </dependency>

        <dependency>
                <groupId>org.slf4j</groupId>
                <artifactId>slf4j-log4j12</artifactId>
                <version>1.7.7</version>
                <scope>runtime</scope>
        </dependency>

  </dependencies>
 </dependencyManagement>
</project>
```

```
<!-- à réutiliser depuis un autre pom.xml via :
        <dependencyManagement>
                <dependencies>

                        <dependency>
           <artifactId>slf4j-log4j-bom</artifactId>
           <groupId>com.mycompany.bom.util</groupId>
           <version>1.0-SNAPSHOT</version>
           <scope>import</scope>
           <type>pom</type>
         </dependency>
      <dependency> ... </dependency>
       </dependencies>
 </dependencyManagement>
```

```
  ET AUSSI :
 <dependencies>
       <dependency>
                 <groupId>log4j</groupId>
                    <artifactId>log4j</artifactId>
              </dependency>
               <dependency>
                   <groupId>org.slf4j</groupId>
                      <artifactId>slf4j-log4j12</artifactId>
             </dependency>
             <dependency>
             <groupId>org.slf4j</groupId>
                   <artifactId>slf4j-log4j12</artifactId>
                </dependency>
</dependencies>
-->
```

NB : cette notion de pack ne fonctionne bien que pour une portée (scope) = compile .
La transitivité est parfaitement bien gérée par maven en scope=compile .
En scope="runtime" , la transitivité n'est pas automatique et la notion de pack est inopérante .

XVI - Annexe – Hudson - Jenkins

1. Premiers pas avec Hudson/Jenkins

1.1. Présentation et installation de kenkins

Premiers pas avec Jenkins

Jenkins est actuellement un **logiciel d'intégration continue très en vogue** car il est **très simple à configurer et à utiliser**.

Installation de Jenkins :
Recopier **jenkins.war** dans **TOMCAT_HOME/webapps** (avec un éventuel Tomcat dédié à l'intégration continue configuré sur le port 8585 ou autre).

Etant donné que la configuration de jenkins ne nécessite pas de base de données relationnelle (mais de simples fichiers sur le disque dur) , il n'y a rien d'autre à configurer lors de l'installation .

Url de la console "jenkins" :

http://localhost:8585/jenkins

Premier menu à activer :

Administrer Jenkins / Configurer le système



1.2. Installation/démarrage de Jenkins sans tomcat

NB : il est également possible de démarrer une version récente de jenkins sans serveur tomcat via un script de ce genre :

startJenkins.bat

```
set JAVA_HOME=C:\Program Files\Java\jdk-17
set MVN_HOME=C:\Prog\apache-maven-3.8.4
set PATH="%JAVA_HOME%/bin";"%MVN_HOME%/bin";%PATH%
REM  java -jar jenkins.jar -D"hudson.plugins.git.GitSCM.ALLOW_LOCAL_CHECKOUT=true"
java -jar jenkins.war --httpPort=8585
```

et après un tel lancement l'url menant à la console jenkins sera simplement **http://localhost:8585**

1.3. Configuration nécessaire lors du premier démarrage

Lire le mot de passe temporaire à la console lors du premier démarrage (ex: *d1223a3ba2a44d079ecb7deec0625de8*)
et **reporter/recopier celui-ci dans la console de jenkins**

Installer quelques plugins fondamentaux (ceux qui sont suggérés)

Configurer un compte principal (administrateur) pour les futurs démarrages :
*par exemple username=**admin** password=**admin123***

En configuration de TP, choisir l'URL http://localhost:8585

1.4. Installation ou mise à jour de plugins pour Jenkins

Menu **"tableau de bord" / "Administrer Jenkins" / "Gestion des plugins"**

1.5. Configuration élémentaire d'une tâche "jenkins / freeStyle"

Menu **"tableau de bord" / "Nouveau item"**
puis **:**
- donner un nom (ex : *jobXy*)
- choisir souvent "projet free-style" pour les cas simples/ordinaires
- OK

Dans la partie "**gestion du code source**", choisir généralement :
- **GIT**
et préciser l'url du référentiel git (par exemple ***https://github.com/…./repoXy.git***)
Attention: les versions récentes de Jenkins n'acceptent des URLS de type file:///c:/xx/yy *qu'avec l'option -D"hudson.plugins.git.GitSCM.ALLOW_LOCAL_CHECKOUT=true" à fixer au démarrage*
et dans la partie "branch to build" on pourra par exemple choisir ***/master** ou ***/main** .

Dans la partie "**build**" , choisir généralement :
- **Invoquer les cibles Maven de haut niveau**
et préciser la "***cible (ou goal)***" maven à déclencher (*ex* : **clean package**)

*NB: si le projet maven à construire est dans un sous répertoire du référentiel git (cas pas très conseillé mais admis), alors au niveau du build on peut préciser un chemin menant au pom.xml de type **sous_rep1/pom.xml** ou bien sous_rep1/sous_sous_rep2/pom.xml dans **config avancée** .*

Sauvegarder assez rapidement ces configurations essentielles.
Les configurations secondaires annexes pourront être ajoutées ultérieurement

Lancement sur demande d'un "build" (associé à un job jenkins configuré)



Affichage des résultats via la console de jenkins

La logique de navigation/sélection de jenkins est la suivante :

Jenkins (server) > **Job (name/type/config)** > **number of instance (with status/results)**

Exemple: 

Après avoir sélectionné un des niveaux , on accède à un menu (coté gauche) pour :
* créer/activer de nouveaux éléments
* (re)configurer plus en détails l'élément sélectionné
* afficher des détails sur l'élément sélectionné
* ...

Concernant les résultats d'un build, la partie la plus intéressante est souvent "***sortie console***" :




```
------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO]
------------------------------------------------------------
[INFO] Total time: 24.003s
[INFO] Finished at: Tue Apr 21 14:51:42 CEST 2015
[INFO] Final Memory: 12M/32M
[INFO]
------------------------------------------------------------
```

1.6. job/item de type "pipeline"

Un job de type "**pipeline**" est assez conseillé au sein de jenkins car :
- il est grandement configurable/extensible
- il peut comporter plusieurs étapes (enchaînement en pipeline)
- un script de type "pipeline_jenkins" peut être placé dans un référentiel git et ainsi être versionné

Exemple de pipeline simple pour un projet java :

```
pipeline {
   agent any
   stages {
      stage('SCM') {
         steps {
           git url : 'https://github.com/didier-tp/test_junit.git' , branch : 'main'
         }
      }
     stage('Build') {
         steps {
            script {
              dir('with_mockito') {
                 //sh "mvn -Dmaven.test.failure.ignore=true clean package"
                 bat "mvn -Dmaven.test.failure.ignore=true clean package"
                 }
            }
         }
        post {
           // If Maven was able to run the tests, even if some of the test failed, .....
           success {
               script {
                dir('with_mockito') {
                      bat "mvn javadoc:javadoc"
                      echo "javadoc generated , ..."
                 }
               }
           }
        }
     }
      stage('sonar scan or prepa_docker') {
          steps {
             echo "sonar scan ou construction container docker (souvent sous linux)"
          }
      }
   }
}
```

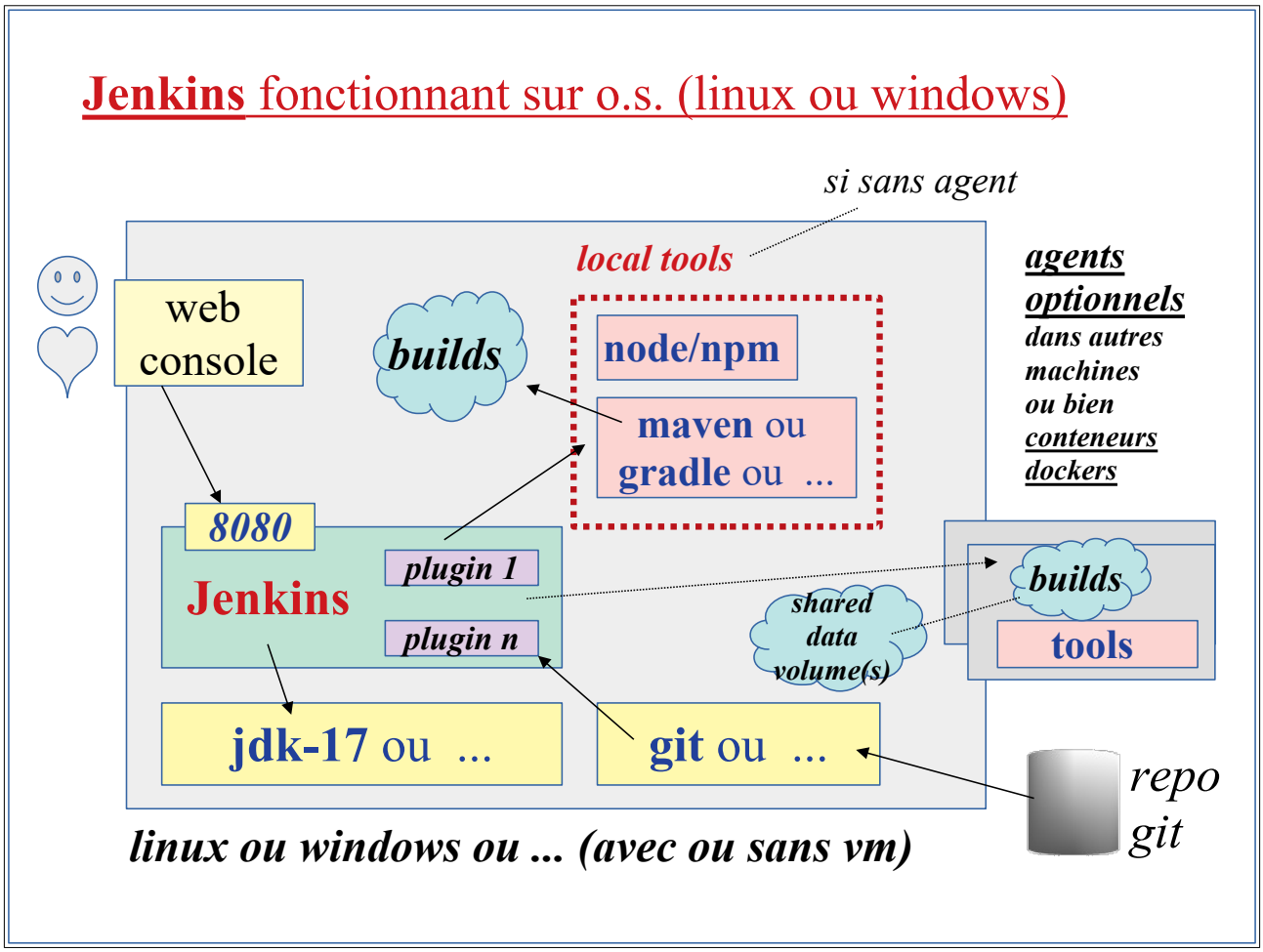
NB: **pas besoin de**

~~script {~~
~~dir('with_mockito') {~~
~~}~~
~~}~~

si pom.xml directement à la racine du référentiel git

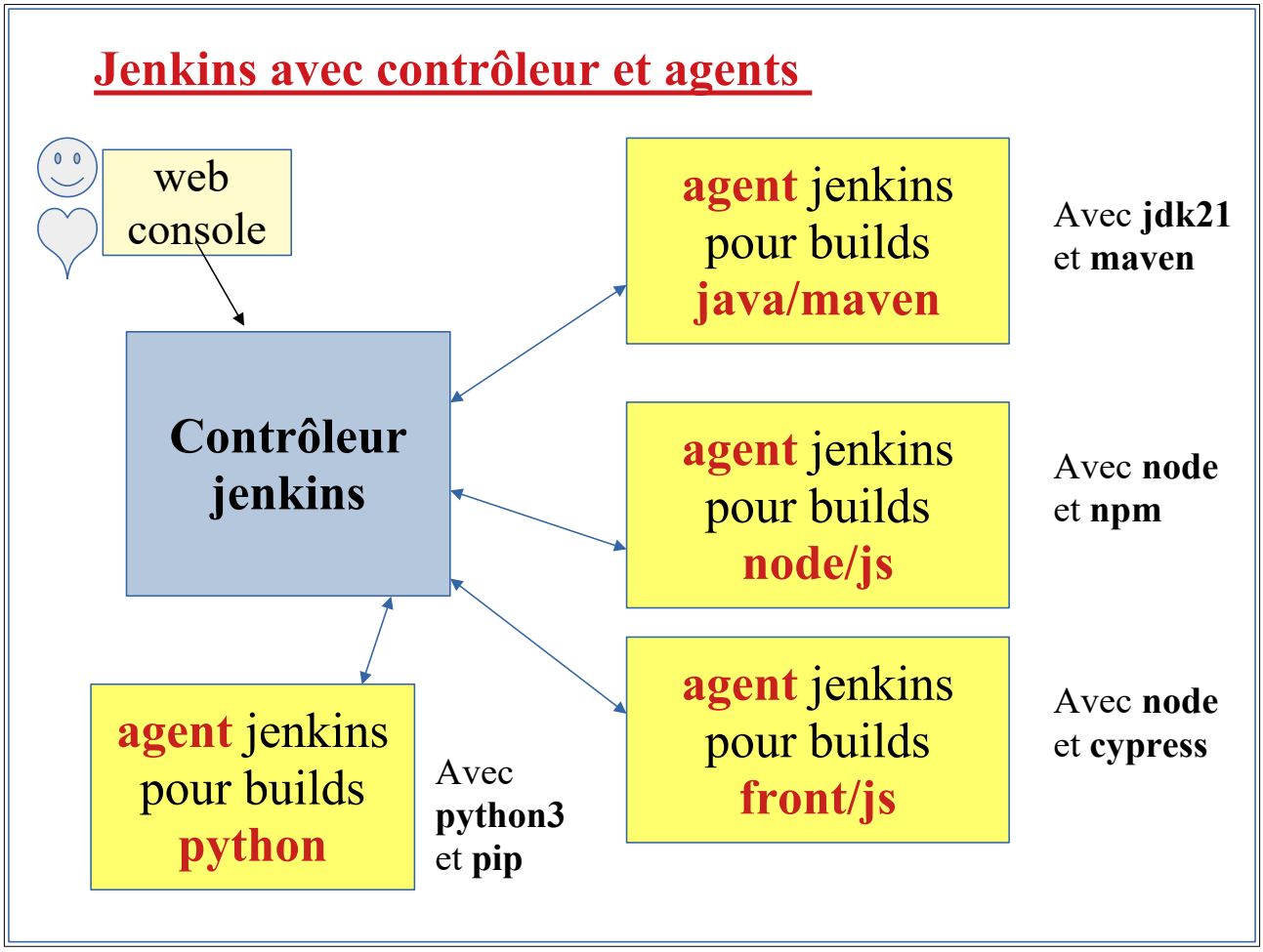
2. Architecture de Jenkins (variantes)

2.1. Jenkins seul sans agent



Dans ce mode (assez basique), le serveur Jenkins fonctionne en mode mono bloc et il doit être accompagné de nombreux outils (jdk, maven , node , npm , …) si l'on souhaite pouvoir déclencher des "builds" avec plusieurs technologies (java, js, python, …).

2.2. Jenkins avec agent



Dans ce mode plus sophistiqué (conseillé) , la partie principale de jenkins va déléguer certains builds à des agents spécialisés dans certaines technologies :
- un agent sera spécialisé "builds java/maven" avec outils jdk et mven
- d'autres agents seront spécialisés "builds node/npm" ou "builds python" ou "build frontend" .

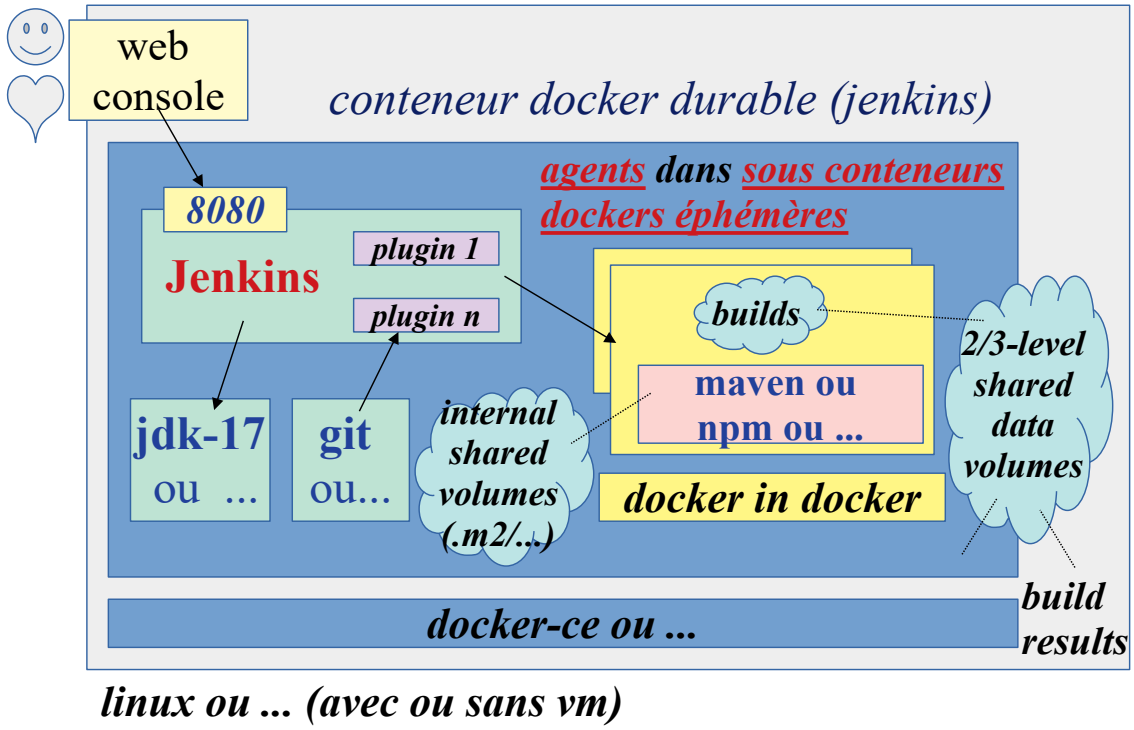
NB : La technologie "docker" peut éventuellement être utilisée pour simplifier la mise en place d'une telle configuration.

Le contrôleur jenkins et des agents peuvent fonctionner au sein de conteneurs "docker" que l'on peut gérer via "docker-compose" .

2.3. Avec agents en mode "DinD" : Docker in Docker

Si le contrôleur "Jenkins" fonctionne au sein d'un conteneur docker , on peut alors avoir des agents qui fonctionnent comme des conteneurs dockers imbriqués (en mode "DinD : docker in docker). Cette solution est assez souple et flexible . Du coté performance , via une optimisation (via partage des images téléchargées en mode "docker in docker") c'est acceptable (après un premier démarrage très lent).

Jenkins avec agents en mode docker-in-docker



Jenkins **pipeline** (*build as code*) with **docker agent**

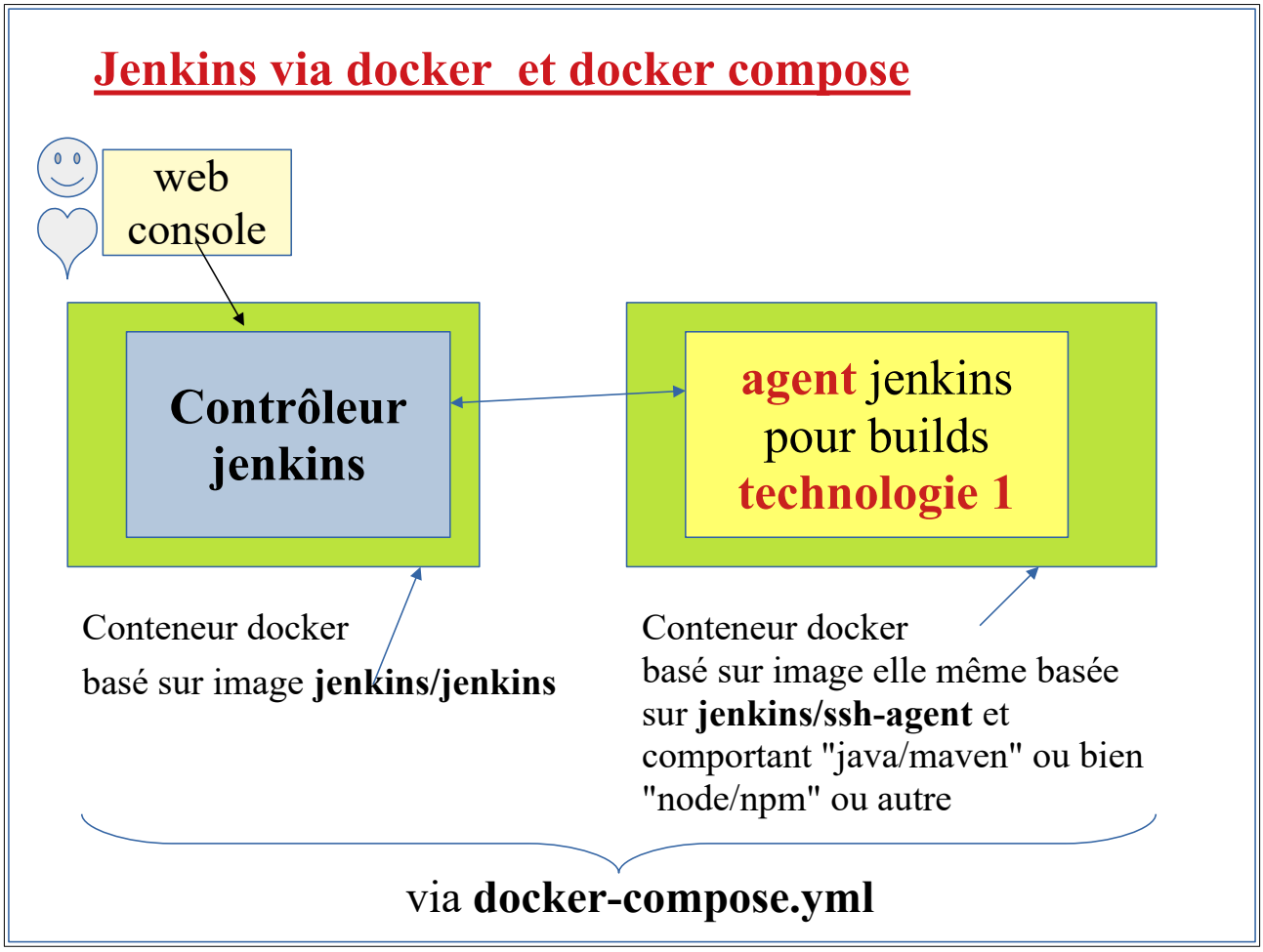
```
pipeline {
  agent {
     docker {
        image 'maven:3-alpine'
        args '-v /root/.m2:/root/.m2'
     }
  }
  stages {
    stage('Build') {
      steps {
        git 'https://github.com/didier-mycontrib/env-ic-my-java-rest-app'
        sh "mvn -Dmaven.test.failure.ignore=true clean package"
      }
      post {
        success {
              echo 'with success'
        }
      }
    }
  }
}
```

image (java ou ...) d'un conteneur docker éphémère (agent)

mapping de volume pour que le repo local .m2/... ne soit pas re-téléchargé à chaque lancement du build

succession d'étapes très flexible

2.4. Avec agents en mode "docker non imbriqué"



Ce mode de fonctionnement peut comporter lui-même plein de variantes :
- conteneurs dockers fonctionnant (ou pas) sur des machines séparées
- en mode "static" ou "cloud élastique"
- …

La technologie "docker compose" sera souvent la bienvenue pour configurer et démarrer un assemblage bien cohérent.

L'exemple officiel "https://github.com/jenkins-docs/quickstart-tutorials" montre une bonne base de configuration "jenkins/docker/docker-compose" pour un bon paquet de technologies (java/main, node/js , python, …) .

3. Préparation machine hôte pour jenkins avec docker

Les premières versions de jenkins (autour des années 2010) fonctionnaient initialement sans docker en s'appuyant directement sur une machine virtuelle java.
Ce mode de fonctionnement est encore possible en 2025.

Aujourd'hui dans les années 2020 et plus, on a tout intérêt à faire fonctionner Jenkins au sein d'un conteneur docker pour simplifier et standardiser l'installation et la maintenance de cet outil CI/CD .

Pour faire fonctionner "jenkins" dans un environnement docker , il faut commencer par mettre en place un environnement docker opérationnel avec au minimum "docker" et "docker compose" .

La première chose à faire consiste donc à préparer un ordinateur compatible docker (linux, mac (avec ou sans VirtualBox ou VmWare, avec ou sans Vagrant) ou windows_10_11_12 + WSL2) .

3.1. éventuelle installation de WSL2 sur une machine windows

WSL = **W**indows **Subsystem** for **L**inux .
La version 2 coïncide à peu près avec l'apparition de windows 11 .

Dans un terminal "powershell as admin"

Enable-WindowsOptionalFeature -Online -FeatureName 'VirtualMachinePlatform' -All

Enable-WindowsOptionalFeature -Online -FeatureName 'Microsoft-Windows-Subsystem-Linux' -All

puis après un éventuel rédémarrage :

installer si besoin la mise à jour du noyau windows en WSL2 :
https://wslstorestorage.blob.core.windows.net/wslblob/wsl_update_x64.msi
ou mieux encore
wsl --update

wsl --set-default-version 2

wsl --install -d Ubuntu

choisir un username/password linux (ex : **didier/pwd**)
(exemple de version de ubuntu : 22.04.3 LTS)
---
exit
wsl --list
Distributions du sous-système Windows pour Linux :
Ubuntu (par défaut)
ubuntu
exit

Un éventuel rédémarrage peut faire du bien ...

Dans powerShell ou CMD :

---
ubuntu
exit
---
ou bien : **menu windows "démarrer"** (tout) / **WSL**

NB: dans certains cas particuliers (WSL2 incompatible avec ordinateur windows) ou bien autres systèmes d'exploitation, on pourra éventuellement installer linux (en version ubuntu ou debian) sur une machine virtuelle elle même prise en charge par virtualBox (ou virtualBox + Vagrant) ou autre.

3.2. Installation de docker dans un env. linux ou compatible

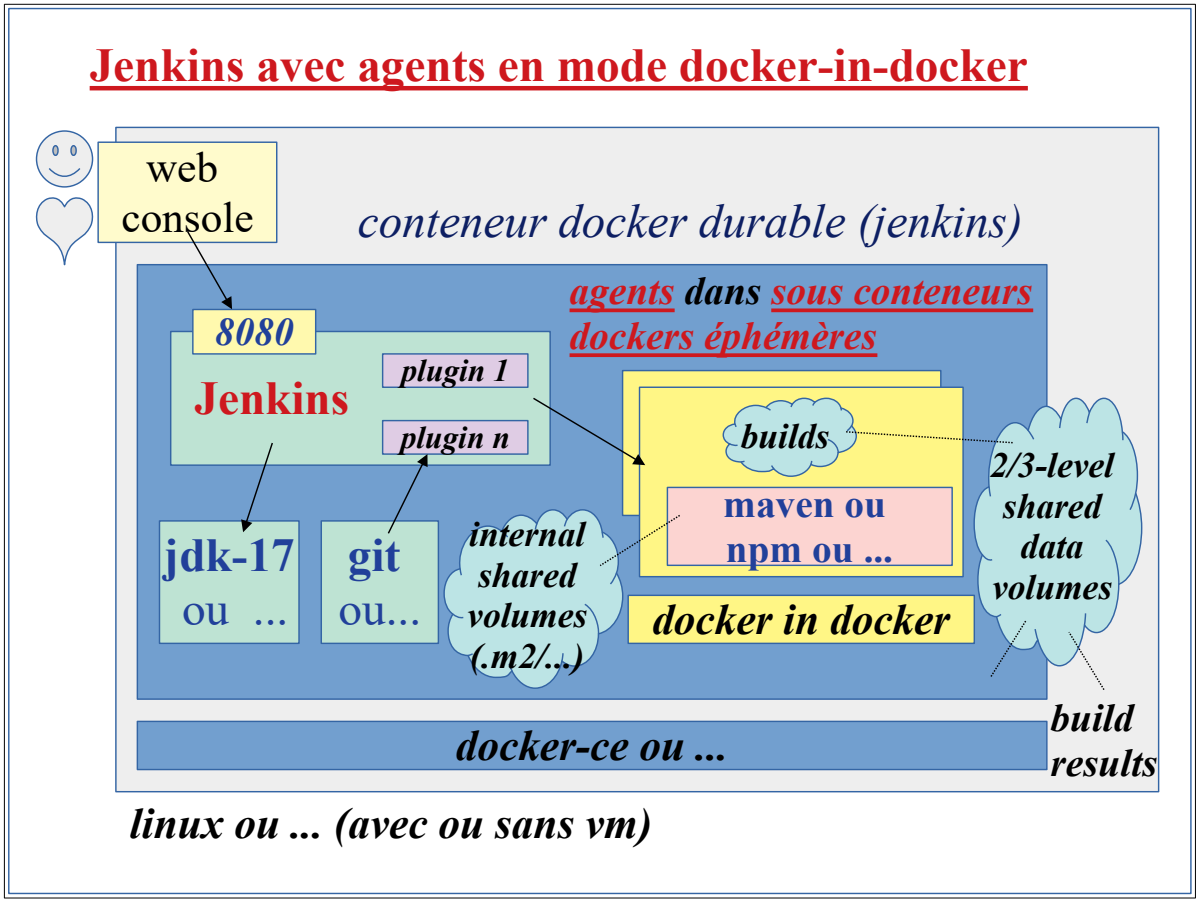
1) installer si besoin GIT (apt install git , git config ...)
et vérifier via git --version

2) se placer quelque part sur la machine virtuelle linux (ex: /home/xyz/local-git-repositories)
puis cloner un référentiel git qui comporte une configuration "docker-jenkins" :
git clone https://github.com/didier-mycontrib/my-jenkins-config
et/ou bien
git clone https://github.com/didier-mycontrib/jenkins-dind (en mode "docker in docker")

3) installer si besoin docker et docker-compose en lançant
sudo sh install-docker.sh (ou un équivalent)
où install-docker.sh est un script que l'on pourra trouver dans
my-jenkins-config ou jenkins-dind /scripts/debian ou bien .../scripts/ubuntu
NB: selon OS exact (debian ou ubuntu) , il faudra peut être se déconnecter/reconnecter
pour que votre compte linux soit bien associé au groupe "docker" permettant
de lancer docker sans sudo.
Vérifications: **docker ps**
docker compose ps

4. Jenkins avec docker en mode "docker in docker"

Pour une première utilisation simple, souple et flexible de jenkins on pourra éventuellement utiliser le mode "docker in docker" présenté ci après :



4.1. Installation de Jenkins (en mode "dind") via docker

Sur une machine (éventuellement virtuelle) linux de type debian ou Ubuntu (fonctionnant par exemple avec VirtualBox && Vagrant ou WSL2) , on pourra enchaîner les installations suivantes:

1) installer si besoin git, docker et docker compose sur l'ordinateur hôte (voir le mode opératoire au sein du premier paragraphe de ce chapitre)

Vérifications:

```
git --version
docker ps
docker compose ps
```

1bis) si ce n'est pas déjà fait se placer quelque part sur la machine virtuelle linux (ex: /home/xyz/local-git-repositories)
puis cloner un référentiel git qui comporte une configuration "docker-jenkins" :
git clone https://github.com/didier-mycontrib/jenkins-dind (en mode "docker in docker")

2) lancer l'installation de jenkins via docker compose en procédant de cette manière:
- se placer dans le répertoire .../local-git-repositories/**jenkins-dind/jenkins-docker-conf**

- vérifier que ce répertoire comporte bien le fichier **docker-compose.yaml** ainsi que le sous répertoire dockerfiles (contenant lui même plein de choses)
- lire éventuellement le fichier README.txt et lancer la commande suivante:
  docker compose up -d
  ...
- vérifier le bon démarrage de jenkins (partie contrôleur)
  via la commande **docker container ls**

3) lancer un navigateur pouvant accéder aux serveurs démarrés au sein de la VM linux
et depuis celui-ci spécifier l'URL suivante **http://localhost:8080**
s'identifier en tant qu'admin via **admin/admin**

---

pour un futur arrêt : *docker compose down*

Quelques configurations clefs :

docker-compose.yaml

```
services:
  jenkins_controller:
    build: dockerfiles/.
    restart: on-failure
    privileged: true
    user: root
    ports:
      - 8080:8080
      - 50000:50000
    container_name: jenkins_controller_container
    volumes:
      - jenkins_home:/var/jenkins_home # Mounts the jenkins_home volume to the /var/jenkins_home path inside the container
      - /var/run/docker.sock:/var/run/docker.sock
      - /usr/bin/docker:/usr/bin/docker


volumes:
  jenkins_home:
```

dockerfiles/**jenkins.yaml**

```
jenkins:
  numExecutors: 4
  securityRealm:
    local:
      allowsSignup: false
      users:
        - id: "admin"
          password: "admin"
  authorizationStrategy: loggedInUsersCanDoAnything
...
```

dockerfiles/**Dockerfile**

```
ARG JENKINS_VERSION=2.504.3

# We then use the official Jenkins image with the specified version as our base image.
FROM jenkins/jenkins:"${JENKINS_VERSION}"

…
COPY plugins.txt /usr/share/jenkins/ref/plugins.txt.override
…
COPY jenkins.yaml /usr/share/jenkins/ref/jenkins.yaml.override
```

dockerfiles/**plugins.txt**

```
…
docker-workflow:621.va_73f881d9232
...
```

Attention: les versions des plugins sont quelques fois à ajuster.
Par exemple, dans un an , si évolution de version et faille de sécurité alors besoin d'ajuster quelques versions de plugin
---
source pour versions récentes/actualisées:
https://github.com/jenkins-docs/quickstart-tutorials/blob/main/dockerfiles/plugins.txt
---
NB: le plugin "Docker pipeline" s'appelle en fait "docker-workflow" (nom à préciser dans plugins.txt)

4.2. Premières manipulations de jenkins en mode "docker in docker" avec un agent docker basé sur java21 et maven

NB: en mode "docker in docker" , via des mappings de répertoires/volumes les éléments dockers vus par la machine hôte seront (par défaut) vus par les conteneurs lancés en mode imbriqués et vice versa .

On peut donc éventuellement lancer l'installation d'une image docker nécessaire à l'exécution d'une application java/maven via de genre de commande :

docker image pull maven:3.9-amazoncorretto-21-debian

Vérifications :

docker container run -t -i maven:3.9-amazoncorretto-21-debian **bash**
java --version
mvn --version
exit

Via la console de jenkins , on peut **créer un nouveau job/item** (par exemple de nom "**essai_dind**") **en mode pipeline** avec ce script (de pipeline) :

```
pipeline {
    agent {
```

```
      docker {
                image 'maven:3.9-amazoncorretto-21-debian'
                args '-v $HOME/.m2:/root/.m2'
               }
   }
   stages {
      stage('Test') {
         steps {
            sh 'mvn --version'
         }
      }
   }
}
```

- **Save**
- **Lancer un build**
- **#1** , console output ⇒ *... Apache Maven 3.9.11 ... Java version: 21.0.8 ...*

4.3. Pipeline CI/CD pour une application java/springBoot :

Exemple parmi plein d'autres variantes envisageables :

Jenkinsfile

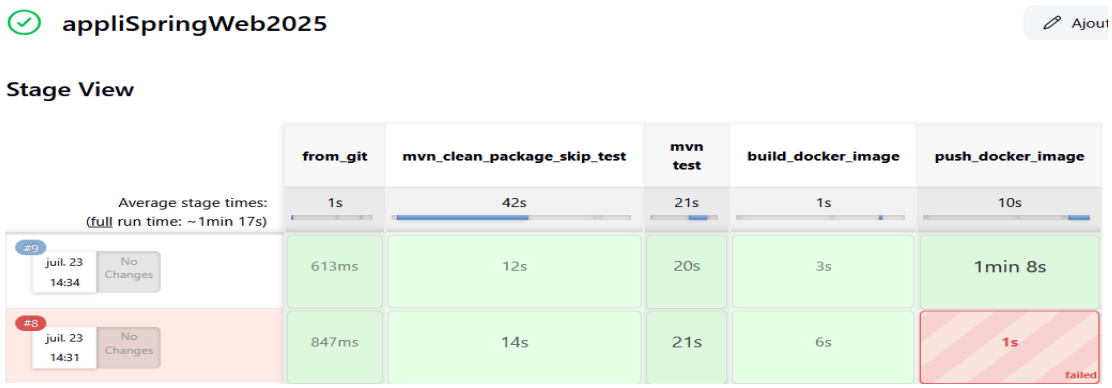
```
pipeline {
   //agent any
   //agent { label 'maven' }
   //agent { label '! without-maven' }
   agent {
      docker { image 'maven:3.9-amazoncorretto-21-debian'
               args '-v $HOME/.m2:/root/.m2'
             }
    }
  environment{
                //NB: credential_dockerhub_didierdefrance69 is ID of credential
                //prepared in "Admin Jenkins / Credentials / system /global"
                dockerhub_credential_id='credential_dockerhub_didierdefrance69'

                //dockerRegistry is dockerhub
                docker_registry= 'https://registry.hub.docker.com'

                docker_image_name='didierdefrance69/appli_spring_v3:1'
        }
   stages {
    stage('from_git') {
        steps {
            git url : 'https://github.com/didier-tp/spring_2025' , branch : 'main'
        }
     }
     stage('mvn_clean_package_skip_test') {
        steps {
                script {
                    dir('tp/mysecurity-autoconfigure') {
                            echo 'mvn clean install mysecurity-autoconfigure (dependency) '
                            sh 'mvn clean install -Dmaven.test.skip=true'
                            }
                    dir('tp/appliSpringWeb') {
                            echo 'mvn clean package '
```

```
                                sh 'mvn clean package -Dmaven.test.skip=true'
                                }
                        }
                }
        }
        stage('mvn test') {
            steps {
                        script {
                        dir('tp/appliSpringWeb') {
                                echo 'mvn test'
                                sh 'mvn test'
                                }
                        }
                }
        }
    stage('build_docker_image') {
        steps {
                script{
                        dir('tp/appliSpringWeb') {
                                echo "docker_image_name=" + docker_image_name
                                dockerImage = docker.build(docker_image_name)
                        }
                }
                }
        }
    stage('push_docker_image') {
            steps {
                    script{
                        dir('tp/appliSpringWeb') {
                                echo "docker_registry=" + docker_registry
                                echo "dockerhub_credential_id=" +dockerhub_credential_id
                                docker.withRegistry( docker_registry, dockerhub_credential_id ) {
                                    dockerImage.push()
                                        }
                                }
                                }
                        }
                }
    }
}
```

Après quelques lancements (et éventuels ajustements) :



NB : au sein du script précédent, toutes les parties facultatives script { dir('tp/appliSpringWeb') { …}} ne sont utiles que dans le cas ou le référentiel git ne comporte pas le fichier **pom.xml** à sa racine (comme c'est généralement conseillé) mais dans un sous répertoire (cas moins conseillé où plusieurs projets

cohabitent au sein du même référentiel git) .

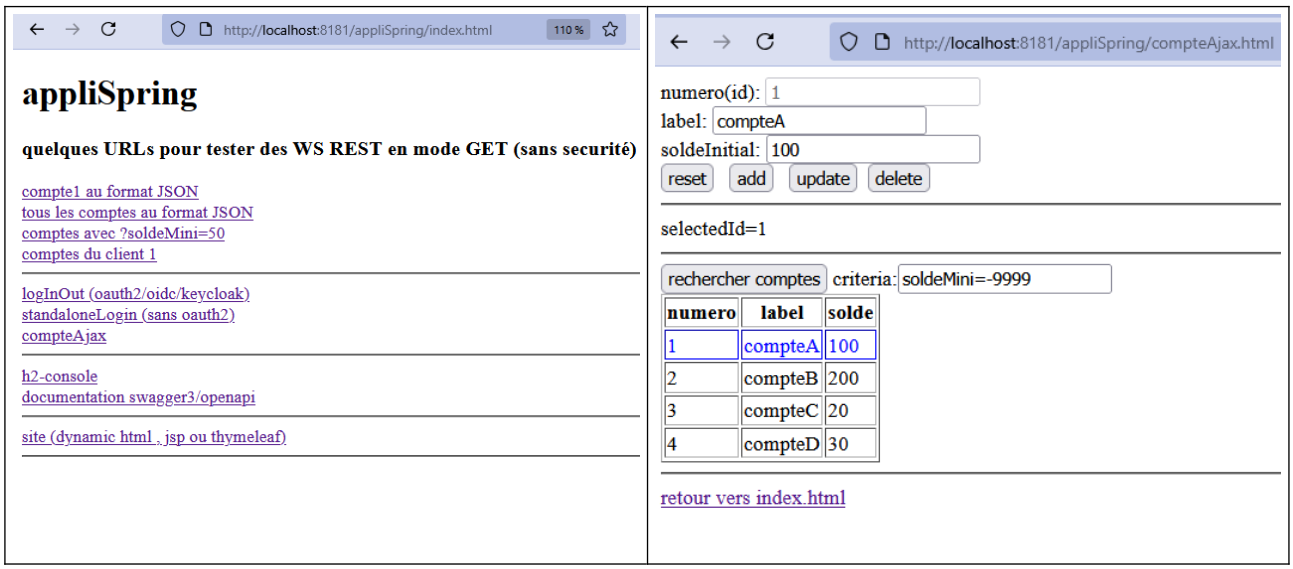
Ce script jenkins va automatiser :

- une récupération du code source de l'application
- une construction de l'application (compilation et construction d'un ".jar" via maven et java21)
- un lancement des tests unitaires (**@Test** dans la partie *src/test/java*)
- une construction d'une image docker
- un déploiement de l'image docker vers le référentiel public dockerhub

Pour vérifier si l'image docker générée fonctionne bien on pourra lancer les commandes suivantes:

- **docker image pull** *didierdefrance69/appli_spring_v3:1*
- **docker run** *--name appli_spring -p 8181:8181* didierdefrance69/appli_spring_v3:1

http://localhost:8181/appliSpring



Ctrl C

- docker stop appli_spring
- docker rm appli_spring

4.4. Pipeline et configuration pour build html/js/cypress

Une application front-end (même élémentaire sans react, angular ou vue) peut être testée via la technologie cypress .
Pour configurer un job jenkins liée à cette application front-end , on pourra commencer par préparer une image docker comportant "nodeJs/javascript + npm + cypress + http-server" de manière à pouvoir construire, lancer et tester l'application web/front-end .

jenkins-dind/build-docker-images/**node_ts_cypress/Dockerfile**

```
FROM node:24

RUN npm install -g typescript && npm install -g http-server && npm install -g start-server-and-test



#NB: xvfb and libglib-2.0.so is a required dependency of cypress
RUN apt-get update && \
  apt-get install --no-install-recommends -y \
  libgtk2.0-0 \
  libgtk-3-0 \
  libnotify-dev \
  libgconf-2-4 \
  libgbm-dev \
  libnss3 \
  libxss1 \
  libasound2 \
  libxtst6 \
  procps \
  xauth \
  xvfb

RUN npm install -g cypress
```

Pour construire localement cette image on pourra lancer une commande de ce genre :

jenkins-dind/build-docker-images/*build_node_ts_cypress_image.sh*

```
cd ./node_ts_cypress
docker image build -t node_ts_cypress .
```

Cette image pourrait éventuellement être déployée sur dockerhub pour être réutilisée ailleurs.

Le **pipeline jenkins** pourra ensuite être basé sur l'utilisation de cette image docker de construction.

jenkins-dind/pipelines/node_js_cypress/**Jenkinsfile**

```
pipeline {
   //agent any
   agent {
      docker {
                    image 'node_ts_cypress'
                   }
    }

    environment{
                    //NB: credential_dockerhub_didierdefrance69 is ID of credential
```

```
                    //prepared in "Admin Jenkins / Credentials / system /global"
                    dockerhub_credential_id='credential_dockerhub_didierdefrance69'

                    //dockerRegistry is dockerhub
                    docker_registry= 'https://registry.hub.docker.com'

                    docker_image_name='didierdefrance69/vanilla_cypress_frontend:1'
        }
  stages {
        stage('from_git') {
        steps {
            git url : 'https://github.com/didier-tp/vanilla_cypress_frontend' , branch : 'main'
        }
    }
     stage('npm_install') {
        steps {
                    sh 'npm install'
        }
    }
        stage('tests') {
        steps {
                    echo 'run ic script of package.json (start-server-and-test(http-server,3000,cypress))'
                    sh 'npm run ic'
        }
    }
     stage('build_docker_image') {
                    steps {
                            script{
                                dockerImage = docker.build(docker_image_name)
                            }
                    }
    }
      stage('push_docker_image') {
        steps {
                    script{
                            docker.withRegistry( docker_registry, dockerhub_credential_id ) {
                                        dockerImage.push()
                                    }
                             }
                     }
        }
   }
}
```

L'application web/front-end à tester (ici https://github.com/didier-tp/vanilla_cypress_frontend) pourra comporter les fichiers de configuration suivants :

vanilla_cypress_frontend/**package.json**

```
{
 "name": "vanilla_cypress_frontend",
 "version": "1.0.0",
 "scripts": {
   "ic" : "start-server-and-test 'http-server -p 3000 --silent' 3000 'npx cypress run --spec
            ./cypress/e2e/myTest.spec.cy.js --reporter json --reporter-options output=test-report.json' "
 }
}
```

vanilla_cypress_frontend/**Dockerfile**

```
FROM node:24
WORKDIR /frontend
COPY . .
RUN npm install -g http-server
EXPOSE 8181
CMD ["http-server","-p","8181" ]
```

Exemple de test cypress :

vanilla_cypress_frontend/cypress/e2e/**myTest.spec.cy**

```
describe('My HTML/JS Tests', () => {
  it('good addition in calculatrice.html', () => {

    //partir de index.html
    cy.visit("http://localhost:3000/index.html")

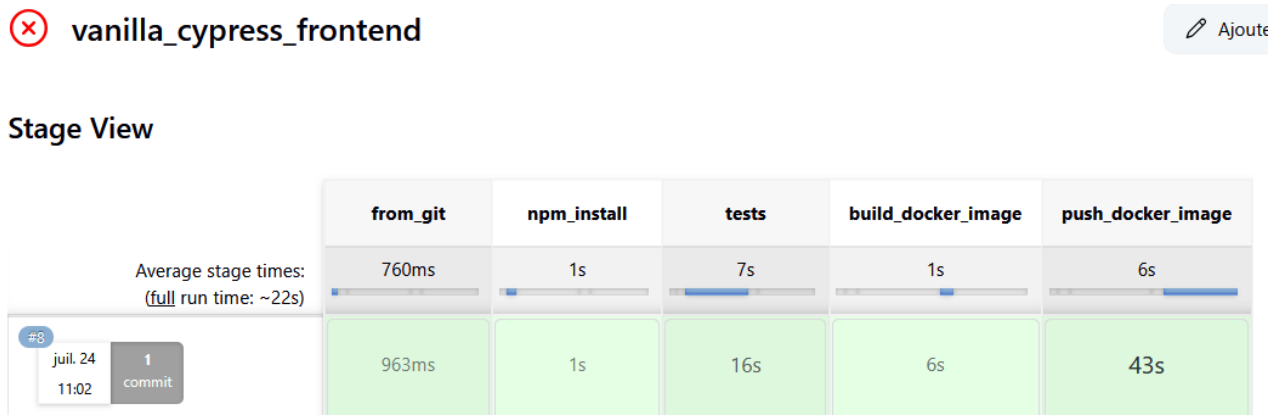
    //cliquer sur le lien comportant 'calculatrice'
    cy.contains('calculatrice').click()
    cy.wait(50)
    // Should be on a new URL which includes '/calculatrice'
    cy.url().should('include', '/calculatrice')

    // Get an input, type data into it
    //and verify that the value has been updated
    cy.get('#a')
      .type('5')
      .should('have.value', '5')

    cy.get('#b')
      .type('6')
      .should('have.value', '6')

    //declencher click sur bouton addition
    cy.get('#btn_op_addition')
      .click()

    //vérifier que la zone d'id spanRes comporte le texte '11'
    cy.get('#spanRes')
      .should('have.text', '11')
  })
})
```

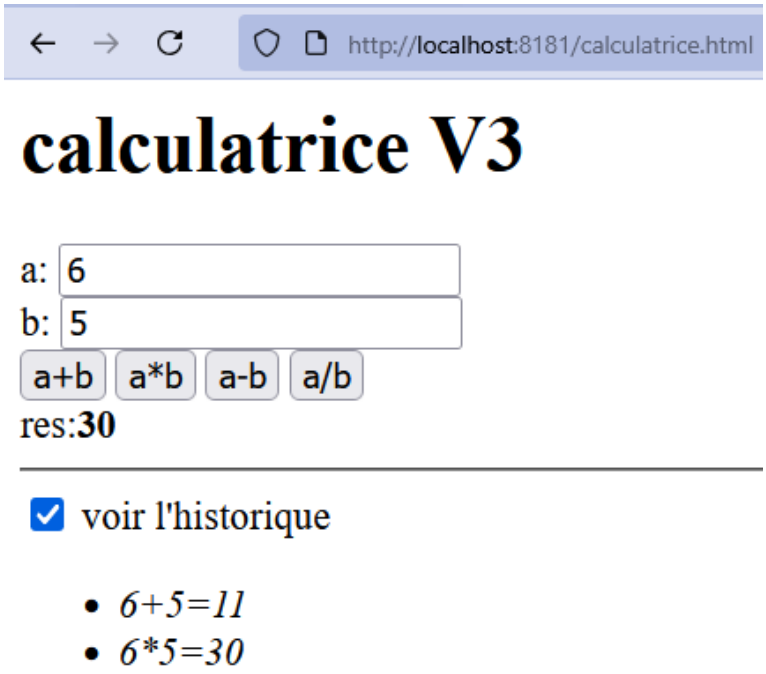


Test de l'image docker construite par le build jenkins "vanilla_cypress_frontend" :

docker image pull didierdefrance69/vanilla_cypress_frontend:1

docker run --name vanilla_frontend -p 8181:8181 didierdefrance69/vanilla_cypress_frontend:1

http://localhost:8181 ou *http://localhost:8181/index.html*



4.5. Pipeline et configuration pour build python

jenkins-dind/build-docker-images/python_venv_pytest/**Dockerfile**

```
FROM python:3.13

ENV VIRTUAL_ENV=/opt/venv
RUN python3 -m venv $VIRTUAL_ENV

#NB: PATH="$VIRTUAL_ENV/bin:$PATH" may be indirectly done via /opt/venv/bin/activate && ...
ENV PATH="$VIRTUAL_ENV/bin:$PATH"

# Install required Python packages in the virtual environment.
RUN pip install pytest

# Install application specific dependencies:
#COPY requirements.txt .
#RUN pip install -r requirements.txt

# Run the application:
#COPY myapp.py .
#CMD ["python", "myapp.py"]
```

jenkins-dind/build-docker-images/*build_python_venv_pytest_image.sh*

```
cd ./python_venv_pytest
docker image build -t python_venv_pytest .
```

jenkins-dind/pipelines/python/**Jenkinsfile**

```
pipeline {
   //agent any
    agent {
       docker {
                    image 'python_venv_pytest'
                   }
    }
     environment{
                   dockerhub_credential_id='credential_dockerhub_didierdefrance69'
                   docker_registry= 'https://registry.hub.docker.com'
                   docker_image_name='didierdefrance69/my_rest_api:1'
    }
   stages {
      stage('from_git') {
         steps {
              git url : 'https://github.com/didier-tp/my_python_rest_api' , branch : 'main'
         }
      }
      stage('build') {
         steps {
            sh 'pip install -r requirements.txt'
            echo 'build'
         }
      }
      stage('tests') {
         steps {
```

```
            sh 'pytest --junit-xml test-reports/results.xml test_devise_api.py'
        }
    }
    stage('build_docker_image') {
            steps {
                script{
                    dockerImage = docker.build(docker_image_name)
                }
            }
    }
    stage('push_docker_image') {
        steps {
            script{
                docker.withRegistry( docker_registry, dockerhub_credential_id ) {
                    dockerImage.push()
                }
            }
        }
    }
}
```

my_python_rest_api

Stage View

	from_git	build	tests	build_docker_image	push_docker_image
Average stage times: (full run time: ~35s)	745ms	8s	1s	13s	14s
#6 juil. 24 14:00 1 commit	941ms	8s	1s	17s	17s

Utilisation de l'image docker construite :
docker image pull didierdefrance69/my_rest_api:1
docker run --name python_rest_api -p 8088:8088 didierdefrance69/my_rest_api:1

http://localhost:8088

devise-api rest

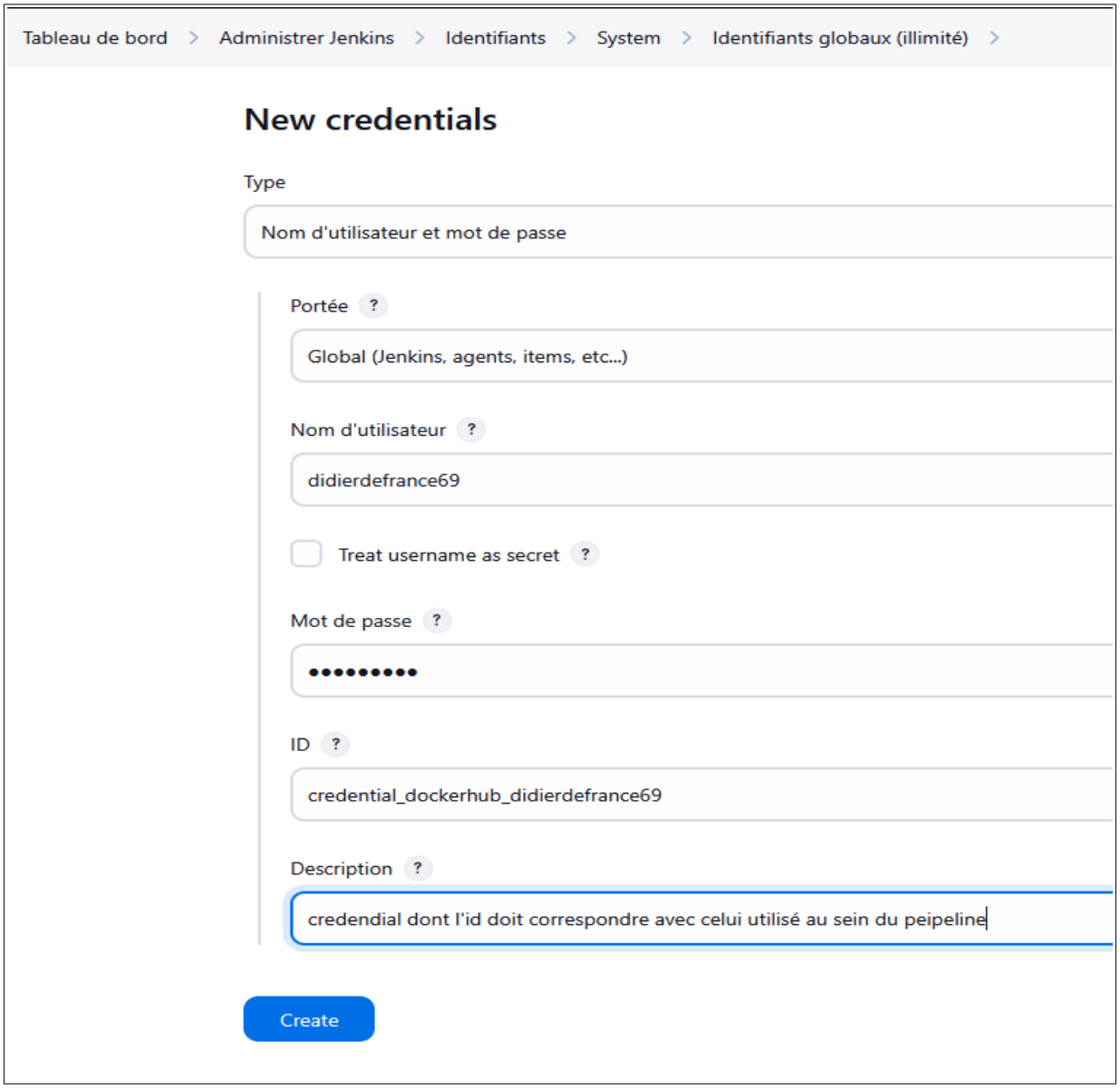
devise EUR
all devises

devise-api docs

index of embedded mini front-end

5. "credentials jenkins" pour "push d'image docker"

NB : la partie "push-docker-image" d'un pipeline jenkins nécessite un compte sur dockerhub (ou bien un autre "docker-registry") et nécessite un paramétrage de ***credential*** de ce genre au niveau de la console d'administration de Jenkins :



NB: l'id du credential (ici *credential_dockerhub_didierdefrance69*) doit correspondre à une partie du pipeline jenkins .

Exemple :

```
environment{
        dockerhub_credential_id='credential_dockerhub_didierdefrance69'
}
…{ … {
docker.withRegistry( docker_registry, dockerhub_credential_id ) {
 dockerImage.push()
 } }….}….
```

6. Notifications des résultats (rss , email , ...)

Notifier les développeurs du résultats des tests (état du projet)

Au niveau d'un logiciel d'intégration continue, on peut souvent paramétrer un mécanisme qui servira à avertir les développeurs sur :

* **résultats des tests** (statistiques , erreurs , …)
* **état du projet (vert , rouge , …)** et **évolution**
* ...

On parle souvent en terme imagé de **météo du projet** .



Les notifications peuvent être effectuées par :
* des **envois d'e-mail** (à partir d' une liste des adresses e-mail des développeurs de l'équipe)
* des "**flux RSS**"
* une **simple consultation régulière du statut du projet dans jenkins**

NB : d'une version à l'autre de Jenkins , certaines présentations/informations peuvent changer.

6.1. Exemple de liens hypertextes sur résultats des builds

Pour une consultation libre depuis un développeur du projet

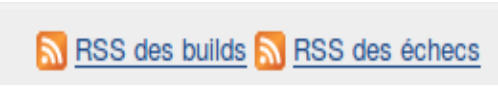
exemple (ici sur job "vanilla_cypress_frontend")



http://localhost:8080/job/vanilla_cypress_frontend/lastBuild/
http://localhost:8080/job/vanilla_cypress_frontend/lastStableBuild/
http://localhost:8080/job/vanilla_cypress_frontend/lastSuccessfulBuild/
http://localhost:8080/job/vanilla_cypress_frontend/lastFailedBuild/

6.2. Notifications par flux rss

Notifications élémentaires par flux rss (jenkins)



Jenkins créer des "**flux RSS" attachés à chaque projet pris en charge**.
Un flux RSS "**tous les builds**" permet d'être averti du résultat de chaque nouveau build .
Un flux RSS "**tous les échecs**" permet de n'être averti qu'en cas d'échec lors d'un build.

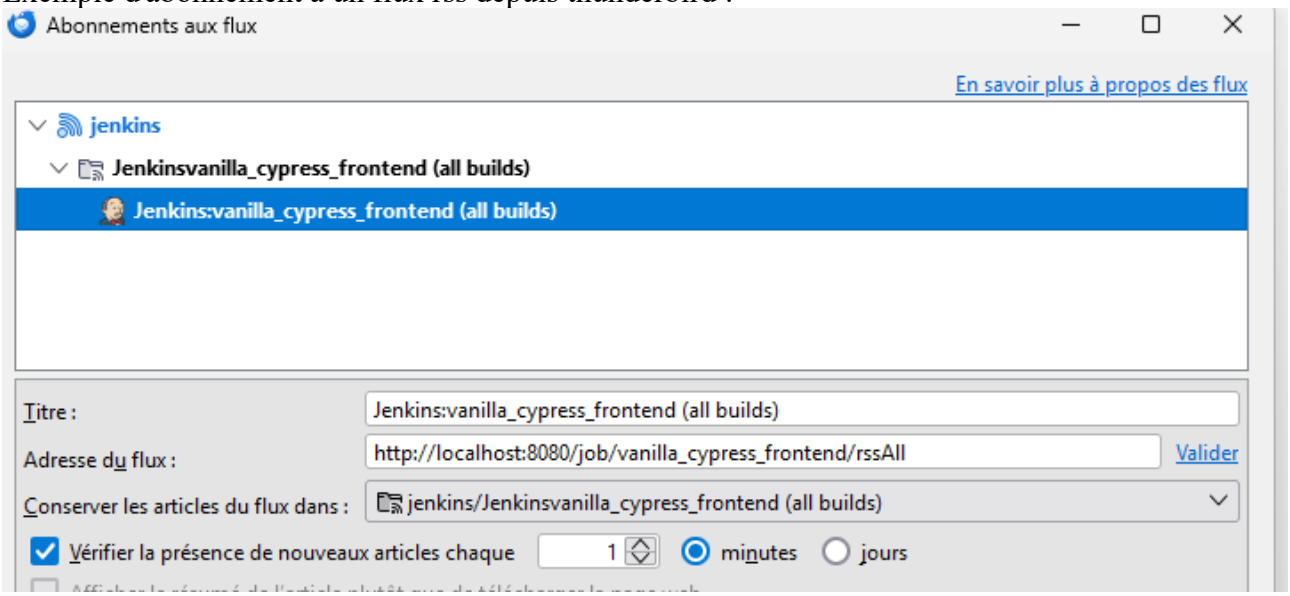
En cliquant sur l'un des icônes "RSS" de l'interface graphique de Jenkins , on peut (via le menu contextuel "copier l'adresse du lien") récupérer l'URL du flux (exemple : http://localhost:8585/jenkins/job/my-java-app1/rssAll) pour ensuite paramétrer un lien dans un navigateur internet (tel que firefox) ou bien dans un logiciel de consultation des emails (tel que ThunderBird ou Oulook).

Pour accrocher un flux rss à *thunderbird*, le mode opératoire est le suivant :
.... / paramètres des comptes / gestions des comptes / nouveau compte de type "blog & news" / gérer les abonnements puis saisir l'adresse du flux et ajouter .

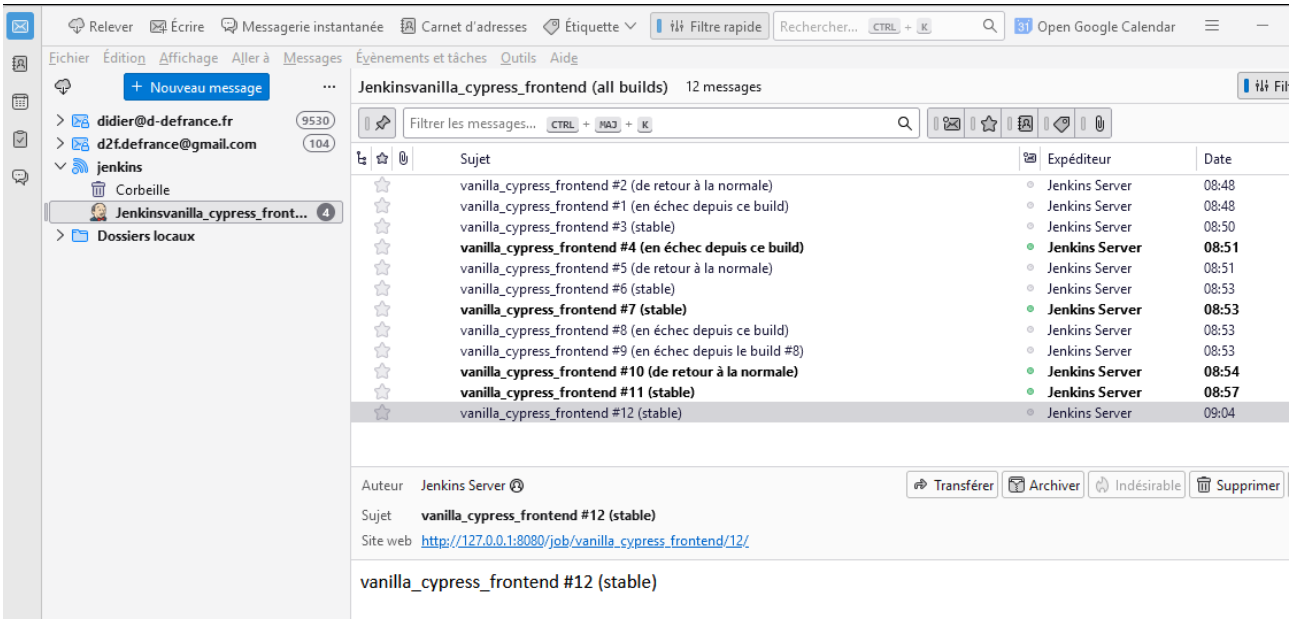
NB : le contenu du flux RSS comporte simplement un message "build … réussi ou en échec" et un lien hypertexte qui renvoie sur la partie "web" de jenkins qui détaille les résultats .

NB : au sein des versions récentes de jenkins , les flux rss sont moins mis en avant mais il sont encore disponibles (via … / Atom feed / …) :
http://localhost:8080/job/vanilla_cypress_frontend/rssFailed
http://localhost:8080/job/vanilla_cypress_frontend/rssAll
http://localhost:8080/job/vanilla_cypress_frontend/rssLatest

Exemple d'abonnement à un flux rss depuis thunderbird :



Exemple de notifications rss jenkins reçues au niveau de thunderbird :



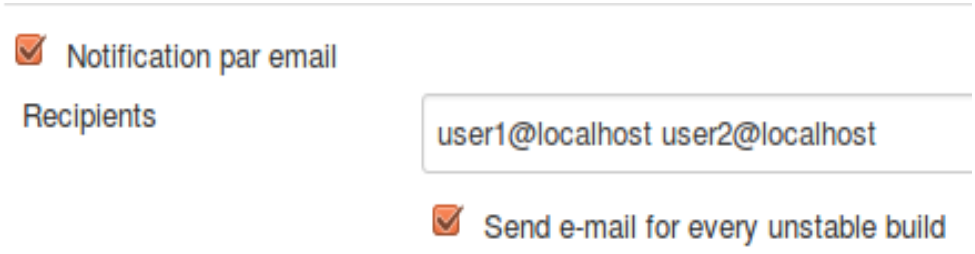
Les messages rss reçus comportent un lien hypertexte vers une partie de la console "jenkins" de manière à obtenir plus de détails

6.3. Notifications par email

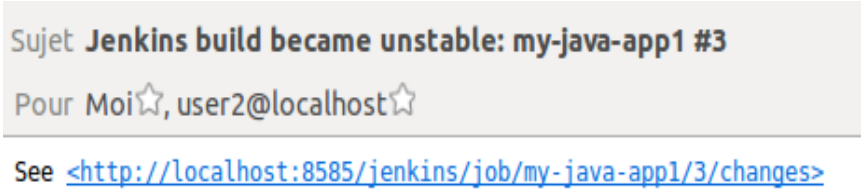
Notification par email (depuis Jenkins)

Pour activer la notification par emails , il faut avoir préalablement paramétrer un lien vers un serveur SMTP depuis la partie "administration" de la console de Jenkins.

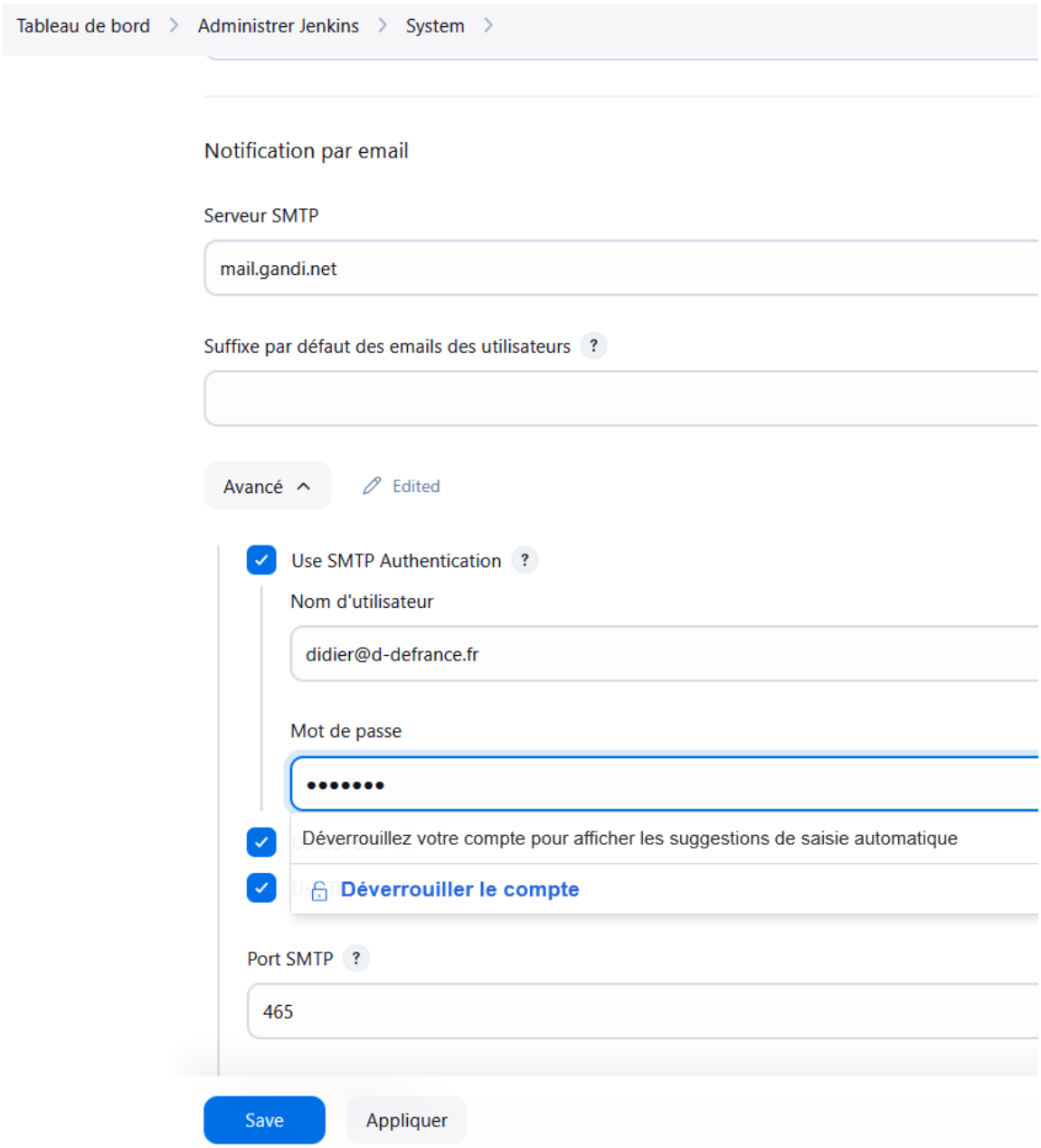
Ensuite, au niveau d'un "**build**" (application à construire), on peut configurer près des "**post-action**" une **notification basique par email** (avertir seulement en cas d' échec) en **précisant une liste d' emails pour les développeurs destinataires** :



e-mail reçu en cas d'échec.



On peut éventuellement installer un plugin jenkins supplémentaire "**email-ext.hpi**" puis s'en servir via la post-action "***Editable Email Notification***" pour contrôler de façon plus fine les notifications envoyées.



Envoi d'un mail depuis une étape d'un pipeline :

```
stage('Send email') {
   def mailRecipients = "your_recipients@company.com"
   def jobName = currentBuild.fullDisplayName

   emailext body: '"${SCRIPT, template="groovy-html.template"}"',
      mimeType: 'text/html',
      subject: "[Jenkins] ${jobName}",
      to: "${mailRecipients}",
      replyTo: "${mailRecipients}",
      recipientProviders: [[$class: 'CulpritsRecipientProvider']]
}
```

7. Différents types de "builds" (avec Jenklins)

Différents types de "builds"

Types de "build"	Commentaires/considérations
Local / privé	**Lancé manuellement** (et idéalement fréquemment) par le développeur (depuis son IDE) . → *Permet de savoir si ses propres changements fonctionnent*
Intégration rapide (de jour)	**Tests d'intégration rapides déclenchés après chaque commit ou bien régulièrement (ex : toutes les 20 minutes)**. Seuls les tests rapides (unitaires + intégrations) sont lancés → *Permet de savoir si l'assemblage des changements de tous les développeur fonctionne .*
Intégration journalière poussée/sophistiquée à heure fixe **(nightly build)**	**Tests sophistiqués (longs)** , tests de performance , génération de documentation, de rapports , …. → *Permet de savoir si la dernière version produite du logiciel est en état de marche .*

Réglages de fréquence via une syntaxe "crontab" (par exemple dans Jenkins) :

Syntaxe "crontab" :

mm hh jj MM JJ *[tâche]*

`mm` représente les minutes (de 0 à 59)
`hh` représente l'heure (de 0 à 23)
`jj` représente le numéro du jour du mois (de 1 à 31)
`MM` représente le numéro du mois (de 1 à 12)
`JJ` représente le numéro du jour dans la semaine
(0 : dimanche , 1 : lundi , 6 : samedi , 7:dimanche)

Si, sur la même ligne, le *« numéro du jour du mois »* et le *« jour de la semaine »* sont renseignés, alors **cron** (ou ….) n'exécutera la `tâche` que quand ceux-ci coïncident .

…/...

Réglage de la fréquence des "builds"

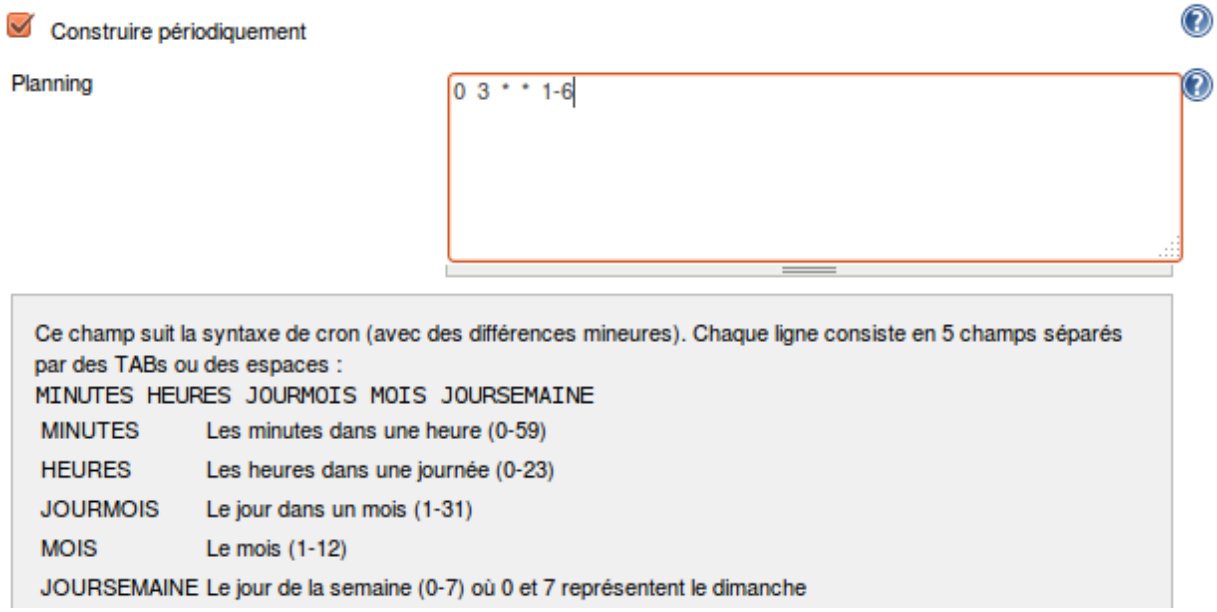
Pour chaque valeur numérique (`mm`, `hh`, `jj`, `MMM`, `JJJ`) les notations possibles sont :

* : à chaque unité (0, 1, 2, 3, 4...)
5,8 : les unités 5 et 8
2-5 : les unités de 2 à 5 (2, 3, 4, 5)
**/3* : toutes les 3 unités (0, 3, 6, 9...)
10-20/3 : toutes les 3 unités, entre la dixième et la vingtième (10, 13, 16, 19)

Et donc pour un nightly build d'intégration continue :
---> **0 3 * * 1-6** *(tous les jours à 3h du matin sauf les dimanches)*

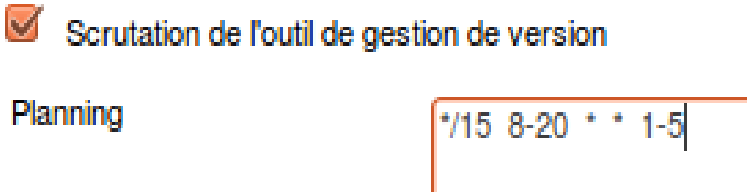
et pour un build rapide de jour :
→ ***/15 8-20 * * 1-5** *(tous les jours sauf les week-ends , toutes les 15 minutes de 8h à 20h)*

Lancement périodique (ex journalier) au niveau de Jenkins



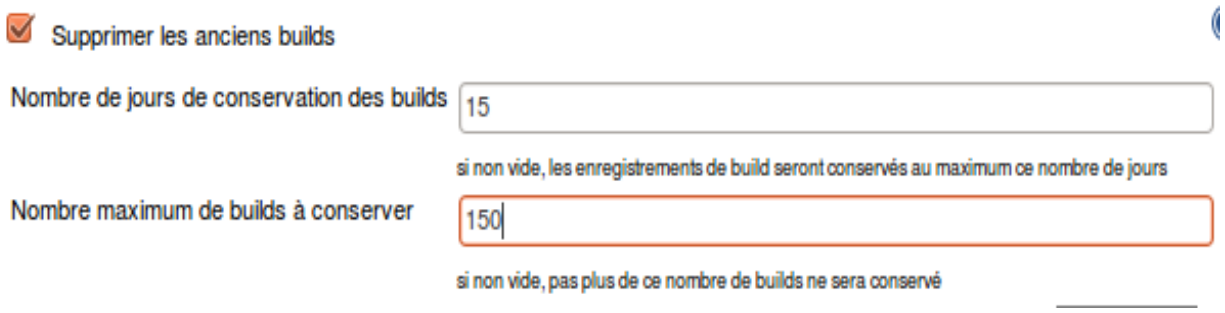
dans cet exemple : tous les jours (de lundi à samedi) à 3h du matin. "nightly build"

Lancement sur détection périodique des changements (SVN/GIT)



dans cet exemple : Jenkins vérifie toutes les 15 minutes si certains changements ont eu lieu au niveau du référentiel de code source (de lundi à vendredi et de 8h à 20h). En cas de changement détecté → lancement (potentiellement très fréquent) d'un build d'intégration.

Suppression des anciens builds (jenkins)



XVII - Annexe – Bibliographie, Liens WEB + TP

1. Bibliographie et liens vers sites "internet"

http://maven.apache.org	Site web officiel sur maven
Livre "Apache Maven" • Auteur(s) : Nicolas De Loof , Arnaud Héritier • Editeur : Pearson Education	Livre en français sur maven

2. TP

Liste de "Tp" progressifs sur maven (selon instruction du formateur) avec:

- scripts (.bat) préparés qui lancent les lignes de commandes "maven" (à étudier et lancer dans l'ordre).
- exemples de code java fournis durant la formation.